

Fording: A Sound Shallow Translation of Indexed Inductive Families into First-Order Logic

AI draft of July 17, 2026

Abstract

Hammer-style proof automation for the Calculus of Inductive Constructions translates goals and library facts into untyped classical first-order logic and runs automated theorem provers on the result. A companion paper does this for a monomorphic fragment with non-indexed datatypes, factoring the translation through program extraction and proving every axiom valid in one proof-irrelevant term model whose domain contains ill-typed “junk”. This paper extends the calculus, translation and soundness proof to *indexed inductive families* — vectors, finite sets, *reflect*-style views, equality-like singletons — by McBride’s *fording*: index constraints become equational payload, stated but never solved at translation time. A family becomes a $(1+m)$ -ary guard predicate; its emitted theory is the first-order image of its ford: constructor typing with index conclusions, guarded inversion carrying constructor payloads and index equations, and unguarded per-constructor match equations. The model interprets a family in stages as a set of (index tuple, element) pairs; an *index-reflection* lemma is the semantic content of fording; every emitted axiom holds, so the translation is sound and the emitted theory consistent. The model arbitrates design questions documented for related encodings: omitting the inversion guard collapses the universe and is *inconsistent* once any two-constructor datatype is declared; index-forgetting guards reproduce the F^* #1542 unsoundness; match equations need no guards even at statically impossible branches, index-premised variants being strictly weaker; per-occurrence expansion of the ford is sound in both polarities and logically inert; index uniqueness is derivable while type-former injectivity is inexpressible, by design; and the singleton collapse is exactly where the model spends uniqueness of identity proofs.

Contents

1 Introduction

3

1.1	The idea: fording	4
1.2	Contributions	5
1.3	What soundness means here	7
1.4	Relation to the implemented translation	8
2	Fording, and what an untyped target changes	9
2.1	Indexed families and their eliminations	9
2.2	Fording	10
2.3	What the untyped target dissolves	11
2.4	What the untyped target does not dissolve	12
3	The source calculus CIC_{ix}^-	13
3.1	Syntax	13
3.2	Environments	14
3.3	Typing	16
3.4	Reduction and conversion	21
3.5	The shallow fragment	23
3.6	Programming with forced case analysis	24
3.7	Relation to CIC	25
4	The intermediate language and program extraction	26
4.1	The calculus $\lambda_{\square}$	27
4.2	Erasure	28
5	The translation	30
5.1	Signature	30
5.2	Compilation of erased programs	31
5.3	Specification extraction	38
5.4	Structural axioms: the ford of a family	39
5.5	Forced-argument simplification	41
6	Semantics	43
6.1	The domain	43
6.2	Interpretation of types, propositions and predicates	44
6.3	The first-order structure $\mathfrak{M}$	47
6.4	Adequacy of the term translation and of compilation	48
6.5	Coincidence: the translation reflects the semantics	51
7	Soundness	52
7.1	Guard disciplines	52
7.2	The fundamental lemma	53
7.3	Validity of the emitted theory	58
7.4	Main theorem and corollaries	60

8	Fording at the level of formulas: design questions settled	62
8.1	Per-occurrence expansion of forded guards	62
8.2	Which guards are load-bearing	65
8.3	Match equations for indexed scrutinees	68
8.4	Indices, elements, and injectivity of type formers	70
8.5	Singleton collapse and uniqueness of identity proofs	71
9	Shallowness	74
10	Related work	75
11	Limitations and future work	78
11.1	Dependent index telescopes	78
11.2	Polymorphism, large indices, large elimination	79
11.3	Well-founded recursion and Acc-style predicates	79
11.4	Acyclicity, induction, completeness	79
11.5	Functional extensionality	80
11.6	Reconstruction	80
11.7	Other restrictions	81
A	Confluence of $\lambda_{\square}$	87

1 Introduction

A *hammer* translates a goal of a proof assistant, together with a selection of library facts, into the input logic of automated theorem provers (ATPs), and reconstructs any proof the ATPs find [BKPU16]. For proof assistants based on the Calculus of Inductive Constructions (CIC) — Rocq (formerly Coq) and its relatives — the reference tool is CoqHammer [CK18, CK26], whose target is untyped classical first-order logic (FOL). The efficiency of such a translation hinges on it being *shallow* [Cza18]: source propositions must become first-order *formulas* whose connectives, quantifiers and equalities are the native first-order ones, so that clausification, superposition and term indexing do real logical work on the translated problem, instead of simulating the object logic through an inhabitation predicate one bridge axiom at a time.

A companion paper [Cza26] carries this programme out for a monomorphic fragment of CIC with *non-indexed* inductive datatypes: the translation of a definition is factored into *program extraction* (the Letouzey-style erasure [Let03, Let04] of the body, compiled to per-constructor rewrite equations) and *specification extraction* (a realizability reading of the type as a first-order formula about the translated constant), and both components are proved sound against a single first-order model: a proof-irrelevant, classical term model over the erasure calculus whose domain deliberately contains

ill-typed “junk” elements. The model decides, uniformly, which guards are semantically mandatory and which may be dropped.

That development stops at a well-marked boundary: its datatypes have no indices. *Indexed inductive families* — length-indexed vectors $\text{vec } A : \text{nat} \rightarrow \text{Set}$, finite sets $\text{fin} : \text{nat} \rightarrow \text{Set}$, bounded numbers, the `ssreflect` view type $\text{reflect } P : \text{bool} \rightarrow \text{Set}$, and the user-declared equality-like and `le`-like predicates — are the workhorses of dependently typed libraries, and they are exactly the point where CIC’s typing discipline diverges most sharply from first-order logic: the type of a constructor *constrains* the indices at which it may appear, case analysis must *unify* the indices of the scrutinee’s type with those of each constructor, and eliminations of propositional families into computational content are governed by the singleton criterion, Streicher’s K, and their refinements [GMM06, CDP14, GCST19]. The present paper extends the companion’s calculus, translation and soundness proof to indexed families, and settles the design questions specific to them.

1.1 The idea: fording

The mechanism is due to McBride [McB99, §3.5], who named it after Henry Ford’s “any colour ... so long as it is black”: a constructor of an indexed family,

$$c : \forall \vec{a} : \vec{A}. I \vec{t}(\vec{a}),$$

is equivalent to a constructor of a *non-indexed* family that accepts arbitrary indices $\vec{u}$ but carries equational premises restoring the lost specificity:

$$c^{\text{Ford}} : \forall \vec{u}. \forall \vec{a} : \vec{A}. (\vec{u} = \vec{t}(\vec{a})) \rightarrow I^{\text{Ford}} \vec{u}.$$

Index constraints thereby become *propositional payload*, on the same footing as the refinement payloads $P x$ of subset types $\{x:A \mid P x\}$, which the companion translation already expands shallowly at every occurrence. Every system that elaborates dependent pattern matching performs this reduction internally — Coq’s *inversion* [CT96, Mon10], the Goguen–McBride–McKinna translation of pattern matching to eliminators (“the equations on the indices are exactly those we must unify” [GMM06]), Agda’s unifier [CDP14, CD18], and the Equations package [SM19]. The present paper performs it in a translation to untyped first-order logic, *per occurrence and per constructor*, and proves the result sound.

Two classical obstacles make fording delicate inside type theory, and both dissolve in an untyped first-order target. First, telescopic index equations relate terms whose *types* are only equal after earlier equations have been substituted; stating them requires heterogeneous (“John Major”) equality, whose eliminator is exactly Streicher’s K [McB99, CDP14]. Untyped first-order equality states them directly. Second, defined functions in index positions — McBride’s *green slime* [McB14] — make elaboration-time

unification stuck: $0 \approx \text{plus}(m, n)$ is neither a positive nor a negative unification success because `plus` is not a rigid symbol. A saturation prover simply *keeps* the equation and reasons with the defining equations of `plus`; specialization by unification becomes ordinary superposition, and branches that a type-theoretic elaborator must leave stuck can be closed by proof. The translation of this paper accordingly never solves index constraints; it only states them.

What does *not* dissolve by itself is soundness. The literature on translations of inductive types into untyped or weakly typed logics is a catalogue of documented accidents precisely at the three points this paper must get right: unguarded constructor-exhaustiveness bounds the cardinality of the whole untyped universe (Meng and Paulson’s two-element universe [MP08, §2.3]; the monotonicity analysis of [BBPS16] identifies exhaustiveness axioms as exactly the sentences that can never lose their guards); encoding a *type-indexed* equality by the native untyped equality of the target lets an equation established at one index leak to another (the F^* unsoundness #1542 [Ase18], where pointwise function equality proved at domain `nat` escaped to domain `int`, yielding `False`); and axioms that are individually plausible — selector totality, rank/height orderings, box/unbox roundtrips — can interact inconsistently on degenerate or empty instances (the Verus unsoundness #1366 [The24b]). Our response to all three is the companion’s discipline, extended: *every* emitted axiom is proved valid in one fixed model, constructed once, so no combination of emitted axioms can be inconsistent (Corollary 7.10), and the model arbitrates which guards are load-bearing. Where a design variant is unsound, we do not merely avoid it: we prove its unsoundness inside the framework, as a specification for implementations (Section 8).

1.2 Contributions

This paper is self-contained: all definitions and proofs are given in full, including the parts shared with the companion paper [Cza26], which are re-developed here in their generalized form. Remaining citations to [Cza26] in proofs indicate provenance; none replaces an argument. Concretely:

1. We define CIC_{ix}^- (Section 3), a stratified, monomorphic, first-order fragment of CIC whose datatypes are *indexed inductive families* with dependent constructor telescopes and propositional constructor premises, subsuming the non-indexed datatypes, refinement types and indexed inductive predicates of the companion fragment. Case analysis on a family is *forded*: each branch binds, besides the constructor’s arguments and premises, proofs of the index equations $\vec{u} = \vec{t}_i(\vec{y}_i)$, so that dependently typed programming idioms (impossible branches, index-directed results) are expressible with first-order motives, without large elimination. The

calculus also has elimination of *index-determined* propositional singletons — the image of CIC’s singleton elimination for equality-like families, with McBride-style forcing in place of CIC’s parameter side condition — and a constructor-discrimination proof former.

2. We define the erasure of CIC_{ix}^- into the untyped calculus $\lambda_{\square}$ and prove the two syntactic pillars of extraction — substitution commutation and simulation of reduction — together with confluence of $\lambda_{\square}$ (Section 4, Appendix A). Erasure is index-blind; in particular proofs occurring *inside index terms* vanish, which is the translation-level form of canonicalizing erased data to a single token [Atk18, Wer08].
3. We define the translation (Section 5). Guard predicates become $(1+m)$ -ary — the guard of an occurrence $I(\vec{u})$ is the atom $\hat{I}(w, \mathcal{C}(\mathcal{E}(\vec{u})))$, carrying the translated indices — and the structural axioms of a family are the first-order image of its ford: constructor typing with index conclusions, and a guarded *inversion* axiom whose disjuncts carry constructor payloads *and index equations*. Pattern-matching definitions compile, unchanged, to unguarded per-constructor equations; the compilation layer itself is the companion’s, with a repair to its treatment of recursive closures (Remark 5.7) that is to be mirrored there. We prove the forced-argument simplification (index equations solvable for a bare variable are redundant) as a first-order equivalence (Lemma 5.17).
4. We construct the model (Section 6): the domain is again the set of closed $\lambda_{\square}$ terms modulo conversion, junk included; an indexed family denotes a set of (index-tuple, element) pairs built in stages. The key new lemma is *index reflection* (Lemma 6.5): every member of the interpretation decomposes uniquely as a constructor form whose index tuple equals the denotation of the constructor’s index terms — the semantic content of fording, and the point where the model validates what uniqueness of identity proofs validates in type theory. Ranks become element ranks, which index reflection makes well defined.
5. We prove soundness (Section 7): every axiom of the emitted theory holds in the model (Theorem 7.8), first-order provability of the translated goal implies truth of the goal in the proof-irrelevant classical semantics (Theorem 7.9), the theory is consistent (Corollary 7.10), and no refutable goal becomes provable (Corollary 7.11). The fundamental lemma (Lemma 7.6) is proved in full; its new cases — forded case analysis, singleton elimination, fixpoints over families — are exactly where index reflection is spent. On premise-free families with first-order arguments the model is standard: $\llbracket \text{vec} \rrbracket$ at index n is in bijection with length- n lists (Proposition 7.12).

6. We settle the indexed-specific design questions (Section 8), each as a theorem about the model or the theory:
 - the inversion axiom’s guard is mandatory in the strongest sense: for *every* indexed family, the unguarded axiom together with the family’s own injectivity and discrimination axioms collapses the universe to a point, so it is inconsistent the moment any two-constructor datatype is declared; for families that specialise an index to a constructor form (`vec`, `fin`), discrimination alone deto-
nates it (Proposition 8.7, exact delimitation in Remark 8.8);
 - the index arguments of guard predicates are mandatory: the index-forgetting variant proves a goal that is false in the source semantics — the precise analogue of $F^* \#1542$ (Proposition 8.9);
 - per-constructor match equations remain valid without any guard or index premise, *including* equations for statically impossible branches; the index-premised variant is strictly weaker and both are sound, so the choice is operational (Propositions 8.11 and 8.12); pruning rigidly clashing branches is sound and prunes only semantically dead equations (Proposition 8.14);
 - per-occurrence inlining of the forced inversion disjunction — the shallow “expansion” of classified occurrences — is sound in both polarities and interderivable with the axiomatic form (Theorem 8.3 and Proposition 8.4);
 - index uniqueness ($\widehat{I}(w, \vec{z}) \wedge \widehat{I}(w, \vec{z}') \rightarrow \vec{z} \approx \vec{z}'$) is a first-order consequence of the emitted axioms, while injectivity of the type former itself is neither expressible in the signature nor emitted — as it must not be, being anti-classical (Proposition 8.16, Remark 8.17);
 - the unconditional singleton collapse is the exact point where the model’s proof irrelevance is spent as uniqueness of identity proofs; Werner’s guarded singleton reduction is the premise-carrying alternative, and both variants are sound (Fact 8.18, Proposition 8.19).
7. We re-verify the shallowness checklist of the companion paper on the extended translation and extend it with an indexed-specific item: index constraints arrive as first-order equations computed at translation time — never as unification problems, never as type-term encodings, never through an inhabitation predicate (Section 9).

1.3 What soundness means here

As in the companion paper, soundness is model-theoretic: if the emitted theory first-order proves the translated goal, the goal is true in a fixed proof-irrelevant, classical semantics of the source development (Theorem 7.9).

This is the property with operational consequences for a hammer — no refutable goal acquires a phantom proof, premise-selection learning is not poisoned, and the emitted theory is consistent — while proof reconstruction inside the proof assistant remains the final arbiter of each individual proof. The semantics validates excluded middle and proof irrelevance by construction (the ATPs are classical; erasure forgets proofs); consequently it also validates uniqueness of identity proofs, and Section 8.5 makes precise where the translation spends that strength and what the corresponding reconstruction obligations are. The converse property, completeness, is deliberately traded away; Section 11 delimits what is lost.

The end-to-end guarantee, stated once. Soundness begins at CIC_{ix}^- . The front end that carves the fragment out of full CIC — stratification, monomorphisation, elaboration of native dependent matches into forded ones, dropping of proof-sorted index positions (Section 3.7) — is part of the pipeline, not of the theorems, and the forded and native case styles are intertranslatable only in CIC extended with UIP [GMM06]. The operational reading above is therefore relative to CIC_{ix}^- : Corollary 7.11 says that no CIC_{ix}^- -refutable goal becomes provable. Composed with the front end, it reads: no goal refutable in the user’s development acquires a phantom proof, *provided* the front end is meaning-preserving — in particular, reflects CIC-refutability into CIC_{ix}^- -refutability — an assumption that silently absorbs UIP on the source side, matching the model’s validation of UIP on the target side (Section 8.5); and proof reconstruction inside the proof assistant remains the final arbiter of every individual proof regardless. Neither the front end nor its meaning preservation is formalised in this paper. This paragraph is the one place where the composed, conditional claim is stated; every theorem in the body is unconditional and starts at CIC_{ix}^- .

1.4 Relation to the implemented translation

CIC_{ix}^- and its translation are an idealisation of a design for CoqHammer’s translation pipeline, not a description of its current code. The current implementation refuses to classify indexed families for erasure-based treatment; declaration-level inversion axioms for inductive predicates already carry index equations, but they are mediated by an inhabitation atom rather than expanded per occurrence; a user-written match on an equality proof receives no definitional axiom at all (only a fixed list of transport constants is special-cased); and per-constructor case equations for indexed matches are emitted without index premises, including for statically impossible branches. Every construction of this paper has a direct implementation site in that pipeline: the forded constructor telescopes of Section 5.4 correspond to classifying the forded form of a family; the expansion theorem of Section 8.1 to per-occurrence guard expansion; the singleton elimination of Section 3

to the erasure of matches on equality-like proofs; and the propositions of Section 8.3 to the choice points (index premises, branch pruning) that the implementation exposes as options. We indicate the correspondences where relevant but keep the development self-contained.

2 Fording, and what an untyped target changes

This section reviews the type-theoretic situation the paper starts from, fixes the reading of *fording* used throughout, and identifies which parts of the classical machinery an untyped first-order target renders unnecessary — and which soundness obligations it creates instead. It contains no formal development; the definitions begin in Section 3.

2.1 Indexed families and their eliminations

An inductive family in the sense of Dybjer [Dyb94] is an inductive definition $I : \vec{\sigma} \rightarrow s$ whose constructors target *instances* $I \vec{t}(\vec{a})$ determined by the constructor’s own arguments. The paradigmatic informative examples are

$$\text{vnil} : \text{vec } A \text{ O}, \quad \text{vcons} : \forall n:\text{nat}. A \rightarrow \text{vec } A \ n \rightarrow \text{vec } A \ (\text{S } n),$$

and the paradigmatic propositional ones are the equality family and `le`. Three features distinguish families from plain datatypes in a way that any translation must confront.

1. *Constructor typing constrains indices.* A constructor inhabits only the instances of the form $I \vec{t}(\vec{a})$; an arbitrary instance $I \vec{u}$ is inhabited by c-forms only when the constraint $\vec{u} = \vec{t}(\vec{a})$ is solvable.
2. *Case analysis specializes indices.* A dependent match on a scrutinee of type $I \vec{u}$ must reconcile $\vec{u}$ with each constructor’s $\vec{t}_i(\vec{y}_i)$. Type-theoretic elaborations do so by first-order unification with constructors as the rigid symbols [GMM06]; branches whose constraints are refutable are discharged, and solvable constraints instantiate the branch.
3. *Propositional families may or may not eliminate into computations.* CIC permits a match on a `Prop`-sorted scrutinee in an informative position only for empty and *singleton* inductives (one constructor, all arguments propositional, parameters aside) [The26, Let03]; the equality family qualifies because its constructor has no arguments beyond parameters. The criterion is known to be simultaneously too strong and too weak [GCST19]: it rejects families whose non-parameter arguments are *forced* by the indices, and accepting it together with proof irrelevance makes uniqueness of identity proofs (UIP) hold.

2.2 Fording

McBride’s thesis [McB99, §3.5] introduces the constraint technique this paper is built on: to use an elimination rule whose motive ranges over the whole “aperture” $I\vec{\tau}$ at a more specific instance, one abstracts the full aperture and *constrains* it with pointwise equations $\vec{\tau} \simeq \vec{u}$ — “the patterns to which it applies are subject to equational constraints which recover their specificity.” Goguen, McBride and McKinna’s elimination of dependent pattern matching [GMM06] runs on the same device: in their specialization by unification, the method for constructor c receives hypotheses $\vec{u} \simeq \vec{t}_c(\vec{y})$ and $c\vec{y} \simeq x$ — “the equations on the indices are exactly those we must unify to allow the instantiation.” At the level of declarations the transformation is folklore, stated compactly by Zhang [Zha23]: a constructor $c : X \rightarrow Fa$ fords to $c^{\text{Ford}} : \forall a'. (a = a') \rightarrow X \rightarrow Fa'$, after which “ F is no longer indexed” — the family has become a *parameterized* type whose constructors carry equational premises, and index specialization has become ordinary propositional reasoning that a consumer may perform whenever convenient.

This paper uses fording in three places, always as a lens and never as a literal re-declaration:

- in the *source calculus*: the branches of a case analysis on $I(\vec{u})$ bind proofs of the index equations $\vec{u} = \vec{t}_i(\vec{y}_i)$ (Figure 2), which is what lets dependently typed idioms — impossible branches, results depending on indices — be written with first-order motives;
- in the *emitted theory*: the inversion axiom of a family quantifies over an arbitrary index tuple and disjoins, per constructor, the payload *and the index equations* (Definition 5.13) — the first-order image of the forded declaration;
- in the *semantics*: the interpretation of a family is a set of (index tuple, element) pairs, and the index-reflection lemma (Lemma 6.5) states that membership holds exactly when a constructor decomposition with matching index denotations exists — fording read as a property of one model.

One point deserves emphasis, because it explains why no “forded re-declaration” appears anywhere in the formal development. Read as a CIC declaration, the forded family has its erstwhile indices as *non-uniform parameters*: recursive constructor arguments still occur at other instantiations (vcons at n , conclusion at Sn). Parameter-versus-index is a property of constructor *conclusions* only, and elimination confronts only conclusions: since every forded constructor returns the generic instance, case analysis needs no index unification, and each branch simply receives its equations as hypotheses. Constraint discharge is pay-as-you-go, one constructor layer per

match. In the untyped first-order output the parameter/index distinction vanishes entirely — there are only function symbols, predicates and equations — which is why fording can remain a per-occurrence, per-constructor lens.

2.3 What the untyped target dissolves

The type-theoretic literature attaches three warning labels to fording and to dependent case analysis generally. All three are analysed precisely in the body of the paper; we preview the outcome.

Heterogeneous equality. Telescopic index equations relate terms whose types agree only after earlier equations are substituted: $v_1 = v_2$ at types $\text{vec } A n_1$ and $\text{vec } A n_2$ cannot even be *stated* homogeneously before $n_1 = n_2$ is substituted. McBride’s heterogeneous $\simeq$ exists to state such equations, and its eliminator is exactly K [McB99, CDP14]. In an untyped target the formation problem does not exist: equality is a single binary predicate on one universe, and telescopic equations are stated pointwise (Definition 5.13). The paper’s source calculus sidesteps the issue differently — its index telescopes are non-dependent (Definition 3.3), so source-level equations are homogeneous — and Section 11 discusses the residual gap.

K, UIP, and the singleton criterion. The without-K analysis of pattern matching [CDP14, CD18] isolates exactly two unification steps that require K: deletion of reflexive equations, and injectivity at indices that are non-linear or not self-unifiable; Gilbert et al. [GCST19] show that eliminating an indexed propositional singleton under proof irrelevance *is* UIP. Every one of these obstructions is a proof-relevance obstruction. The model of Section 6 is proof-irrelevant by construction, hence validates UIP outright, so the *unrestricted* rule set is sound for the translation: deletion, non-linear injectivity and singleton collapse need no side conditions (Section 8.5). What survives of the without-K literature is its *negative* space: discrimination only for fully applied constructor forms (under-applied conflict is refutable under functional extensionality — Counterexample 35 of [CD18]), and no injectivity for type formers (Remark 8.17).

Green slime. Defined functions in index positions make unification-based elaboration stuck [McB14]: $0 \simeq \text{plus } m n$ is neither a clash nor a solution. The standard type-theoretic workarounds (ford the offending index, relational re-expression, derived eliminators) all amount to *keeping the equation as data*. The translation does exactly that, uniformly, for every index: the emitted equation $z \approx \widehat{\text{plus}}(m, n)$ sits in an inversion disjunct or a guard formula next to the defining equations of $\widehat{\text{plus}}$, and a saturation prover performs

specialization by unification as ordinary equational reasoning — including closing branches that elaboration-time unification cannot (a refutation of $0 \approx \text{plus}(m, n)$ under $m \approx S(k)$ is one paramodulation step away from the defining equations). Nothing in the soundness proof distinguishes rigid from non-rigid index terms; decidability is neither claimed nor needed (Remark 5.19).

2.4 What the untyped target does not dissolve

Three families of documented accidents mark the soundness obligations that an untyped target *creates*; the formal development treats each as a theorem rather than a guideline.

First, *exhaustiveness*: in a one-universe target, “every x is a constructor form” bounds the cardinality of the whole universe [MP08, §2.3], and the monotonicity analysis of [BBPS16] shows such sentences are exactly the ones whose guards can never be erased. For indexed families the situation is strictly worse: an *unguarded* inversion axiom is not merely false in the intended model — its index equations force any two index tuples of the same element to coincide, which for arbitrary tuples collapses the universe to a point, so the axiom is inconsistent with the family’s own injectivity and discrimination axioms as soon as the environment declares *any* two-constructor datatype (Proposition 8.7). For families that specialise an index to a constructor form — vectors, finite sets, and their relatives — discrimination alone detonates it, and enumeration-shaped families detonate by cardinality as well (Remark 8.8).

Second, *index erasure in atoms*: a type-indexed notion encoded by an index-blind target notion lets facts established at one index leak to another. The F* unsoundness #1542 [Ase18] is the documented instance — extensional function equality at domain `nat`, encoded into the SMT solver’s untyped `=`, leaked to domain `int` and proved **False**. The analogue inside this paper’s design space is a guard predicate that forgets its index arguments; Proposition 8.9 shows the resulting translation proves a semantically false goal. The translation of Section 5 therefore keeps the indices as arguments of every guard atom, and its native untyped $\approx$ is sound for a different reason than F*’s was unsound: the *semantics of source equality* in the model is itself untyped conversion-class equality, so no typed reading exists to be violated — at the documented price that the model is intensional and functional extensionality is not validated (Remark 8.20).

Third, *degenerate instances*: axioms asserting existence or structure at empty instances — selector totality, box/unbox roundtrips, rank orderings — have produced inconsistencies in deployed systems (Verus #1366 [The24b]). The translation emits no rank axioms, no selectors and no unboxing roundtrips; at an empty instance the guard atom is simply false in the model and every structural axiom is vacuously satisfied there, while refu-

tations of inhabitation (“ $\text{fin}(0)$ is empty”) become one-step consequences of the guarded inversion axiom and discrimination (Example 5.15).

3 The source calculus CIC_{ix}^-

This section defines the source calculus: a monomorphic, first-order fragment of the Calculus of Inductive Constructions whose inductive datatypes are *indexed families*. As in the companion paper [Cza26], the presentation is *stratified*: set types, propositions, predicate kinds, terms and proofs are separate syntactic categories, so the outcome of CIC’s sort analysis — which subterms are computational, which are logical — is built into the grammar, and the erasure of Section 4 and the translation of Section 5 are definable by structural recursion on raw syntax, with no typing oracle. The fragment of [Cza26] is literally the sub-calculus in which every datatype has zero indices and premise-free constructors with closed argument types; Section 3.7 discusses how the extended fragment is carved out of full CIC.

3.1 Syntax

We fix countably infinite, pairwise disjoint sets of *term variables* (written $x, y, z, w, f, \dots$), of *proof variables* ($p, q, h, \dots$) and of *predicate variables* ($X, \dots$), and disjoint sets of names for *datatypes* (I), for *datatype constructors* (c, d), for *inductive predicates* (J), for their *clause constructors* (k), and for *constants* (c).

Definition 3.1 (Syntax of CIC_{ix}^-). The syntactic categories of *set types* σ, τ , *propositions* φ, ψ , *predicate kinds* K , *predicate expressions* P , *terms* t, a, b, u, v and *proofs* π are defined by the following grammar.

$$\begin{aligned}
\sigma, \tau &::= I(\vec{a}) \mid \Pi x:\sigma. \tau \mid \varphi \rightarrow \tau \mid \{x:\sigma \mid \varphi\} \\
\varphi, \psi &::= \top \mid \perp \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \forall x:\sigma. \varphi \mid \exists x:\sigma. \varphi \\
&\quad \mid \forall X:K. \varphi \mid a =_{\sigma} b \mid J(\vec{a}) \mid X(\vec{a}) \\
K &::= (\sigma_1, \dots, \sigma_n) \rightarrow \text{Prop} \quad (n \geq 1) \\
P &::= X \mid \lambda(x_1:\sigma_1, \dots, x_n:\sigma_n). \varphi \\
t, a, b &::= x \mid c \mid t a \mid t \pi \mid \lambda x:\sigma. t \mid \lambda p:\varphi. t \mid \text{c}(\vec{a}; \vec{\pi}) \\
&\quad \mid \text{case}(a; \vec{z}.z.\tau; \vec{br}) \mid \text{fix } f:T. t \mid \text{refine}(t, \pi) \mid \text{val}(t) \\
&\quad \mid \text{split}(\pi; p_1 p_2. t) \mid \text{cast}(\pi; z.\tau; t) \mid \text{scase}_J(\pi; \vec{z}.\tau; \vec{p}.t) \mid \text{efq}(\pi; \sigma) \\
\pi &::= p \mid c \mid \lambda p:\varphi. \pi \mid \pi \pi' \mid \lambda x:\sigma. \pi \mid \pi a \mid \Lambda X:K. \pi \mid \pi P \mid \langle \pi_1, \pi_2 \rangle \mid \pi.1 \mid \pi.2 \\
&\quad \mid \iota_1(\pi) \mid \iota_2(\pi) \mid \text{vcase}(\pi; p_1.\pi_1; p_2.\pi_2) \mid \langle a, \pi \rangle_{\exists x:\sigma. \varphi} \mid \text{ecase}(\pi; x p. \pi') \\
&\quad \mid \text{tt} \mid \text{efq}(\pi; \varphi) \mid \text{refl}_{\sigma}(a) \mid \text{rew}(\pi; z.\psi; \pi') \mid \text{discr}(\pi) \mid \text{k}(\vec{a}; \vec{\pi}) \\
&\quad \mid \text{ind}_J(\vec{z}.\psi; \vec{\pi}; \pi) \mid \text{ind}_I(\vec{z}.z.\psi; \vec{\Pi}; a) \mid \text{valid}(t)
\end{aligned}$$

where:

- a branch list $\vec{br}$ has the form $\{c_1(\vec{y}_1; \vec{p}_1; \vec{q}_1) \Rightarrow t_1 \mid \dots \mid c_M(\vec{y}_M; \vec{p}_M; \vec{q}_M) \Rightarrow t_M\}$ with one branch for every constructor of the datatype concerned; in the branch for c_i , the term variables $\vec{y}_i$ bind the constructor's informative arguments, the proof variables $\vec{p}_i$ its propositional premises, and the proof variables $\vec{q}_i$ (one per index of the datatype) the *index equations* of the ford;
- in $\text{case}(a; \vec{z}.z.\tau; \vec{br})$ the term variables $\vec{z}$ (one per index) and z bind into the motive τ ; in $\text{scase}_J(\pi; \vec{z}.\tau; \vec{p}.t)$ the variables $\vec{z}$ (one per index of J) bind into τ and the proof variables $\vec{p}$ bind into t ; in ind_I each clause proof Π_i has the form $\vec{y}_i \vec{p}_i \vec{h}_i. \pi_i$, binding the constructor telescope, its premises, and one induction hypothesis per recursive argument;
- T in $\text{fix } f:T. t$ ranges over set types; $\{x:\sigma \mid \varphi\}$ binds x in φ ; the binding structure of the remaining forms is the evident one.

Terms of the form $\text{refine}(t, \pi)$ carry a refinement-type annotation, and $\langle a, \pi \rangle$ carries its $\exists$ -type; we omit the annotations when irrelevant. We write $\neg\varphi$ for $\varphi \rightarrow \perp$ and $\varphi \leftrightarrow \psi$ for $(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$. All categories are identified up to α -equivalence; substitution is capture-avoiding.

Notation 3.2. For an m -indexed datatype or predicate we abbreviate index tuples as $\vec{u} = u_1, \dots, u_m$, and write $\vec{u} =_{\vec{\sigma}} \vec{v}$ for the list of propositions $u_1 =_{\sigma_1} v_1, \dots, u_m =_{\sigma_m} v_m$; a proof-variable telescope $\vec{q} : (\vec{u} =_{\vec{\sigma}} \vec{v})$ declares $q_h : u_h =_{\sigma_h} v_h$ for $1 \leq h \leq m$. When a constructor has no premises and the datatype no indices we write $c(\vec{a})$ for $c(\vec{a};)$ and $c(\vec{y}) \Rightarrow t$ for a branch, recovering the notation of [Cza26].

Compared with the companion calculus, the new material is: datatype occurrences $I(\vec{a})$ applied to *index terms*; constructor terms $c(\vec{a}; \vec{\pi})$ carrying proofs of the constructor's propositional premises; the *forded* case former, whose branches bind index-equation proofs $\vec{q}_i$; the singleton elimination $\text{scase}_J(; ;)$ for index-determined propositional singletons (Definition 3.7); the discrimination proof former discr ; and the generalized datatype induction ind_I . The three function spaces, the refinement formers, the propositional singleton eliminations $\text{split} (;)$, $\text{cast} (; ;)$, $\text{efq} (;)$ and the proof-level logic are unchanged. Dependency of types on terms now enters through *three* channels: refinement payloads, propositions embedded in types, and index positions of datatype occurrences. There are still no type-level functions, no universes and no large elimination, so set types remain *rigid* (Lemma 3.12).

3.2 Environments

Definition 3.3 (Declarations and environments). A *global environment* $\mathbb{E}$ is a finite list of declarations of the following three kinds.

1. A *datatype declaration*

$$\text{data } I : (\sigma_1, \dots, \sigma_m) \rightarrow \text{Set} := \{ c_1 : K_1; \dots; c_M : K_M \}$$

with $m \geq 0$ *indices* and $M \geq 1$ constructors, where the index types σ_h are closed set types over the preceding environment, and each *constructor scheme* K_i has the form

$$K_i = \forall(y_1:A_{i1}) \cdots (y_{r_i}:A_{ir_i}). \varphi_{i1} \Rightarrow \cdots \Rightarrow \varphi_{il_i} \Rightarrow I(t_{i1}, \dots, t_{im})$$

subject to: each *argument type* A_{ij} is either a *recursive occurrence* $I(\vec{v})$ with $\vec{v}$ terms over the preceding environment and $y_1, \dots, y_{j-1}$ (of the index types, in the sense of Figure 1), or a set type over the preceding environment and $y_1, \dots, y_{j-1}$ in which I does not occur; each *premise* φ_{il} is a proposition over the preceding environment and $\vec{y}_i$ in which I does not occur; and each *index term* t_{ih} is a term over the preceding environment and $\vec{y}_i$ with $t_{ih} : \sigma_h$. We call $\vec{t}_i = t_{i1}, \dots, t_{im}$ the *index pattern* of c_i and say the argument position j is *recursive* if A_{ij} is a recursive occurrence.

2. An *inductive-predicate declaration* $\text{pred } J : (\vec{\sigma}) \rightarrow \text{Prop}$ with a finite list of clauses

$$k_j : \forall \vec{x}_j : \vec{\tau}_j. \varphi_{j1} \rightarrow \cdots \rightarrow \varphi_{j\ell_j} \rightarrow J(\vec{u}_j)$$

where the index types $\vec{\sigma}$ are closed set types over the preceding environment, the $\vec{\tau}_j$ are set types over the preceding environment (possibly mentioning $\vec{x}_j$ bound earlier in the same telescope) in which J does not occur, the indices $\vec{u}_j$ are terms over $\vec{x}_j$ with $u_{jh} : \sigma_h$, and each premise φ_{ji} is either of the form $J(\vec{v})$ (a *recursive premise*, $\vec{v}$ terms over the telescope of the index types) or a proposition over the preceding environment and $\vec{x}_j$ in which J does not occur.

3. A *definition*, either a *term definition* $c := t : \sigma$ with σ a closed set type and t a closed term, or a *lemma* $c := \pi : \varphi$ with φ a closed proposition and π a closed proof.

All names declared in $\mathbb{E}$ are distinct. An environment in which every constant has a body is *definitional*; this paper considers definitional environments (see Remark 7.13 for postulates). A datatype with $m = 0$ and premise-free constructors with closed argument types is exactly a datatype of [Cza26]; we say I is a *family* when $m > 0$.

Example 3.4 (Running declarations). Fix nat with $\text{O} : \text{nat}$ -constructors as in [Cza26] (formally: $\text{data nat} : () \rightarrow \text{Set} := \{ \text{O} : \text{nat}; \text{S} : \forall y : \text{nat}. \text{nat} \}$, where we write nat for the 0-index occurrence $\text{nat}()$), bool likewise, and a datatype

A (the element type, e.g. nat). The running families are:

```

data vec : (nat) → Set :=
  { vnil : vec(O); vcons : ∀(n:nat)(a:A)(v:vec(n)). vec(S n) }
data fin : (nat) → Set :=
  { fz : ∀n:nat. fin(S n); fs : ∀(n:nat)(k:fin(n)). fin(S n) }
data bnd : (nat) → Set := { B : ∀(n:nat)(k:nat). It(k, n) ⇒ bnd(n) }
data rflφ0 : (bool) → Set :=
  { RT : φ0 ⇒ rflφ0(true); RF : ¬φ0 ⇒ rflφ0(false) }
pred le : (nat, nat) → Prop :=
  { len : ∀n:nat. le(n, n);
    leS : ∀(n m:nat). le(n, m) → le(n, S m) }
pred eqσ : (σ, σ) → Prop := { refleq : ∀x:σ. eqσ(x, x) }

```

where It is some previously declared inductive predicate and φ_0 a closed proposition. vec and fin are informative recursive families; bnd is a subset-shaped family (one constructor, informative carrier, propositional payload, index); rfl_{φ_0} is an enumeration-shaped family (constant constructors, propositional payloads, distinct rigid index patterns) — the monomorphic image of ssreflect 's $\text{reflect } P \ b$ at $P = \varphi_0$; le is a predicate family; eq_σ is a user-declared equality — the primitive $=_\sigma$ remains available, and Example 5.16 compares the two.

Remark 3.5 (Function-indexed families). The index types σ_h of Definition 3.3 are arbitrary closed set types — including Π -types, so families indexed by *functions* are in the fragment. This is deliberate and sound: index equality means conversion-class equality in the model (Definition 6.3), and nothing in the development inspects the shape of index types. It deserves a warning, though, because function indices are exactly where the intensionality of that equality bites hardest (Remark 8.20): the instances $I(\lambda x:\text{nat}.\text{add } 0 \ x)$ and $I(\lambda x:\text{nat}.x)$ have distinct index classes, their interpretations are disjoint (Lemma 6.5(3)), and no emitted axiom relates them — so goals that transport members between extensionally equal but non-convertible indices are soundly, and irremediably within this model, out of reach.

Definition 3.6 (Contexts). A *context* Γ is a finite list of declarations $x : \sigma$, $p : \varphi$ and $X : K$ with pairwise distinct variables, each type, proposition or kind well formed over the preceding declarations (Figure 1).

3.3 Typing

The judgements are: $\vdash \mathbb{E} \text{ wf}$; and, relative to a fixed well-formed $\mathbb{E}$ left implicit, $\Gamma \vdash \sigma \text{ type}$, $\Gamma \vdash \varphi \text{ prop}$, $\Gamma \vdash P : K$, $\Gamma \vdash t : \sigma$, and $\Gamma \vdash \pi : \varphi$. The

$$\begin{array}{c}
\frac{I : (\vec{\sigma}) \rightarrow \text{Set} \in \mathbb{E} \quad \Gamma \vdash \vec{a} : \vec{\sigma}}{\Gamma \vdash I(\vec{a}) \text{ type}} \quad \frac{\Gamma \vdash \sigma \text{ type} \quad \Gamma, x:\sigma \vdash \tau \text{ type}}{\Gamma \vdash \Pi x:\sigma. \tau \text{ type}} \\
\frac{\Gamma \vdash \varphi \text{ prop} \quad \Gamma \vdash \tau \text{ type}}{\Gamma \vdash \varphi \rightarrow \tau \text{ type}} \quad \frac{\Gamma \vdash \sigma \text{ type} \quad \Gamma, x:\sigma \vdash \varphi \text{ prop}}{\Gamma \vdash \{x:\sigma \mid \varphi\} \text{ type}} \\
\frac{\Gamma \vdash \varphi \text{ prop} \quad \Gamma \vdash \psi \text{ prop}}{\Gamma \vdash \varphi \otimes \psi \text{ prop}} \quad \frac{}{\Gamma \vdash \varphi \otimes \psi \text{ prop}} \quad \otimes \in \{\wedge, \vee, \rightarrow\} \\
\frac{\Gamma \vdash \top \text{ prop} \quad \Gamma \vdash \perp \text{ prop}}{\Gamma \vdash \sigma \text{ type} \quad \Gamma, x:\sigma \vdash \varphi \text{ prop}} \quad \frac{}{\Gamma \vdash \sigma \text{ type} \quad \Gamma, x:\sigma \vdash \varphi \text{ prop}} \quad Q \in \{\forall, \exists\} \quad \frac{\Gamma, X:K \vdash \varphi \text{ prop} \quad \Gamma \vdash K \text{ kind}}{\Gamma \vdash \forall X:K. \varphi \text{ prop}} \\
\frac{\Gamma \vdash a : \sigma \quad \Gamma \vdash b : \sigma}{\Gamma \vdash a =_{\sigma} b \text{ prop}} \quad \frac{J : (\vec{\sigma}) \rightarrow \text{Prop} \in \mathbb{E} \quad \Gamma \vdash \vec{a} : \vec{\sigma}}{\Gamma \vdash J(\vec{a}) \text{ prop}} \\
\frac{X : (\vec{\sigma}) \rightarrow \text{Prop} \in \Gamma \quad \Gamma \vdash \vec{a} : \vec{\sigma}}{\Gamma \vdash X(\vec{a}) \text{ prop}} \quad \frac{\Gamma \vdash \sigma_i \text{ type} \ (1 \leq i \leq n)}{\Gamma \vdash (\sigma_1, \dots, \sigma_n) \rightarrow \text{Prop} \text{ kind}} \\
\frac{X : K \in \Gamma}{\Gamma \vdash X : K} \quad \frac{\Gamma, \vec{x}:\vec{\sigma} \vdash \varphi \text{ prop}}{\Gamma \vdash \lambda(\vec{x}:\vec{\sigma}). \varphi : (\vec{\sigma}) \rightarrow \text{Prop}}
\end{array}$$

Figure 1: Well-formedness of set types, propositions, kinds and predicate expressions. $\Gamma \vdash \vec{a} : \vec{\sigma}$ abbreviates $\Gamma \vdash a_h : \sigma_h$ for all h ; since index types are closed, no substitution occurs in index telescopes.

rules are given in Figures 1 to 5. Environment well-formedness requires each declaration to be well formed over its prefix: for a datatype declaration, each index type must satisfy $\vdash \sigma_h \text{ type}$, and for each constructor scheme, working in the telescope $y_1 : A_{i1}, \dots$ extended left to right, each argument type must satisfy $\Gamma_j \vdash A_{ij} \text{ type}$ (where for a recursive occurrence $I(\vec{v})$ this means $\Gamma_j \vdash v_h : \sigma_h$ for all h , the formation rule for I -occurrences being available at this point for the declaration being processed), each premise $\Gamma_{r_i} \vdash \varphi_{il} \text{ prop}$, and each index term $\Gamma_{r_i} \vdash t_{ih} : \sigma_h$; predicate declarations are analogous; a definition $c := t : \sigma$ (resp. $c := \pi : \varphi$) requires $\vdash \sigma \text{ type}$ and $\vdash t : \sigma$ (resp. $\vdash \varphi \text{ prop}$ and $\vdash \pi : \varphi$) over the prefix.

Definition 3.7 (Forced singletons). An inductive predicate $J : (\vec{\sigma}) \rightarrow \text{Prop}$ is a *forced singleton* if it has exactly one clause

$$k : \forall \vec{x}:\vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow J(\vec{u}^0)$$

and there is a function $j : \{1, \dots, |\vec{x}|\} \rightarrow \{1, \dots, m\}$, called a *solution*, with $u_{j(i)}^0 = x_i$ (syntactic identity) for every i . Note that j is automatically injective ($u_{j(i)}^0$ determines x_i), that repeated variables among other index positions are permitted (non-linear index patterns), and that the remaining index positions may hold arbitrary terms. For an occurrence $J(\vec{u})$ the induced *solution substitution* is $\theta = [u_{j(1)}/x_1, \dots, u_{j(|\vec{x}|)}/x_{|\vec{x}|}]$. In the $\text{scase}_J(; ;)$ rule of Figure 3, $\theta(\vec{\varphi})$ and $\theta(\vec{u}^0)$ denote the simultaneous application of θ . When the elimination is used we consider j fixed once per declaration (any choice works; Remark 3.9).

$$\begin{array}{c}
\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \quad \frac{(c := t : \sigma) \in \mathbb{E}}{\Gamma \vdash c : \sigma} \quad \frac{\Gamma \vdash t : \Pi x : \sigma. \tau \quad \Gamma \vdash a : \sigma}{\Gamma \vdash ta : \tau[a/x]} \\
\frac{\Gamma \vdash t : \varphi \rightarrow \tau \quad \Gamma \vdash \pi : \varphi}{\Gamma \vdash t\pi : \tau} \quad \frac{\Gamma, x : \sigma \vdash t : \tau}{\Gamma \vdash \lambda x : \sigma. t : \Pi x : \sigma. \tau} \quad \frac{\Gamma, p : \varphi \vdash t : \tau}{\Gamma \vdash \lambda p : \varphi. t : \varphi \rightarrow \tau} \\
\frac{\begin{array}{c} \mathbf{c}_i : \forall \vec{y} : \vec{A}. \varphi_1 \Rightarrow \dots \Rightarrow \varphi_\ell \Rightarrow I(\vec{t}) \in I \\ \Gamma \vdash a_j : A_j[\vec{a}_{<j}/\vec{y}_{<j}] \ (1 \leq j \leq r) \\ \Gamma \vdash \pi_l : \varphi_l[\vec{a}/\vec{y}] \ (1 \leq l \leq \ell) \end{array}}{\Gamma \vdash \mathbf{c}_i(\vec{a}; \vec{\pi}) : I(\vec{t}[\vec{a}/\vec{y}])} \quad \frac{\Gamma \vdash t : \{x : \sigma \mid \varphi\}}{\Gamma \vdash \text{val}(t) : \sigma} \\
\frac{\begin{array}{c} \Gamma \vdash a : I(\vec{u}) \quad \Gamma, \vec{z} : \vec{\sigma}, z : I(\vec{z}) \vdash \tau \text{ type} \\ \Gamma, \vec{y}_i : \vec{A}_i, \vec{p}_i : \vec{\varphi}_i, \vec{q}_i : (\vec{u} =_{\vec{\sigma}} \vec{t}_i) \vdash t_i : \tau[\vec{t}_i/\vec{z}, \mathbf{c}_i(\vec{y}_i; \vec{p}_i)/z] \quad (\text{all } i) \end{array}}{\Gamma \vdash \text{case}(a; \vec{z}. z. \tau; \vec{b}\vec{r}) : \tau[\vec{u}/\vec{z}, a/z]} \\
\frac{\Gamma \vdash \{x : \sigma \mid \varphi\} \text{ type} \quad \Gamma \vdash t : \sigma \quad \Gamma \vdash \pi : \varphi[t/x]}{\Gamma \vdash \text{refine}(t, \pi) : \{x : \sigma \mid \varphi\}} \quad \frac{\Gamma \vdash \pi : \perp \quad \Gamma \vdash \sigma \text{ type}}{\Gamma \vdash \text{efq}(\pi; \sigma) : \sigma}
\end{array}$$

Figure 2: Typing of terms, part I. In the constructor rule, argument types are instantiated left to right and premises at the full argument list. In the case rule, $\vec{b}\vec{r}$ abbreviates the branch list $\{\mathbf{c}_i(\vec{y}_i; \vec{p}_i; \vec{q}_i) \Rightarrow t_i\}_i$, and $\vec{u} =_{\vec{\sigma}} \vec{t}_i$ are the index equations of the ford (Notation 3.2); with $m = 0$ and premise-free constructors the rule degenerates to the case rule of [Cza26].

Example 3.8. eq_σ is a forced singleton: the clause $\text{refl}_{\text{eq}} : \forall x : \sigma. \text{eq}_\sigma(x, x)$ has $\vec{u}^0 = (x, x)$ and $j(1) = 1$ (or 2). For an occurrence $\text{eq}_\sigma(a, b)$, the solution substitution is $\theta = [a/x]$, so $\text{sca}_{\text{eq}_\sigma}(\pi; z_1 z_2. \tau; .t)$ requires $t : \tau[a/z_1, a/z_2]$ and produces $\tau[a/z_1, b/z_2]$ — precisely transport in the second index. The primitive $\text{cast}(\cdot; \cdot)$ is the same elimination for the primitive equality. The predicate le is not a forced singleton (two clauses); **Acc**-style predicates are excluded from CIC_{ix}^- altogether because their recursive premises sit under quantifiers (Section 3.7).

Remark 3.9 (Choice of solution). If j and j' are both solutions, the two instantiations of the $\text{sca}(\cdot; \cdot)$ rule are interderivable: from $\pi : J(\vec{u})$ one obtains, by ind_J with motive $\vec{z}. (z_{j(i)} =_{\sigma_{j(i)}} z_{j'(i)})$ — whose clause obligation is $u_{j(i)}^0 = u_{j'(i)}^0$, i.e. $x_i = x_i$, discharged by refl — a proof of $u_{j(i)} = u_{j'(i)}$ for every i , and $\text{cast}(\cdot; \cdot)$ along these equations mediates between the two premise and motive instantiations. (The equation is well formed because both index types are convertible to the type of x_i .) Semantically both instantiations are validated by the same argument (Lemma 7.6), so fixing j loses no generality.

Remark 3.10 (Relation to CIC’s singleton criterion). CIC permits a match on a **Prop**-sorted scrutinee in an informative position when the inductive is empty or a *singleton*: one constructor whose arguments are all propositional, *parameters put aside* [The26, Let03]. Under the monomorphic stratifica-

$$\begin{array}{c}
\frac{\Gamma \vdash \pi : \varphi_1 \wedge \varphi_2 \quad \Gamma, p_1 : \varphi_1, p_2 : \varphi_2 \vdash t : \tau}{\Gamma \vdash \text{split}(\pi; p_1 p_2.t) : \tau} \quad \frac{\Gamma \vdash t : \sigma \quad \sigma \simeq \sigma' \quad \Gamma \vdash \sigma' \text{ type}}{\Gamma \vdash t : \sigma'} \\
\frac{\Gamma \vdash \pi : a =_{\sigma} b \quad \Gamma, z : \sigma \vdash \tau \text{ type} \quad \Gamma \vdash t : \tau[a/z]}{\Gamma \vdash \text{cast}(\pi; z.\tau; t) : \tau[b/z]} \\
\begin{array}{l}
J \text{ a forced singleton with clause } k : \forall \vec{x} : \vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_{\ell} \rightarrow J(\vec{u}^0) \\
\text{and solution } j \text{ (Definition 3.7); } \quad \theta = [u_{j(i)}/x_i]_i \\
\Gamma \vdash \pi : J(\vec{u}) \quad \Gamma, \vec{z} : \vec{\sigma} \vdash \tau \text{ type} \quad \Gamma, \vec{p} : \theta(\vec{\varphi}) \vdash t : \tau[\theta(\vec{u}^0)/\vec{z}]
\end{array} \\
\hline
\Gamma \vdash \text{scase}_J(\pi; \vec{z}.\tau; \vec{p}.t) : \tau[\vec{u}/\vec{z}] \\
\begin{array}{l}
T = \Pi x_1 : \sigma_1 \dots \Pi x_n : \sigma_n. \tau \quad \sigma_k = I(\vec{v}) \quad \Gamma \vdash T \text{ type} \\
t = \lambda \vec{x} : \vec{\sigma}. \text{case}(x_k; \vec{z}.z.\tau_0; \vec{br}) \quad \tau = \tau_0[\vec{v}/\vec{z}, x_k/z] \\
\Gamma, f : T, \vec{x} : \vec{\sigma}, \vec{y}_i : \vec{A}_i, \vec{p}_i : \vec{\varphi}_i, \vec{q}_i : (\vec{v} =_{\vec{\sigma}'} t_i) \\
\vdash b_i : \tau_0[t_i/\vec{z}, c_i(\vec{y}_i; \vec{p}_i)/z] \quad (\text{all } i) \\
\text{grd}_k(f; \vec{y}_i^{\text{rec}}; b_i) \quad (\text{all } i)
\end{array} \\
\hline
\Gamma \vdash \text{fix } f : T. t : T
\end{array}$$

Figure 3: Typing of terms, part II. In the $\text{scase}_J(; ;)$ rule, J , its clause data and j are as in Definition 3.7. In the fix rule, $\vec{br}$ abbreviates the branch list with bodies b_i ; x_k is the designated *guard argument*; $\vec{\sigma}'$ are the index types of I ; $\vec{y}_i^{\text{rec}} \subseteq \vec{y}_i$ denotes the recursive argument positions of c_i ; and the *guard condition* $\text{grd}_k(f; V; b)$ requires every occurrence of f in b to lie at the head of an application spine $f a_1 \dots a_n$ occurring as a subterm of b , with $a_k \in V$ (so f may head such a spine but not occur as an argument, of itself or of anything else; Definition 7.4 makes the reading formal). We further require $f, \vec{x}, \vec{y}_i$ not to be shadowed in b_i .

$$\begin{array}{c}
\frac{p : \varphi \in \Gamma}{\Gamma \vdash p : \varphi} \quad \frac{(c := \pi : \varphi) \in \mathbb{E}}{\Gamma \vdash c : \varphi} \quad \frac{\Gamma, p : \varphi \vdash \pi : \psi}{\Gamma \vdash \lambda p : \varphi. \pi : \varphi \rightarrow \psi} \\
\frac{\Gamma \vdash \pi : \varphi \rightarrow \psi \quad \Gamma \vdash \pi' : \varphi}{\Gamma \vdash \pi \pi' : \psi} \quad \frac{\Gamma, x : \sigma \vdash \pi : \varphi}{\Gamma \vdash \lambda x : \sigma. \pi : \forall x : \sigma. \varphi} \quad \frac{\Gamma \vdash \pi : \forall x : \sigma. \varphi \quad \Gamma \vdash a : \sigma}{\Gamma \vdash \pi a : \varphi[a/x]} \\
\frac{\Gamma, X : K \vdash \pi : \varphi}{\Gamma \vdash \Lambda X : K. \pi : \forall X : K. \varphi} \quad \frac{\Gamma \vdash \pi : \forall X : K. \varphi \quad \Gamma \vdash P : K}{\Gamma \vdash \pi P : \varphi[P/X]} \\
\frac{\Gamma \vdash \pi_1 : \varphi_1 \quad \Gamma \vdash \pi_2 : \varphi_2}{\Gamma \vdash \langle \pi_1, \pi_2 \rangle : \varphi_1 \wedge \varphi_2} \quad \frac{\Gamma \vdash \pi : \varphi_1 \wedge \varphi_2}{\Gamma \vdash \pi.i : \varphi_i} \quad \frac{\Gamma \vdash \pi : \varphi_i}{\Gamma \vdash \iota_i(\pi) : \varphi_1 \vee \varphi_2} \\
\frac{\Gamma \vdash \pi : \varphi_1 \vee \varphi_2 \quad \Gamma, p_i : \varphi_i \vdash \pi_i : \psi \ (i = 1, 2)}{\Gamma \vdash \text{vcase}(\pi; p_1.\pi_1; p_2.\pi_2) : \psi} \quad \frac{\Gamma \vdash a : \sigma \quad \Gamma \vdash \pi : \varphi[a/x]}{\Gamma \vdash \langle a, \pi \rangle : \exists x : \sigma. \varphi} \\
\frac{\Gamma \vdash \pi : \exists x : \sigma. \varphi \quad \Gamma, x : \sigma, p : \varphi \vdash \pi' : \psi \quad x, p \notin \text{FV}(\psi)}{\Gamma \vdash \text{ecase}(\pi; x p. \pi') : \psi} \quad \frac{}{\Gamma \vdash \text{tt} : \top} \\
\frac{\Gamma \vdash \pi : \perp \quad \Gamma \vdash \varphi \text{ prop}}{\Gamma \vdash \text{efq}(\pi; \varphi) : \varphi} \quad \frac{\Gamma \vdash a : \sigma}{\Gamma \vdash \text{refl}_\sigma(a) : a =_\sigma a} \\
\frac{\Gamma \vdash \pi : a =_\sigma b \quad \Gamma, z : \sigma \vdash \psi \text{ prop} \quad \Gamma \vdash \pi' : \psi[a/z]}{\Gamma \vdash \text{rew}(\pi; z. \psi; \pi') : \psi[b/z]}
\end{array}$$

Figure 4: Typing of proofs, part I.

$$\begin{array}{c}
\frac{\Gamma \vdash \pi : c(\vec{a}; \vec{\pi}_1) =_\sigma d(\vec{b}; \vec{\pi}_2) \quad c \neq d \text{ constructors of a datatype } I}{\Gamma \vdash \text{discr}(\pi) : \perp} \\
\frac{\Gamma \vdash t : \{x : \sigma \mid \varphi\}}{\Gamma \vdash \text{valid}(t) : \varphi[\text{val}(t)/x]} \quad \frac{\Gamma \vdash \pi : \varphi \quad \varphi \simeq \varphi' \quad \Gamma \vdash \varphi' \text{ prop}}{\Gamma \vdash \pi : \varphi'} \\
\frac{\begin{array}{c} k : \forall \vec{x} : \vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow J(\vec{u}) \in \mathbb{E} \\ \Gamma \vdash a_i : \tau_i[\vec{a}_{<i}/\vec{x}_{<i}] \text{ (all } i) \quad \Gamma \vdash \pi_l : \varphi_l[\vec{a}/\vec{x}] \text{ (all } l) \end{array}}{\Gamma \vdash k(\vec{a}; \vec{\pi}) : J(\vec{u}[\vec{a}/\vec{x}])} \\
\frac{\Gamma \vdash \pi : J(\vec{a}) \quad \Gamma, \vec{z} : \vec{\sigma} \vdash \psi \text{ prop} \quad \Gamma \vdash \pi_j : \Xi_j(\vec{z}. \psi) \text{ (all clauses } j)}{\Gamma \vdash \text{ind}_J(\vec{z}. \psi; \vec{\pi}; \pi) : \psi[\vec{a}/\vec{z}]} \\
\frac{\begin{array}{c} \Gamma \vdash a : I(\vec{u}) \quad \Gamma, \vec{z} : \vec{\sigma}, z : I(\vec{z}) \vdash \psi \text{ prop} \\ \Gamma, \vec{y}_i : \vec{A}_i, \vec{p}_i : \vec{\varphi}_i, \vec{h}_i : (\psi[\vec{v}_{ij}/\vec{z}, y_{ij}/z])_{j \in \text{rec}(i)} \\ \vdash \pi_i : \psi[\vec{t}_i/\vec{z}, c_i(\vec{y}_i; \vec{p}_i)/z] \text{ (all } i) \end{array}}{\Gamma \vdash \text{ind}_I(\vec{z}. \psi; (\vec{y}_i \vec{p}_i \vec{h}_i.\pi_i)_i; a) : \psi[\vec{u}/\vec{z}, a/z]}
\end{array}$$

Figure 5: Typing of proofs, part II. For a predicate clause $k_j : \forall \vec{x} : \vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow J(\vec{u})$, the *clause obligation* is $\Xi_j(\vec{z}. \psi) = \forall \vec{x} : \vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow \psi[\vec{v}_1/\vec{z}] \rightarrow \dots \rightarrow \psi[\vec{v}_s/\vec{z}] \rightarrow \psi[\vec{u}/\vec{z}]$, where $J(\vec{v}_1), \dots, J(\vec{v}_s)$ are the recursive premises among the φ_i . In the datatype induction ind_I , $\text{rec}(i)$ is the set of recursive argument positions of c_i , and for $j \in \text{rec}(i)$ with $A_{ij} = I(\vec{v}_{ij})$ the hypothesis h_{ij} has the displayed type; the clause proofs bind $\vec{y}_i, \vec{p}_i, \vec{h}_i$ because the conclusion instance mentions the constructor term $c_i(\vec{y}_i; \vec{p}_i)$, which requires names for the premise proofs.

tion of Section 3.7, the parameters of an equality-like family become clause variables that occur verbatim among the result indices, and the remaining CIC constructor arguments become the premises $\vec{\varphi}$. Every CIC singleton elimination is therefore an instance of $\text{scase}(\cdot; \cdot)$. A taxonomy note: what Definition 3.7 demands is that the clause variables be *verbatim-forced* — literally among the indices. The forcing analyses of Brady et al. [BMM04] and Gilbert et al. [GCST19] are more liberal: they also count *pattern-forced* arguments, occurring under constructors in an index term (x under $\mathbf{S}$, recoverable by injectivity). $\text{scase}(\cdot; \cdot)$ therefore sits strictly between CIC’s parameter criterion and the Gilbert et al. criterion. The pattern-forced generalisation appears semantically available — index reflection (Lemma 6.5) applied to the index datatype recovers the argument — but the solution substitution θ would no longer be a syntactic substitution, and we leave it as future work. In the other direction, the converse of the inclusion above fails: Definition 3.7 does not require the forced variables to occur *uniformly* in recursive premises, so it accepts clauses that CIC’s parameter mechanism cannot express (e.g. $k : \forall x:\text{nat}. J(\mathbf{S}x, \mathbf{O}) \rightarrow J(x, \mathbf{S}\mathbf{O})$). The permissiveness is deliberate and harmless: soundness is proved against the model (Lemma 7.6), which validates the general rule; only proof *reconstruction* of uses of the extra generality requires inversion-style reasoning rather than a primitive CIC match (Section 11.6). We note that the intended front end never *produces* that extra generality: it elaborates CIC singleton eliminations, whose images satisfy the parameter criterion. The margin is robustness of the calculus, not a feature the pipeline exercises, so the corresponding reconstruction obligation is expected to be vacuous in practice. Empty elimination needs no new former: for a predicate J declared with zero clauses, ind_J with motive $\perp$ and no clause obligations yields $\perp$ from $J(\vec{a})$, and $\text{efq}(\cdot)$ concludes.

3.4 Reduction and conversion

Definition 3.11 (Reduction). The relation $\rightarrow$ on terms is the contextual closure — in all term, proof, type, proposition and predicate positions — of the following root rules:

$$\begin{array}{ll}
(\lambda x:\sigma.t) a \rightarrow t[a/x] & (\lambda p:\varphi.t) \pi \rightarrow t[\pi/p] \\
\text{fix } f:T. t \rightarrow t[\text{fix } f:T. t/f] & \text{val}(\text{refine}(t, \pi)) \rightarrow t \\
c \rightarrow t \quad \text{if } (c := t : \sigma) \in \mathbb{E} & \text{cast}(\text{refl}_\sigma(a); z.\tau; t) \rightarrow t \\
\text{split}(\langle \pi_1, \pi_2 \rangle; p_1 p_2.t) \rightarrow t[\pi_1/p_1][\pi_2/p_2]
\end{array}$$

together with the singleton-elimination rule

$$\text{scase}_J(k(\vec{a}; \vec{\pi}); \vec{z}.\tau; \vec{p}.t) \rightarrow t[\vec{\pi}/\vec{p}]$$

and the rule for case analysis meeting a constructor,

$$\text{case}(c_i(\vec{a}; \vec{\pi}); \vec{z}.z.\tau; \{\dots\}) \rightarrow t_i[\vec{a}/\vec{y}_i, \vec{\pi}/\vec{p}_i, \overrightarrow{\text{refl}}(\vec{t}_i[\vec{a}/\vec{y}_i])/\vec{q}_i],$$

where $\overrightarrow{\text{refl}}(\vec{t}_i[\vec{a}/\vec{y}_i])$ abbreviates the proof list

$$\text{refl}_{\sigma_1}(t_{i1}[\vec{a}/\vec{y}_i]), \dots, \text{refl}_{\sigma_m}(t_{im}[\vec{a}/\vec{y}_i])$$

substituted for the index-equation variables. Source *conversion* $\simeq$ is the least congruence (on each syntactic category, closing under all formers) containing $\rightarrow$.

Reduction is untyped and unrestricted. As in the companion paper, no metatheoretic property of $\rightarrow$ itself is needed anywhere: $\simeq$ enters only through the conversion typing rules, and the semantics of Section 6 is invariant under it (Lemma 6.8). Note that the index-equation variables $\vec{q}_i$ receive *reflexivity* proofs when a case meets a constructor: at the redex, the occurrence indices are convertible to the constructor's index pattern, and the conversion typing rule absorbs the mismatch. Since proofs are erased (Definition 4.5), this substitution is invisible in the target.

Lemma 3.12 (Rigidity of type formers). *If $\sigma \simeq \sigma'$ then σ and σ' have the same outermost former: both are occurrences $I(\vec{u})$, $I(\vec{u}')$ of the same datatype I with $u_h \simeq u'_h$ for all h , or both are $\Pi x:\rho.\tau$ -types whose domains and codomains are respectively convertible, or both are $\varphi \rightarrow \tau$ -types with convertible components, or both are refinement types with convertible components. The analogous statement holds for propositions.*

Proof. No root rule of Definition 3.11 applies at the root of a set type or a proposition — the root rules rewrite terms and proofs only — so every $\rightarrow$ -step inside a type or proposition happens strictly below its outermost former: inside an index term of a datatype occurrence, inside a component type or proposition, or inside an embedded term or proof. The claim follows by induction on the length of the conversion path, observing that a single step preserves the outermost former and relates components (index terms included) pointwise. $\square$

Lemma 3.13 (Substitution). *All judgements admit substitution: if $\Gamma, x:\sigma, \Gamma' \vdash \mathcal{J}$ and $\Gamma \vdash a : \sigma$, then $\Gamma, \Gamma'[a/x] \vdash \mathcal{J}[a/x]$, and analogously for proof variables and predicate variables.*

Proof. By induction on the derivation of the second judgement, simultaneously for all judgement forms and all three substitution kinds; write $[\cdot]$ for the substitution being pushed ($[a/x]$ in the notation of the statement).

Three mechanisms carry the routine cases, and two new rules deserve their arguments in full.

Routine mechanisms. (i) Every rule is homomorphic in its components, so the induction hypotheses on the premises reassemble the rule instance, using the composition identity $\tau[a'/y][a/x] = \tau[a/x][a'[a/x]/y]$ (for $y \neq x$, $y \notin \text{FV}(a)$, after α -renaming) wherever the conclusion contains a substituted phrase; in the constructor, case and ind_I rules the declaration data $(A_{ij}, \varphi_{il}, \vec{t}_i)$ mention only the constructor telescope and earlier declarations, so they are untouched by $[\cdot]$ and only the instantiating terms absorb it, e.g. $(A_{ij}[\vec{a}_{<j}/\vec{y}_{<j}])[a/x] = A_{ij}[(\vec{a}_{<j}[a/x])/\vec{y}_{<j}]$. (ii) $\simeq$ is closed under substitution: each root rule of Definition 3.11 is — for the forced-case rule, both sides substitute componentwise and the reflexivity list $\vec{\text{refl}}(\vec{t}_i[\vec{a}/\vec{y}_i])$ becomes the reflexivity list of the substituted instances — so the two conversion rules follow from the induction hypothesis. (iii) Substitution of a predicate expression into an atom, $X(\vec{a})[P/X] = P \cdot \vec{a}$ (predicate application, β -reducing the λ -form), yields a well-formed proposition by the kind premise, and the process terminates because P cannot mention X after α -renaming.

The $\text{scase}(\cdot; \cdot; \cdot)$ case (commutation of θ with the ambient substitution). Let the rule be applied at an occurrence $J(\vec{u})$, with clause data $k : \forall \vec{x} : \vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow J(\vec{u}^0)$, solution j , and solution substitution $\theta = [u_{j(i)}/x_i]_i$. After the ambient substitution the scrutinee proof has type $J(\vec{u}[a/x])$, whose solution substitution is $\theta' = [u_{j(i)}[a/x]/x_i]_i$ (the clause data $\vec{u}^0, j$ are closed declaration components and do not change). For every clause phrase ξ — a premise φ_l or the pattern tuple $\vec{u}^0$ — the free variables of ξ lie among $\vec{x}$ (Definition 3.3), so the composed substitution acts on ξ only through the images of $\vec{x}$:

$$(\theta(\xi))[a/x] = \xi[u_{j(i)}[a/x]/x_i]_i = \theta'(\xi).$$

Hence $\theta(\vec{\varphi})[a/x] = \theta'(\vec{\varphi})$ and $\tau[\theta(\vec{u}^0)/\vec{z}][a/x] = \tau[a/x][\theta'(\vec{u}^0)/\vec{z}]$ (using (i) for the motive), and the induction hypotheses re-derive the rule at $J(\vec{u}[a/x])$ with solution substitution θ' , producing the substituted conclusion type $\tau[a/x][\vec{u}[a/x]/\vec{z}] = (\tau[\vec{u}/\vec{z}])[a/x]$.

The discr case. From $\Gamma, x:\sigma, \Gamma' \vdash \pi : c(\vec{a}'; \vec{\pi}_1) =_\sigma d(\vec{b}'; \vec{\pi}_2)$ with $c \neq d$: substitution is homomorphic on constructor terms, $(c(\vec{a}'; \vec{\pi}_1))[a/x] = c(\vec{a}'[a/x]; \vec{\pi}_1[a/x])$, so both equation sides remain constructor-headed with the same distinct head constructors, and the rule re-applies to the substituted premise supplied by the induction hypothesis. (This is the only rule whose side condition inspects the *shape* of a term, which is why it is singled out.) $\square$

3.5 The shallow fragment

Definition 3.14 (Shallow fragment $\mathfrak{S}$). A proposition is *shallow* if it contains no subformula $\forall X:K.\varphi$ and no atom $X(\vec{a})$ with X a predicate variable;

a set type is shallow if all propositions occurring in it (refinement payloads and premises) are shallow. $\mathfrak{S}$ is the class of shallow propositions and set types.

Index terms of datatype occurrences are terms and are always shallow material; only genuine predicate quantification is excluded. As in the companion paper, the translation is defined on $\mathfrak{S}$; proofs inside the environment may use the full calculus, but a lemma whose statement leaves $\mathfrak{S}$ contributes no premise axiom, a definition whose type leaves $\mathfrak{S}$ contributes no specification axiom, and goals must be shallow and closed. We require the declarations of $\mathbb{E}$ (datatype index types, constructor argument types, premises and index terms; inductive-predicate components) to lie in $\mathfrak{S}$.

3.6 Programming with forded case analysis

The index equations $\vec{q}_i$ in branches are what makes dependently typed idioms expressible with *first-order* motives. We record the two standard examples and the derivability of injectivity; both are used in Section 5 as running translation examples.

Lemma 3.15 (Derivable injectivity). *Let c be a constructor of a datatype I whose j -th argument type A_j is closed — it mentions no earlier telescope variables; it may be a non-recursive type or a recursive occurrence $I(\vec{v})$ at closed indices (for a non-indexed datatype, in particular, any recursive position qualifies). Then from $\Gamma \vdash \pi : c(\vec{a}; \vec{\pi}_1) =_\sigma c(\vec{b}; \vec{\pi}_2)$ one can derive $\Gamma \vdash \text{inj}_j(\pi) : a_j =_{A_j} b_j$ for a proof term $\text{inj}_j(\pi)$ definable uniformly from the derivation.*

Proof. Since $\Gamma \vdash c(\vec{a}; \vec{\pi}_1) : \sigma$ and the constructor rule produces the type $I(\vec{t}[\vec{a}/\vec{y}])$, by Lemma 3.12 $\sigma \simeq I(\vec{u})$ for some $\vec{u}$; fix such an occurrence. Define the projection

$$F := \lambda w : I(\vec{u}). \text{case}(w; \vec{z}.z.A_j; \{c(\vec{y}; \vec{p}; \vec{q}) \Rightarrow y_j \mid d(\dots) \Rightarrow a_j \ (d \neq c)\}).$$

The motive is the constant type A_j , well formed in the motive context since A_j is closed (this includes the recursive case $A_j = I(\vec{v})$ with $\vec{v}$ closed: a legal occurrence like any other); the c -branch is well typed because $y_j : A_j$ (again by closedness, no instantiation of earlier telescope variables occurs) and the branch obligation is A_j itself; the default branches use a_j , well typed over Γ by weakening (its type $A_j[\vec{a}_{<j}/\vec{y}_{<j}] = A_j$ by closedness). Then F is well typed, $F c(\vec{a}; \vec{\pi}_1) \simeq a_j$ and $F c(\vec{b}; \vec{\pi}_2) \simeq b_j$ by a β - and a case-step. Now

$$\text{inj}_j(\pi) := \text{rew}(\pi; z. (a_j =_{A_j} F z); \text{refl}_{A_j}(a_j)),$$

where the base proof is well typed at $(a_j =_{A_j} F c(\vec{a}; \vec{\pi}_1)) \simeq (a_j =_{A_j} a_j)$ by the conversion rule, and the conclusion converts to $a_j =_{A_j} b_j$. $\square$

Example 3.16 (Head and tail). With the declarations of Example 3.4:

```

vhead := λn:nat. λv:vec(S n).
  case(v; z.z.A; {vnil(;; q) ⇒ efq(discr(q); A) | vcons(k, a, w;; q) ⇒ a})

vtail := λn:nat. λv:vec(S n).
  case(v; z.z.vec(n); {vnil(;; q) ⇒ efq(discr(q); vec(n)) |
    vcons(k, a, w;; q) ⇒
      cast(sym(inj1(q)); z.vec(z); w) })

```

where $\text{sym}(\pi) := \text{rew}(\pi; z.(z =_{\text{nat}} n); \text{refl}_{\text{nat}}(n))$ derives $k =_{\text{nat}} n$ from $\text{inj}_1(q) : n =_{\text{nat}} k$, and the vcons branch of vtail is, written in full, $\text{cast}(\text{sym}(\text{inj}_1(q)); z.\text{vec}(z); w)$. In the vnil branches, $q : S n =_{\text{nat}} 0$ and $\text{discr}(q) : \perp$ discharges the impossible branch — with a *constant* motive; no large elimination, no unit-valued return-type trick. In the vcons branch of vtail , $q : S n =_{\text{nat}} S k$ gives $\text{inj}_1(q) : n =_{\text{nat}} k$ by Lemma 3.15, whence $\text{sym}(\text{inj}_1(q)) : k =_{\text{nat}} n$, and the cast transports $w : \text{vec}(k)$ to $\text{vec}(n)$. Both definitions are well typed in CIC_{ix}^- ; their translations are computed in Example 5.9.

3.7 Relation to CIC

CIC_{ix}^- is designed as the image of a first-order, monomorphic slice of CIC under analyses that are part of the CoqHammer translation pipeline or of program extraction. The discussion of stratification, monomorphisation, refinement classification, structural fixpoints and the propositional connectives in [Cza26, §2.6] applies verbatim; we record only what is new for indexed families.

- *Parameters and indices.* CIC distinguishes uniform parameters from indices; the distinction matters for elimination (a parameter is fixed across a match, an index is generalized). After monomorphisation, *type* parameters are ground and disappear into the datatype name (vec above is really vec_A). *Term* and *proposition* parameters become indices (eq_σ 's left argument) or premises (rfl_{φ_0} 's P), and the forced-variable analysis of Definition 3.7 recovers, per occurrence, exactly the specialization that CIC's parameter mechanism provided — this is the content of Remark 3.10. Non-uniform parameters are indices here, as they are in CIC's kernel.
- *Dependent case analysis.* CIC's $\text{match } a \text{ in } I \text{ } \vec{z} \text{ return } \tau \text{ with } \dots$ has branches typed at each constructor's own indices, with no equational premises; impossible branches are discharged by large-eliminating motives. The forced case of CIC_{ix}^- instead hands each branch the index equations. The two styles are intertranslatable in CIC extended with

UIP — this is precisely the Goguen–McBride–McKinna elaboration [GMM06] — and the front end is expected to produce forced branches by the same analysis Coq’s `inversion` performs [CT96, Mon10]: generalize the indices, introduce equations, and (optionally) simplify them. Branches the front end can discharge by rigid-index conflict arrive as `efq(discr(·); ·)` as in Example 3.16; branches it cannot discharge (green slime) arrive with their equations intact, which is the point (Remark 5.19). Large elimination remains excluded from CIC_{ix}^- : fording removes its main *programming* use (impossible branches), and its *logical* uses (proving discrimination) are covered by the primitive `discr`, which is CIC-derivable by exactly such a large elimination.

- *Propositional index positions.* Index types in CIC_{ix}^- are set types; CIC families indexed by *propositions* or *proofs* have no direct image. The front end drops proof-sorted index positions before entering the fragment; this is sound for the same reason the model is proof-irrelevant (all proofs of a proposition denote one element — Section 6), and is the translation-level counterpart of the rule that erased data may appear in indices only where nothing relevant observes them [Atk18, GCST19]. Proof *subterms* of index terms, by contrast, are in the fragment and are erased structurally (Remark 4.6).
- *Recursion.* The `fix` former captures structural recursion on a family element with a designated guard argument. Recursive calls may be at arbitrary index instantiations (the call `f k w` inside `vmap`-style definitions instantiates the index argument with the pattern variable `k`); the guard condition constrains only the guard position. Well-founded (Acc-based) recursion remains excluded, for the reasons analysed in [Cza26, §10.3]: its unfolding equations are sound only with their propositional premises retained, and its guardedness is semantic rather than syntactic [GCST19]. Relatedly, predicate clauses with recursive premises under quantifiers (Acc’s constructor) are excluded from the declaration format.

4 The intermediate language and program extraction

The first half of the translation is Letouzey-style program extraction: an erasure of CIC_{ix}^- terms into an untyped functional calculus $\lambda_{\square}$. The calculus is *the same* as in the companion paper — indexed families change constructor *arities* (a constructor keeps only its informative arguments) but not the shape of the target language — and the erasure function extends that of [Cza26] with clauses for the new formers. Two design points carry over: since the target of the overall translation is an equational theory rather

than a programming language with an evaluation order, erased arguments are *dropped* rather than replaced by dummies; and the singleton eliminations collapse to their branches structurally. A third point is new: erasure is *index-blind*. Neither the motive indices of a case, nor the index equations of its branches, nor the index terms of types leave any trace; index terms occurring as *constructor arguments* (the length argument of `vcons`) are informative and are kept, exactly as in Letouzey’s extraction [Let03, Let08], where redundant stored indices are likewise kept. The re-materialisation of index information is the job of specification extraction (Section 5.3), not of the program.

4.1 The calculus $\lambda_{\square}$

Definition 4.1 (Syntax of $\lambda_{\square}$). Fix the datatype constructors c of a $\text{CIC}_{\text{ix}}^{-}$ environment $\mathbb{E}$, each with arity r equal to its number of *informative* arguments (premises do not count), the term-definition constants c of $\mathbb{E}$, and a distinguished constant $\square$ (“dead code”). The terms of $\lambda_{\square}$ are

$$e ::= x \mid c \mid \square \mid \lambda x.e \mid e e' \mid c(e_1, \dots, e_r) \mid \text{case}(e; \{c_i(\vec{x}_i) \Rightarrow e_i\}_i) \mid \text{fix } f.e$$

where constructors are fully applied and a case expression carries one branch for each constructor of the datatype of its constructors.

Definition 4.2 (Reduction of $\lambda_{\square}$). The relation $\rightarrow_{\square}$ is the contextual closure of:

$$\begin{aligned} (\lambda x.e) e' &\rightarrow_{\square} e[e'/x] & (\beta) \\ \text{case}(c_i(\vec{e}); \{c_j(\vec{x}_j) \Rightarrow e_j\}_j) &\rightarrow_{\square} e_i[\vec{e}/\vec{x}_i] & (\iota) \\ \text{fix } f.e &\rightarrow_{\square} e[\text{fix } f.e/f] & (\mu) \\ c &\rightarrow_{\square} \mathcal{E}(t) \quad \text{if } (c := t : \sigma) \in \mathbb{E} & (\delta) \end{aligned}$$

where $\mathcal{E}$ is the erasure of Definition 4.5 (the δ -rule is well defined because the body of c mentions only constants declared before c). We write $=_{\square}$ for the least congruence containing $\rightarrow_{\square}$.

The four rule families have pairwise distinct, non-overlapping, left-linear redex shapes with rigid heads; $\lambda_{\square}$ is an orthogonal higher-order rewrite system, and its confluence is independent of the constructor arities:

Theorem 4.3 (Confluence of $\lambda_{\square}$). $\rightarrow_{\square}$ is confluent. Consequently, $e_1 =_{\square} e_2$ if and only if e_1 and e_2 have a common $\rightarrow_{\square}^*$ -reduct.

Proof. By the Tait–Martin–Löf parallel-reduction method in Takahashi’s formulation [Tak95]; the full proof is in Appendix A. Alternatively, $\lambda_{\square}$ is an orthogonal combinatory reduction system, so confluence follows from Klop’s theorem [Klo80]. $\square$

Corollary 4.4 (Constructor discrimination and injectivity). *Let c, d be constructors.*

1. *If $c(\vec{e}) =_{\square} d(\vec{e}')$ then $c = d$ and $e_i =_{\square} e'_i$ for all i .*
2. *An application $e_1 e_2$ in which e_1 is normal and not a λ -abstraction (for instance $e_1 = c(\vec{e})$ with $\vec{e}$ normal) is not convertible to any constructor form $d(\vec{e}')$.*

Proof. (1) Constructor-headed terms only reduce internally: every reduct of $c(\vec{e})$ has the form $c(\vec{e}'')$ with $e_i \rightarrow_{\square}^* e''_i$. By Theorem 4.3, the two sides have a common reduct, which is then simultaneously c -headed and d -headed, forcing $c = d$; its arguments are common reducts of the respective e_i, e'_i . (2) Such an application is $\rightarrow_{\square}$ -normal apart from reductions inside e_1 and e_2 , which preserve the application shape, so a common reduct with a constructor form is impossible as in (1). $\square$

4.2 Erasure

Definition 4.5 (Erasure). The function $\mathcal{E}$ from CIC_{ix}^- terms to $\lambda_{\square}$ terms is defined by structural recursion:

$$\begin{aligned}
\mathcal{E}(x) &= x & \mathcal{E}(c) &= c \\
\mathcal{E}(t a) &= \mathcal{E}(t) \mathcal{E}(a) & \mathcal{E}(t \pi) &= \mathcal{E}(t) \\
\mathcal{E}(\lambda x:\sigma. t) &= \lambda x. \mathcal{E}(t) & \mathcal{E}(\lambda p:\varphi. t) &= \mathcal{E}(t) \\
\mathcal{E}(c(\vec{a}; \vec{\pi})) &= c(\mathcal{E}(\vec{a})) & \mathcal{E}(\text{fix } f:T. t) &= \text{fix } f. \mathcal{E}(t) \\
\mathcal{E}(\text{case}(a; \vec{z}.z.\tau; \{c_i(\vec{y}_i; \vec{p}_i; \vec{q}_i) \Rightarrow t_i\}_i)) &= \text{case}(\mathcal{E}(a); \{c_i(\vec{y}_i) \Rightarrow \mathcal{E}(t_i)\}_i) \\
\mathcal{E}(\text{refine}(t, \pi)) &= \mathcal{E}(t) & \mathcal{E}(\text{val}(t)) &= \mathcal{E}(t) \\
\mathcal{E}(\text{split}(\pi; p_1 p_2. t)) &= \mathcal{E}(t) & \mathcal{E}(\text{efq}(\pi; \sigma)) &= \square \\
\mathcal{E}(\text{cast}(\pi; z.\tau; t)) &= \mathcal{E}(t) \\
\mathcal{E}(\text{sca}_{\text{sc}}(\pi; \vec{z}.\tau; \vec{p}.t)) &= \mathcal{E}(t)
\end{aligned}$$

The clauses for the new formers say: constructor terms keep their informative arguments and lose their premise proofs; a case on a family keeps its scrutinee and branch bodies and loses the motive, the premise binders and the index-equation binders; the singleton elimination $\text{sca}_{\text{sc}}(\pi; \vec{z}; \vec{p}.t)$ collapses to its branch like $\text{split}(\pi; p_1 p_2. t)$ and $\text{cast}(\pi; z.\tau; t)$. Proofs are never inspected — $\mathcal{E}$ does not recurse into the category of proofs at all — which is the syntactic form of proof irrelevance.

Remark 4.6 (Proofs inside index terms). An index term may contain proof subterms — $\text{vec}(f x \pi)$ for $f : \Pi x:\text{nat}. \varphi \rightarrow \text{nat}$ — and these vanish under erasure: $\mathcal{E}(f x \pi) = f x$. Consequently the translated index (Definition 5.10) and its interpretation (Definition 6.2) are independent of the proof, for arbitrary f , not only parametric ones. This is the translation-level form of

interpreting all erased data by one canonical element, with three precedents: Letouzey’s and MetaCoq’s $\square$ (operational) [Let03, SBF⁺20], Atkey’s $!_0x = !_0y$ canonical erased realiser (algebraic) [Atk18], and the canonical proof element of set-theoretic proof-irrelevant models (denotational) [LW11]. Lemma 6.7(2) is the precise statement in our model.

Lemma 4.7 (Erasure commutes with substitution). $\mathcal{E}(t[a/x]) = \mathcal{E}(t)[\mathcal{E}(a)/x]$; $\mathcal{E}(t[\pi/p]) = \mathcal{E}(t)$; $\mathcal{E}(t[P/X]) = \mathcal{E}(t)$.

Proof. Induction on t . For the first equation the variable case is immediate, and all other cases follow from the induction hypothesis because $\mathcal{E}$ is homomorphic in exactly the informative positions and substitution acts pointwise on them; positions discarded by $\mathcal{E}$ (proofs, motives, annotations, premise and equation binders) absorb the substitution without residue. For the second equation, observe that p can occur in t only inside proof positions or inside types, propositions and annotations, all of which $\mathcal{E}$ discards; similarly for the third. $\square$

Lemma 4.8 (Erasure simulates reduction). *If $t \rightarrow u$ then $\mathcal{E}(t) \rightarrow_{\square} \mathcal{E}(u)$ or $\mathcal{E}(t) = \mathcal{E}(u)$. Consequently $t \simeq u$ implies $\mathcal{E}(t) =_{\square} \mathcal{E}(u)$.*

Proof. By induction on the derivation of $t \rightarrow u$. For root steps:

- $(\lambda x:\sigma.t) a \rightarrow t[a/x]$: the erasure β -reduces to $\mathcal{E}(t[a/x])$ by Lemma 4.7.
- $(\lambda p:\varphi.t) \pi \rightarrow t[\pi/p]$: both sides erase to $\mathcal{E}(t)$; zero steps.
- Case on a constructor:

$$\begin{aligned} \mathcal{E}(\text{case}(c_i(\vec{a}; \vec{\pi}); \vec{z}.z.\tau; \dots)) &= \text{case}(c_i(\mathcal{E}(\vec{a})); \{c_j(\vec{y}_j) \Rightarrow \mathcal{E}(t_j)\}_j) \\ &\rightarrow_{\square} \mathcal{E}(t_i)[\mathcal{E}(\vec{a})/\vec{y}_i], \end{aligned}$$

one ι -step, and by Lemma 4.7 the target equals $\mathcal{E}(t_i[\vec{a}/\vec{y}_i, \vec{\pi}/\vec{p}_i, \overrightarrow{\text{refl}}/\vec{q}_i])$: the informative substitution is simulated pointwise and the proof substitutions (for $\vec{p}_i$ and $\vec{q}_i$) are invisible.

- μ (fixpoint unfolding) and δ : erase to the corresponding $\lambda_{\square}$ steps via Lemma 4.7.
- The singleton rules for $\text{val}()/\text{refine}()$, $\text{cast}();$, $\text{split}();$ and $\text{scase}_J();$: both sides erase to $\mathcal{E}(t)$ (using Lemma 4.7 for the proof substitutions); zero steps.

For congruence steps: a step inside an informative position maps to a step (or equality) in the corresponding position of the erasure by the induction hypothesis; a step inside a proof, type, proposition, motive, index or annotation position leaves the erasure unchanged. $\square$

Remark 4.9. Lemma 4.8 is the equational shadow of extraction correctness [Let04, §2.3]; the semantic half — the extracted term *realizes* its type — becomes the fundamental lemma of Section 7. Note where the indexed extension is invisible: the $\lambda_\square$ image of `vhead` is $\lambda n v. \text{case}(v; \{\text{vnil} \Rightarrow \square \mid \text{vcons}(k, a, w) \Rightarrow a\})$ — the impossible branch has become dead code, the possible branch a projection — and nothing in $\lambda_\square$ records that v was drawn from an indexed family. Soundness of the translation cannot, therefore, be an invariant of the program half alone; it is recovered in Section 6 by interpreting the *types*.

5 The translation

This section defines the emitted first-order theory. Throughout, fix a well-formed definitional environment $\mathbb{E}$ whose declarations lie in $\mathfrak{S}$ and a closed shallow goal proposition G . The target logic is classical first-order logic with equality $\approx$; $\vdash_{\text{FOL}}$ is provability in any standard sound and complete calculus [Sho67]; the propositional constants $\top, \perp$ are assumed available.

5.1 Signature

Definition 5.1 (Signature $\Sigma_{\mathbb{E}}$). The signature $\Sigma_{\mathbb{E}}$ contains:

- a $(1+m)$ -ary predicate symbol $\hat{I}$ (the *guard predicate*) for each datatype $I : (\sigma_1, \dots, \sigma_m) \rightarrow \mathbf{Set}$ of $\mathbb{E}$; the first argument place holds the guarded term, the remaining m the indices;
- a k -ary predicate symbol $\hat{J}$ for each inductive predicate $J : (\sigma_1, \dots, \sigma_k) \rightarrow \mathbf{Prop}$ of $\mathbb{E}$;
- an r -ary function symbol $\hat{c}$ for each constructor c of a datatype of $\mathbb{E}$, where r is the number of informative arguments of c (its $\lambda_\square$ arity);
- a function symbol $\hat{c}$ of arity a_c , and if $a_c > 0$ additionally a constant $\hat{c}^0$ (the *pointer* of c), for each term definition $c := t : \tau$ of $\mathbb{E}$, where the *erasure arity* a_c is determined from $\mathcal{E}(t)$ in Definition 5.4;
- a binary function symbol $@$ (infix, associating left), and a constant $\odot$ (the image of dead code $\square$);
- countably many *auxiliary* function symbols, produced by the compilation procedure below. As in [Cza18, §4] we assume injective naming functions $\Lambda(\cdot)$, $X(\cdot)$, $\Phi(\cdot)$, $\Phi^0(\cdot)$ to auxiliary symbols (hash-consing). Their domain is load-bearing, so we fix it exactly: the argument is a *pair* $(h, \theta \upharpoonright \text{FV}(h))$ of a $\lambda_\square$ term and the restriction, to the free variables of h , of the template map θ in force at the lifting site (Definition 5.2),

taken up to consistent renaming of free variables (the domain of the restriction renamed along). Two liftings receive the same symbol exactly when their terms *and* their templates agree; since the axioms attached to a lifted symbol are computed from this pair alone (Definitions 5.2 and 5.4 consult θ only at variables free in the lifted term), hash-consed merging is well defined. Keying on the bare term would identify a lifting whose free variable is a recursion variable with one whose free variable is ordinary, and attach the equations of both readings to a single symbol (Remark 5.7).

Source term variables double as first-order variables — with the exception of fix-bound recursion variables, which never occur in emitted formulas (Lemma 5.6).

Two negative observations, both load-bearing. First, there is no binary inhabitation predicate and no encoding of *types* as first-order terms: the indices of a family are *term* arguments of its guard predicate, so the type $\text{vec}(S\ n)$ contributes the atom $\widehat{\text{vec}}(w, \widehat{S}(n))$ — native first-order structure — rather than a term $\text{vec}(S(n))$ inside a `HasType` atom. Second, because types are not terms, no formula of the signature can even *state* injectivity of the type former I itself; Remark 8.17 explains why this inexpressibility is a feature.

5.2 Compilation of erased programs

The compilation of $\lambda_\square$ programs into defining equations follows the companion paper and is given here in full. Indexed families require no change to it whatsoever, because $\lambda_\square$ is unchanged; the present formulation is nevertheless not a verbatim reproduction — its treatment of *recursive closures* (the pinning of Definition 5.2, the template refinement of rule (2), and the naming keys of Definition 5.1) repairs a defect of the companion’s clauses, under which a recursion variable could leak into the closure of a lifted subterm and end up universally quantified in emitted axioms (Remark 5.7); the repair is to be read back into the companion. Compilation is a recursive procedure that *emits* first-order axioms and, for every symbol f it introduces, records a *witness*: a $\lambda_\square$ term e_f with free variables among the closure parameters of f , used only by the model construction of Section 6.

Compilation carries a finite map θ assigning to certain $\lambda_\square$ variables (fix-bound recursion variables) a *template* $(f, \vec{w}, m)$: an $(|\vec{w}| + m)$ -ary symbol f , a list of first-order terms $\vec{w}$ over ordinary first-order variables (the *closure terms*), and an argument count m . Every symbol occurring first in a template has an associated *pointer* of arity $|\vec{w}|$: the constant $\widehat{c}^0$ when $f = \widehat{c}$ is the root symbol of a fix-defined constant (it exists, $a_c \geq 1$ for such c), and Φ^0 of the same key when $f = \Phi(\cdot)$ is a fix-lift. The *pointer value* of a

θ -bound variable g with $\theta(g) = (f, \vec{w}, m)$ is the first-order term

$$\text{val}_\theta(g) := f^0(\vec{w})$$

(the constant f^0 itself when $\vec{w}$ is empty). For a list $\vec{w}$ of $\lambda_\square$ variables, the *pinned* list $\vec{w}^\theta$ replaces each θ -bound member g by $\text{val}_\theta(g)$ and keeps ordinary variables unchanged. Pinning is the entire treatment of recursion variables at lifting sites: a θ -bound variable is never itself a variable of an emitted formula (Lemma 5.6). In compiled *terms* it occurs only as the head of an application spine with at least m arguments — the image under erasure of the guard condition grd of Figure 3, whose spine shape survives because the guard argument is informative.

Definition 5.2 (Term translation $\mathcal{C}_\theta$). For a $\lambda_\square$ term e whose free variables are first-order variables or θ -bound, the first-order term $\mathcal{C}_\theta(e)$ is defined by recursion on e :

- $\mathcal{C}_\theta(x) = x$ for an ordinary variable; $\mathcal{C}_\theta(\square) = \odot$; $\mathcal{C}_\theta(\mathbf{c}(\vec{e})) = \widehat{\mathbf{c}}(\mathcal{C}_\theta(\vec{e}))$.
- For an application spine $e = h e_1 \cdots e_j$ ($j \geq 0$, h not an application, e itself not of the previous forms):
 - if $h = g$ with $\theta(g) = (f, \vec{w}, m)$: by the guard condition $j \geq m$, and $\mathcal{C}_\theta(e) = f(\vec{w}, \mathcal{C}_\theta(e_1), \dots, \mathcal{C}_\theta(e_m)) @ \mathcal{C}_\theta(e_{m+1}) @ \cdots @ \mathcal{C}_\theta(e_j)$;
 - if $h = c$ is a term-definition constant: if $j \geq a_c$ then $\mathcal{C}_\theta(e) = \widehat{\mathbf{c}}(\mathcal{C}_\theta(e_1), \dots, \mathcal{C}_\theta(e_{a_c})) @ \cdots @ \mathcal{C}_\theta(e_j)$, and otherwise $\mathcal{C}_\theta(e) = \widehat{c}^0 @ \mathcal{C}_\theta(e_1) @ \cdots @ \mathcal{C}_\theta(e_j)$;
 - if $h = x$ is a variable: $\mathcal{C}_\theta(e) = x @ \mathcal{C}_\theta(e_1) @ \cdots @ \mathcal{C}_\theta(e_j)$;
 - if $h = \square$ (necessarily $j \geq 1$): $\mathcal{C}_\theta(e) = \odot @ \mathcal{C}_\theta(e_1) @ \cdots @ \mathcal{C}_\theta(e_j)$;
 - if $h = \mathbf{c}(\vec{e}')$ is a constructor form (necessarily $j \geq 1$): $\mathcal{C}_\theta(e) = \widehat{\mathbf{c}}(\mathcal{C}_\theta(\vec{e}')) @ \mathcal{C}_\theta(e_1) @ \cdots @ \mathcal{C}_\theta(e_j)$;
 - if h is a λ -abstraction, case or fix (a *complex head*): let u be its *value translation* defined below; then $\mathcal{C}_\theta(e) = u @ \mathcal{C}_\theta(e_1) @ \cdots @ \mathcal{C}_\theta(e_j)$.
- Value translations of complex heads h , with $\vec{w} = \text{FV}(h)$ (ordered) and pinned closure $\vec{w}^\theta$. Every naming function is applied, here and in Definition 5.4, to the key $(h, \theta | \text{FV}(h))$ of Definition 5.1; we suppress the second component. The recursive B-calls below carry $\theta | \text{FV}(h)$ — templates of variables not free in h are never consulted in the sub-derivation — extended at a fix by the new binding; in the displayed calls, θ stands for this restriction:
 - $h = \lambda x. e_0$: let $f = \Lambda(h)$, an $|\vec{w}|$ -ary symbol with witness $e_f = h$; run $\text{B}_\theta(f(\vec{w}^\theta); h)$; the value is $f(\vec{w}^\theta)$.

- $h = \text{case}(s; \vec{br})$: let $g = X(\lambda z. \text{case}(z; \vec{br}))$ for fresh z , of arity $1 + |\vec{w}'|$ with $\vec{w}' = \text{FV}(\vec{br})$, with witness $e_g = \lambda z. \text{case}(z; \vec{br})$; run $B_\theta(g(z, \vec{w}'^\theta); \text{case}(z; \vec{br}))$; the value is $g(\mathcal{C}_\theta(s), \vec{w}'^\theta)$.
- $h = \text{fix } g_0. \lambda z_1 \dots \lambda z_m. e_0$ (maximal prefix): let $f = \Phi(h)$ of arity $|\vec{w}| + m$ with witness $e_f = h$, and $f^0 = \Phi^0(h)$ of arity $|\vec{w}|$ with witness $e_{f^0} = h$; emit the pointer axiom (over fresh variables $\vec{x}, \vec{z}$) $\forall \vec{x} \vec{z}. f^0(\vec{x}) @ z_1 @ \dots @ z_m \approx f(\vec{x}, \vec{z})$; run $B_{\theta'}(f(\vec{w}^\theta, \vec{z}); e_0)$ where $\theta' = \theta[g_0 \mapsto (f, \vec{w}^\theta, m)]$; the value is $f^0(\vec{w}^\theta)$.

The case analysis is exhaustive, so $\mathcal{C}_\theta$ is total: every $\lambda_\square$ term is an atomic form or an application spine, and the head of a spine — not itself an application — is θ -bound, a constant, an ordinary variable, $\square$, a constructor form, or complex.

Remark 5.3 (The two degenerate head clauses). The $\square$ -headed clause is not a safety margin: such spines are reachable from well-typed sources. The definition

$$c := \lambda p. \perp. \lambda x. \text{nat}. \text{efq}(p; \text{nat} \rightarrow \text{nat}) \ x : \perp \rightarrow \text{nat} \rightarrow \text{nat}$$

erases to $\lambda x. \square x$, whose compilation reaches rule (5) of Definition 5.4 with body $\square x$ and emits $\forall x. \widehat{c}(x) \approx \odot @ x$ — dead code applied, denoting the junk class $[\square \hat{x}]$ (Lemma 6.11). Constructor-headed spines $c(\vec{e}') e_1$, by contrast, we believe unreachable from erasures of well-typed CIC_{ix}^- terms — by rigidity (Lemma 3.12) a constructor term never has a Π -type — but $\mathcal{C}$ is also invoked on terms embedded in goals, premises and specifications, so relying on unreachability would require maintaining a typing invariant across every invocation site. The clause costs one line and completes the head analysis of $\mathcal{C}_\theta$. Together with the pinning of recursive closures — which likewise rests on no unreachability belief; the closure configurations are reachable from everyday programs (Remark 5.7) — it makes every totality and well-formedness statement in this section (Lemmas 5.5 and 5.6, the totality of $\mathcal{G}$ and $\mathcal{F}$ in Definition 5.10) unconditional.

Definition 5.4 (Body compilation B). $B_\theta(L; e)$ takes a first-order term L (the *left-hand side*), whose variables include $\text{FV}(e)$ minus the θ -bound ones together with all variables of the closure terms stored in θ , and a $\lambda_\square$ term e , and emits axioms; throughout, $\forall. \phi$ denotes the universal closure of ϕ (all free first-order variables quantified — ordinary variables only, by Lemma 5.6):

1. If $e = \lambda x. e'$: run $B_\theta(L @ x; e')$.
2. If $e = \text{case}(x; \{c_i(\vec{z}_i) \Rightarrow b_i\}_i)$ with x a variable of L : for each i , with $\vec{z}_i$ fresh, run $B_{\theta_i}(L[\widehat{c}_i(\vec{z}_i)/x]; b_i[c_i(\vec{z}_i)/x])$, where θ_i applies the same substitution $[\widehat{c}_i(\vec{z}_i)/x]$ to every closure term stored in θ . The template

map is refined *together with* L : a stored closure term may mention x (the closure of an enclosing fix-lift, or of a pointer value pinned into L), and the sub-derivation's emissions consult the stored templates, which must continue to denote the values of the corresponding subterms of the refined left-hand side (Remark 5.7).

3. If $e = \text{case}(s; \vec{br})$ with s not a variable of L : let $g = X(\lambda z. \text{case}(z; \vec{br}))$ with closure $\vec{w}' = FV(\vec{br})$ and witness as in Definition 5.2; emit $\forall. L \approx g(\mathcal{C}_\theta(s), \vec{w}'^\theta)$, and run $B_\theta(g(z, \vec{w}'^\theta); \text{case}(z; \vec{br}))$ for fresh z , with θ restricted as in Definition 5.2.
4. If $e = \text{fix } g_0.e'$: compute the value translation $u = f^0(\vec{w})$ of e as in Definition 5.2 (which also compiles e); emit $\forall. L \approx u$.
5. Otherwise: emit $\forall. L \approx \mathcal{C}_\theta(e)$.

For a term definition $c := t : \tau$ of $\mathbb{E}$, compilation is initialised on $e = \mathcal{E}(t)$:

- if $e = \text{fix } g_0. \lambda z_1 \dots \lambda z_m. e_0$ (maximal prefix): set $a_c := m$ and run $B_{[g_0 \mapsto (\hat{c}, (), m)]}(\hat{c}(\vec{z}); e_0)$ — the pointer value of g_0 is the constant $\hat{c}^0$, which exists since $m \geq 1$ (the guard argument of a fix is informative);
- otherwise, with $e = \lambda z_1 \dots \lambda z_m. e_0$ (maximal prefix, possibly $m = 0$): set $a_c := m$ and run $B_\emptyset(\hat{c}(\vec{z}); e_0)$.

The witness of $\hat{c}$ and of $\hat{c}^0$ is the $\lambda_\square$ constant c ; if $a_c > 0$, the pointer axiom $\forall \vec{z}. \hat{c}^0 @ z_1 @ \dots @ z_{a_c} \approx \hat{c}(\vec{z})$ is emitted.

Lemma 5.5 (Compilation terminates). *The procedure of Definitions 5.2 and 5.4 terminates on every input and, with the naming functions of Definition 5.1, emits a finite set of axioms for a given environment and goal.*

Proof. As in [Cza26]: order calls by the pair (number of case/ λ /fix nodes, node count) lexicographically, measured on the $\lambda_\square$ argument. Rule (2) consumes the outer case node and passes to a branch with a variable-for-pattern replacement that creates no case, λ or fix nodes (the refinement of L and of the stored templates concerns first-order data, outside the measure); rule (3) replaces a non-variable scrutinee by a variable; rules (1), (4), (5) and all $\mathcal{C}$ recursions strictly decrease the pair. Finiteness follows since each emitted axiom is attached to a symbol, each symbol to a key in the finite compilation tree, and hash-consing merges liftings with equal keys. $\square$

Lemma 5.6 (Emitted axioms are sentences over ordinary variables). *Every axiom emitted by Definitions 5.2 and 5.4 is the universal closure of a formula over ordinary first-order variables: θ -bound variables occur in no emitted formula.*

Proof. By induction on the tree of procedure calls, with the invariant that the left-hand side L and all closure terms stored in θ are first-order terms over ordinary variables. The invariant holds at roots (the root template's closure list is empty) and is preserved: rule (1) appends a fresh ordinary variable; rule (2) substitutes a constructor pattern over fresh ordinary variables into L and into θ ; the lift clauses of Definition 5.2 store and use *pinned* closures, in which every θ -bound member has been replaced by its pointer value — a term over the stored closure of its template, ordinary by the invariant — and the pointer axioms range over fresh ordinary variables. Emitted right-hand sides are images of $\mathcal{C}_\theta$, which replaces every occurrence of a θ -bound variable through the template clause (θ -bound variables occur in compiled terms only as spine heads, as noted above Definition 5.2) by an application of the template symbol to stored closure terms. So both sides of every emitted equation lie over ordinary variables, and $\forall$ binds exactly those. $\square$

Remark 5.7 (Recursive closures: reachable, pinned, keyed). Pinning, the template refinement of rule (2), and the naming keys of Definition 5.1 are the three coordinated parts of one discipline: the value of a recursion variable at a lifting site is first-order data, not a quantifiable variable. None of it may rest on unreachability claims — the configurations are exercised by everyday programs. Two instances over the running environment (nat , bool , $A := \text{nat}$, vec , and $\text{iszero} := \lambda m:\text{nat}. \text{case}(m; \{\text{O} \Rightarrow \text{true} \mid \text{S}(p) \Rightarrow \text{false}\})$).

First, *recursion under a computed case*. The definition

$$c := \text{fix } f:(\Pi n:\text{nat}. \text{vec}(n) \rightarrow \text{nat}). \lambda n v. \text{case}(v; \{\text{vnil} \Rightarrow \text{O} \mid \text{vcons}(k, a, w) \Rightarrow \text{case}(\text{iszero } k; \{\text{true} \Rightarrow f k w \mid \text{false} \Rightarrow a\})\})$$

— “recurse if a computed test succeeds” — is well typed, guarded and shallow. Its vcons branch reaches rule (3) with $f \in \text{FV}(\vec{br})$, so the case-lift g carries the recursion variable in its closure; with the pinned closure $(\widehat{c}^0, k, w, a)$ the emissions are

$$\begin{aligned} \forall n k a w. \widehat{c}(n, \widehat{\text{vcons}}(k, a, w)) &\approx g(\widehat{\text{iszero}}(k), \widehat{c}^0, k, w, a), \\ \forall k w a. g(\widehat{\text{true}}, \widehat{c}^0, k, w, a) &\approx \widehat{c}(k, w), \quad \forall k w a. g(\widehat{\text{false}}, \widehat{c}^0, k, w, a) \approx a, \end{aligned}$$

valid in $\mathfrak{M}$ (Lemma 6.12): the closure place of f holds the *term* $\widehat{c}^0$, whose denotation is the class of c — exactly the value the witness of g substitutes there. Had the closure been the raw free-variable list, that place would have been universally quantified, and the middle equation would be *false* in $\mathfrak{M}$: instantiating the f -place at the junk class $[\square]$ and w at $[\text{vnil}]$, the left side denotes the stuck class $[\square \hat{e}_k \text{vnil}]$ while the right side denotes $[c \hat{e}_k \text{vnil}] = [\text{O}]$, distinct normal forms (Theorem 4.3). Worse, a naming key that does not see θ identifies the lifted term with an α -copy in which the recursion variable is an *ordinary* variable, attaching to one symbol the equations of

both readings; with a few further harmless definitions the emitted theory then derives $\widehat{\mathbf{O}} \approx \widehat{\mathbf{S}}(\widehat{\mathbf{O}})$, hence $\perp$. Pinning removes the quantified place; the keys remove the collision.

Second, *template refinement*. In

$$c' := \lambda b:\text{bool}. \text{fix } f:(\Pi n:\text{nat}. \text{vec}(n) \rightarrow \text{nat}). \lambda n v. \text{case}(v; \{\text{vnil} \Rightarrow \mathbf{O} \mid \text{vcons}(k, a, w) \Rightarrow \text{case}(b; \{\text{true} \Rightarrow f \ k \ w \mid \text{false} \Rightarrow a\})\})$$

the inner fix-lift Φ has closure (b) and template $(\Phi, (b), 2)$, and the branch case on b — a variable of the left-hand side — is refined by rule (2), whose substitution acts on the stored template as well; the **true** leaf therefore emits $\forall n \ k \ a \ w. \Phi(\widehat{\text{true}}, n, \widehat{\text{vcons}}(k, a, w)) \approx \Phi(\widehat{\text{true}}, k, w)$ — the recursion correctly specialised to the refined closure. An unrefined template would emit $\dots \approx \Phi(b, k, w)$ with b universally quantified but absent from the left-hand side, false in $\mathfrak{M}$ at junk values of b by the same witness computation as above. Recursive closures under λ — passing the recursive call to a functional argument, as in $\text{vcons}(k, a, w) \Rightarrow u(\lambda x:\text{nat}. f \ k \ w \ u)$ — route f through a λ -lift's closure, and pinning and keying act identically. The discipline formalises what practical λ -lifters, the implemented translation included, have always done: the symbol introduced for a fixpoint is substituted for its recursion variable before closures are computed.

Remark 5.8 (Shape of the equations). All axioms emitted by compilation are universally quantified equations $L \approx R$ whose left-hand sides have the form

$$f(\vec{\pi}) @ \pi'_1 @ \dots @ \pi'_k \quad (k \geq 0),$$

where f is a compiled symbol and each of $\vec{\pi}, \pi'_1, \dots, \pi'_k$ is generated by the grammar

$$\pi ::= x \mid \widehat{\mathbf{c}}(\vec{\pi}) \mid p(\vec{\pi})$$

— a variable, a constructor pattern, or a *pinned closure term* headed by a pointer symbol p (rule (2) may instantiate variables inside pinned terms, which is how constructor patterns enter them). Constructor patterns occur in @-positions too: rule (2) of Definition 5.4 may refine a variable that rule (1) appended — a branch body that is a λ -abstraction followed by a case on the new variable — producing left-hand sides such as $f(\widehat{\mathbf{O}}) @ \widehat{\mathbf{c}}(\vec{z})$.

The left-hand sides are pairwise non-overlapping. The equations emitted for one head symbol are the leaves of a *refinement tree* in which rule (1) wraps the current left-hand side L in one @-application of a fresh variable, rule (2) splits one variable of L into all constructors of the scrutinised datatype (simultaneously at every occurrence, pinned subterms included), and rules (3)–(5) emit and stop. Two distinct leaves diverge at a rule-(2) split of some variable x of a common intermediate left-hand side L_0 ; in the two final left-hand sides, L_0 persists as the subterm at a fixed depth on the left @-spine (later steps only wrap it in further @-applications or instantiate its

variables), and any fixed occurrence of x in L_0 — a position that stays rigid in the persisted instance, whether at an argument place or inside a pinned term, since constructor and pointer symbols have fixed arities — carries an instance of $\widehat{c}(\dots)$ in one leaf and of $\widehat{d}(\dots)$ in the other with $c \neq d$, so no unifier exists between either final left-hand side and the other or a left-spine subterm of the other. Left-hand sides with distinct head symbols cannot unify at all (the head symbol sits at the bottom of the left spine), and pinned subterms create no new overlaps: no left-hand side has a pointer symbol at its spine bottom — each pointer axiom’s left-hand side is an @-spine over $f^0(\vec{x})$ with at least one extension argument — so the p -headed subterms sitting at argument places of left-hand sides unify with no left-hand side and no left-spine subterm of one, while each pointer axiom remains the sole equation of its head f^0 , with an all-variable left-hand side that cannot unify with a proper left-spine subterm of a renamed copy of itself (@-depths differ).

One property is weaker than in the companion paper: left-linearity may fail, since a variable may occur both inside a pinned closure term and at another position of L (the closure of an enclosing fix-lift and the closure of a lift within its body may share a variable). On the fragment without recursive closures — no pinned terms, in particular whenever no fix body lifts a complex subterm mentioning its recursion variable — the left-hand sides are linear and the oriented equations form an orthogonal rewrite system [BN98]; in general they form a non-overlapping one whose duplicated variables are confined to pinned positions. Nothing about indexed families disturbs any of this: the patterns are patterns of *informative* constructor arguments, and index information never reaches $\lambda_\square$.

Example 5.9 (Compilation over families). For the definitions of Example 3.16 (writing hatted symbols without hats):

$$\mathcal{E}(\text{vhead}) = \lambda n v. \text{case}(v; \{\text{vnil} \Rightarrow \square \mid \text{vcons}(k, a, w) \Rightarrow a\}),$$

so $a_{\text{vhead}} = 2$ and rule (2) emits

$$\forall n. \text{vhead}(n, \text{vnil}) \approx \odot, \quad \forall n k a w. \text{vhead}(n, \text{vcons}(k, a, w)) \approx a,$$

and likewise $\text{vtail}(n, \text{vnil}) \approx \odot$ and $\forall n k a w. \text{vtail}(n, \text{vcons}(k, a, w)) \approx w$. Note three things. (i) The equations are unguarded and unconditional: no premise relates n to k . (ii) The first equation of each pair describes a *statically impossible* branch (vnil cannot inhabit $\text{vec}(S n)$); it is nevertheless *valid* in the model (Lemma 6.12), because the $\lambda_\square$ case fires on any constructor form regardless of typability — the branch body is the image of the $\perp$ -elimination, dead code. Section 8.3 analyses the index-premised and pruned variants. (iii) For a recursive example, $\text{vmap}_g := \text{fix}f: (\Pi n: \text{nat}. \text{vec}(n) \rightarrow \text{vec}(n)). \lambda n v. \text{case}(v; \dots; \dots)$ (mapping a previously defined $g : A \rightarrow A$

over a vector) compiles to

$$\begin{aligned} \forall n. \text{vmap}_g(n, \text{vnil}) &\approx \text{vnil}, \\ \forall n \, k \, a \, w. \text{vmap}_g(n, \text{vcons}(k, a, w)) &\approx \text{vcons}(k, g(a), \text{vmap}_g(k, w)), \end{aligned}$$

orientable rewrite rules in which the recursive call's index argument is the pattern variable k — the equations a human would state.

5.3 Specification extraction

Definition 5.10 (Guard formulas and proposition translation). For every shallow set type σ and first-order term u , the *guard formula* $\mathcal{G}_\sigma(u)$, and for every shallow proposition φ , the first-order formula $\mathcal{F}(\varphi)$, are defined by mutual structural recursion:

$$\begin{aligned} \mathcal{G}_{I(\vec{a})}(u) &= \widehat{I}(u, \mathcal{C}(\mathcal{E}(a_1)), \dots, \mathcal{C}(\mathcal{E}(a_m))) \\ \mathcal{G}_{\Pi x:\sigma.\tau}(u) &= \forall x. \mathcal{G}_\sigma(x) \rightarrow \mathcal{G}_\tau(u @ x) \\ \mathcal{G}_{\varphi \rightarrow \tau}(u) &= \mathcal{F}(\varphi) \rightarrow \mathcal{G}_\tau(u) \\ \mathcal{G}_{\{x:\sigma|\varphi\}}(u) &= \mathcal{G}_\sigma(u) \wedge \mathcal{F}(\varphi)[u/x] \\ \mathcal{F}(\top) &= \top, \quad \mathcal{F}(\perp) = \perp \\ \mathcal{F}(\varphi \circledast \psi) &= \mathcal{F}(\varphi) \circledast \mathcal{F}(\psi) \quad (\circledast \in \{\wedge, \vee, \rightarrow\}) \\ \mathcal{F}(\forall x:\sigma.\varphi) &= \forall x. \mathcal{G}_\sigma(x) \rightarrow \mathcal{F}(\varphi) \\ \mathcal{F}(\exists x:\sigma.\varphi) &= \exists x. \mathcal{G}_\sigma(x) \wedge \mathcal{F}(\varphi) \\ \mathcal{F}(a =_\sigma b) &= \mathcal{C}(\mathcal{E}(a)) \approx \mathcal{C}(\mathcal{E}(b)) \\ \mathcal{F}(J(\vec{a})) &= \widehat{J}(\mathcal{C}(\mathcal{E}(a_1)), \dots, \mathcal{C}(\mathcal{E}(a_k))) \end{aligned}$$

where in the binder clauses the bound source variable is reused as the bound first-order variable, and $\mathcal{C} = \mathcal{C}_\emptyset$ is the term translation of Definition 5.2, whose complex-head clauses may emit further axioms. $\mathcal{G}$ and $\mathcal{F}$ are total on $\mathfrak{S}$ and structurally recursive ($\mathcal{C}$ is total on $\lambda_\square$ terms, Definition 5.2); no normalisation of the source is involved.

The new clause is the first: the guard of a family occurrence carries the *translated index terms* as arguments of the guard atom. Index terms are ordinary terms — they may contain defined functions ($\text{vec}(\text{add}(m, n))$ arises from $\text{vec}(\text{add } m \, n)$), matches (lifted by $\mathcal{C}$'s complex-head clauses), and proof subterms (erased, Remark 4.6). Nothing is solved, nothing is normalised, nothing is encoded: the constraint material of the family arrives at the prover as first-order terms in predicate arguments, available to superposition and to nothing else.

Definition 5.11 (Specification of a constant). For a term definition $c := t : \tau$ with $\tau \in \mathfrak{S}$, the *specification axiom* $\mathcal{S}(c)$ is $S(u_0; \tau)$ where $u_0 = \widehat{c}$ if $a_c = 0$

and otherwise $u_0 = \widehat{c}()$ is the empty accumulator, and

$$\begin{aligned} S(u; \Pi x:\sigma.\tau') &= \forall x. \mathcal{G}_\sigma(x) \rightarrow S(u \cdot x; \tau') \\ S(u; \varphi \rightarrow \tau') &= \mathcal{F}(\varphi) \rightarrow S(u; \tau') \\ S(u; \tau') &= \mathcal{G}_{\tau'}(\bar{u}) \quad \text{if } \tau' \text{ is a datatype occurrence or refinement type,} \end{aligned}$$

where $u \cdot x$ appends x to the accumulator if fewer than a_c arguments have been accumulated and otherwise applies with $@$, and $\bar{u}$ closes the accumulator (an under-applied accumulator is rendered as the pointer chain, as in [Cza26]).

Example 5.12. $\mathcal{S}(\text{vtail})$ is

$$\forall n. \widehat{\text{nat}}(n) \rightarrow \forall v. \widehat{\text{vec}}(v, \widehat{S}(n)) \rightarrow \widehat{\text{vec}}(\widehat{\text{vtail}}(n, v), n),$$

the statement of vtail 's type as a first-order formula: the index arithmetic of the family has become term structure inside guard atoms. For vmap_g the axiom is $\forall n. \widehat{\text{nat}}(n) \rightarrow \forall v. \widehat{\text{vec}}(v, n) \rightarrow \widehat{\text{vec}}(\widehat{\text{vmap}}_g(n, v), n)$ — index preservation, for free, from the type.

5.4 Structural axioms: the ford of a family

Definition 5.13 (Emitted theory). The first-order theory $\Delta(\mathbb{E}, G)$ consists of the following axioms.

1. For each datatype $I : (\vec{\sigma}) \rightarrow \text{Set}$ with constructors $\mathbf{c}_i : \forall \vec{y}:\vec{A}_i. \varphi_{i1} \Rightarrow \dots \Rightarrow \varphi_{i\ell_i} \Rightarrow I(\vec{t}_i)$:

- (*constructor typing*) for each i ,

$$\forall \vec{y}. \bigwedge_j \mathcal{G}_{A_{ij}}(y_j) \rightarrow \bigwedge_l \mathcal{F}(\varphi_{il}) \rightarrow \widehat{I}(\widehat{\mathbf{c}}_i(\vec{y}), \mathcal{C}(\mathcal{E}(t_{i1})), \dots, \mathcal{C}(\mathcal{E}(t_{im})));$$

- (*inversion, forded*)

$$\begin{aligned} \forall w \vec{z}. \widehat{I}(w, \vec{z}) &\rightarrow \bigvee_i \exists \vec{y}. \bigwedge_j \mathcal{G}_{A_{ij}}(y_j) \wedge \bigwedge_l \mathcal{F}(\varphi_{il}) \wedge \\ &w \approx \widehat{\mathbf{c}}_i(\vec{y}) \wedge \bigwedge_{h=1}^m z_h \approx \mathcal{C}(\mathcal{E}(t_{ih})); \end{aligned}$$

- (*injectivity*) $\forall \vec{y} \vec{y}'. \widehat{\mathbf{c}}_i(\vec{y}) \approx \widehat{\mathbf{c}}_i(\vec{y}') \rightarrow \bigwedge_j y_j \approx y'_j$ for each i with $r_i > 0$;
- (*discrimination*) $\forall \vec{y} \vec{y}'. \neg(\widehat{\mathbf{c}}_i(\vec{y}) \approx \widehat{\mathbf{c}}_i(\vec{y}'))$ for all $i \neq j$.

2. For each inductive predicate J with clauses $\mathbf{k}_j : \forall \vec{x}:\vec{\tau}_j. \varphi_{j1} \rightarrow \dots \rightarrow \varphi_{j\ell_j} \rightarrow J(\vec{u}_j)$:

- (*clause*) $\forall \vec{x}. \bigwedge_i \mathcal{G}_{\tau_{ji}}(x_i) \rightarrow \mathcal{F}(\varphi_{j1}) \rightarrow \dots \rightarrow \mathcal{F}(\varphi_{j\ell_j}) \rightarrow \widehat{J}(\mathcal{C}(\mathcal{E}(\vec{u}_j)))$;

- (*case analysis*) $\forall \vec{z}. \widehat{J}(\vec{z}) \rightarrow \bigvee_j \exists \vec{x}. \bigwedge_i \mathcal{G}_{\tau_{ji}}(x_i) \wedge \bigwedge_i \mathcal{F}(\varphi_{ji}) \wedge \bigwedge_h z_h \approx \mathcal{C}(\mathcal{E}(u_{jh}))$.
- 3. For each term definition $c := t : \tau$: the equations, pointer axioms and auxiliary axioms emitted by compiling $\mathcal{E}(t)$ (Definition 5.4), and the specification axiom $\mathcal{S}(c)$ if $\tau \in \mathfrak{S}$.
- 4. For each lemma $c := \pi : \varphi$ with $\varphi \in \mathfrak{S}$: the *premise axiom* $\mathcal{F}(\varphi)$.
- 5. The auxiliary axioms emitted by the computations of $\mathcal{F}(G)$ and of the guard formulas and $\mathcal{F}$ -images of declarations (via the complex-head clauses of $\mathcal{C}$ on embedded terms).

The *translation of the problem* $(\mathbb{E}, G)$ is the pair $(\Delta(\mathbb{E}, G), \mathcal{F}(G))$.

The forced inversion axiom is the first-order image of the forced declaration: it quantifies over an *arbitrary* index tuple $\vec{z}$ — the whole aperture, in McBride’s phrase — and each disjunct carries, beside the constructor equation and payload, the index equations $z_h \approx \mathcal{C}(\mathcal{E}(t_{ih}))$ that recover specificity. The predicate case-analysis axiom (present already in the companion paper) is the same shape one level down — predicates have no element, so the constructor equation is absent; this is why indexed inductive *predicates* required no new mechanism there, while indexed *datatypes* do. Note also the division of labour inherited from the companion paper: exhaustiveness lives *only* in the guarded inversion axiom; the defining equations of item 3 make no such claim.

Example 5.14 (The running families, translated). For **vec** (constructor typing, then inversion):

$$\begin{aligned} & \widehat{\text{vec}}(\widehat{\text{vnil}}, \widehat{\text{O}}), \\ & \forall n a v. \widehat{\text{nat}}(n) \rightarrow \mathcal{G}_A(a) \rightarrow \widehat{\text{vec}}(v, n) \rightarrow \widehat{\text{vec}}(\widehat{\text{vcons}}(n, a, v), \widehat{\text{S}}(n)), \\ & \forall w z. \widehat{\text{vec}}(w, z) \rightarrow (w \approx \widehat{\text{vnil}} \wedge z \approx \widehat{\text{O}}) \vee \\ & \quad \exists n a v. (\widehat{\text{nat}}(n) \wedge \mathcal{G}_A(a) \wedge \widehat{\text{vec}}(v, n) \wedge \\ & \quad w \approx \widehat{\text{vcons}}(n, a, v) \wedge z \approx \widehat{\text{S}}(n)), \end{aligned}$$

plus injectivity for **vcons** and discrimination $\widehat{\text{vnil}} \not\approx \widehat{\text{vcons}}(n, a, v)$. For rfl_{φ_0} :

$$\mathcal{F}(\varphi_0) \rightarrow \widehat{\text{rfl}}(\widehat{\text{RT}}, \widehat{\text{true}}), \quad \mathcal{F}(\neg \varphi_0) \rightarrow \widehat{\text{rfl}}(\widehat{\text{RF}}, \widehat{\text{false}}),$$

$$\begin{aligned} \forall w b. \widehat{\text{rfl}}(w, b) \rightarrow & (\mathcal{F}(\varphi_0) \wedge w \approx \widehat{\text{RT}} \wedge b \approx \widehat{\text{true}}) \vee \\ & (\mathcal{F}(\neg \varphi_0) \wedge w \approx \widehat{\text{RF}} \wedge b \approx \widehat{\text{false}}), \end{aligned}$$

with $\widehat{\text{RT}} \not\approx \widehat{\text{RF}}$: the propositional payloads and the index equations sit side by side in the disjuncts — “index equations are just more propositional

payload.” For $\widehat{\text{bnd}}$, inversion is $\forall w z. \widehat{\text{bnd}}(w, z) \rightarrow \exists n k. \widehat{\text{nat}}(n) \wedge \widehat{\text{nat}}(k) \wedge \widehat{\text{lt}}(k, n) \wedge w \approx \widehat{\text{B}}(n, k) \wedge z \approx n$, which Lemma 5.17 simplifies by solving $n := z$. For the predicate $\widehat{\text{le}}$, the case-analysis axiom is

$$\forall z_1 z_2. \widehat{\text{le}}(z_1, z_2) \rightarrow \left(\exists n. (\widehat{\text{nat}}(n) \wedge z_1 \approx n \wedge z_2 \approx n) \right) \vee \left(\exists n m. (\widehat{\text{nat}}(n) \wedge \widehat{\text{nat}}(m) \wedge \widehat{\text{le}}(n, m) \wedge z_1 \approx n \wedge z_2 \approx \widehat{\text{S}}(m)) \right),$$

the classical inversion principle.

Example 5.15 (Empty instances refute for free). No axiom asserts any instance inhabited; empty instances are refutable. From $\widehat{\text{fin}}(w, \widehat{\text{O}})$, the inversion axiom for $\widehat{\text{fin}}$ yields a disjunction whose every disjunct contains an equation $\widehat{\text{O}} \approx \widehat{\text{S}}(n)$, refuted by discrimination for $\widehat{\text{nat}}$; hence $\Delta(\mathbb{E}, G) \vdash_{\text{FOL}} \neg \exists w. \widehat{\text{fin}}(w, \widehat{\text{O}})$, and a goal $\mathcal{F}(\forall x: \text{fin}(\text{O}). \varphi) = \forall w. \widehat{\text{fin}}(w, \widehat{\text{O}}) \rightarrow \mathcal{F}(\varphi)$ is provable outright. The same one-step pattern refutes $\widehat{\text{vec}}(\text{vnil-form}, \widehat{\text{S}}(n))$ -style hypotheses. Contrast the hazard class of selector/rank axioms on degenerate instances (Verus #1366 [The24b]): the theory of Definition 5.13 contains no axiom whose *hypothesis-free* content mentions an arbitrary instance, so empty instances can only make guards false, never derive new equations.

Example 5.16 (A user-declared equality). For eq_σ of Example 3.4, the emitted axioms are the clause $\forall x. \mathcal{G}_\sigma(x) \rightarrow \widehat{\text{eq}}_\sigma(x, x)$ and the case analysis $\forall z_1 z_2. \widehat{\text{eq}}_\sigma(z_1, z_2) \rightarrow \exists x. \mathcal{G}_\sigma(x) \wedge z_1 \approx x \wedge z_2 \approx x$, which Lemma 5.17 simplifies to $\forall z_1 z_2. \widehat{\text{eq}}_\sigma(z_1, z_2) \rightarrow \mathcal{G}_\sigma(z_1) \wedge z_2 \approx z_1$. Thus the declared equality implies the primitive $\approx$ and membership in σ , while $\mathcal{F}(a =_\sigma b) = \mathcal{C}(\mathcal{E}(a)) \approx \mathcal{C}(\mathcal{E}(b))$ is bare untyped equality; both directions of the correspondence are first-order derivable under the guard of σ . A user-written transport through eq_σ — the $\text{sca}_{\text{eq}_\sigma}(\cdot; \cdot)$ former — erases (Definition 4.5), so a definition $c := \lambda x y: \sigma. \lambda e: \text{eq}_\sigma(x, y). \lambda w: \tau[x]. \text{sca}_{\text{eq}_\sigma}(e; \lambda z. \tau[z_2]; .w)$ compiles to the single unguarded equation $\forall x y w. \widehat{c}(x, y, w) \approx w$ together with its specification axiom

$$\forall x. \mathcal{G}_\sigma(x) \rightarrow \forall y. \mathcal{G}_\sigma(y) \rightarrow \widehat{\text{eq}}_\sigma(x, y) \rightarrow \forall w. \mathcal{G}_{\tau[x]}(w) \rightarrow \mathcal{G}_{\tau[y]}(\widehat{c}(x, y, w))$$

(displayed in the order Definition 5.11 emits: each Π -binder contributes its quantifier and guard in place, each propositional premise its $\mathcal{F}$ -image in place). This closes, in the idealised setting, the implementation gap in which a user-written match on an equality proof receives no definitional axiom at all while a fixed list of library transport constants is special-cased (Section 1.4); the soundness of the unguarded equation is exactly the business of Section 8.5.

5.5 Forced-argument simplification

The inversion and case-analysis axioms frequently contain existentially quantified variables that some index equation solves outright — the image of

McBride’s coalescence [McB99] and of the forced-argument analysis of Brady et al. [BMM04, SM19]. Eliminating them is a first-order equivalence, not a soundness matter:

Lemma 5.17 (Solving forced equations). *Let $\Psi(\vec{y})$ be a first-order formula, y_j one of the variables, and z a term not containing any y . Then*

$$\vdash_{\text{FOL}} \left(\exists \vec{y}. z \approx y_j \wedge \Psi(\vec{y}) \right) \leftrightarrow \exists \vec{y} \setminus y_j. \Psi(\vec{y})[z/y_j],$$

and the same holds with the equation oriented $y_j \approx z$, and under any prefix of universal quantifiers and implications. Consequently, replacing any disjunct of an inversion or case-analysis axiom by its solved form yields a logically equivalent axiom, and Theorem 7.9 is unaffected by performing any number of such replacements.

Proof. Standard: left to right, instantiate the conjunct and rewrite; right to left, witness $y_j := z$. Both directions are derivations in pure first-order logic with equality, so the replacement preserves logical equivalence of the whole axiom (replacement of equivalents), and a theory with an axiom replaced by an equivalent proves the same formulas. $\square$

Remark 5.18. After solving, the bnd inversion of Example 5.14 becomes $\forall w z. \widehat{\text{bnd}}(w, z) \rightarrow \exists k. \widehat{\text{nat}}(z) \wedge \widehat{\text{nat}}(k) \wedge \widehat{\text{lt}}(k, z) \wedge w \approx \widehat{\text{B}}(z, k)$ — the refinement-style unfolding with no existential residue on the index. Which equations are solvable is exactly the forced-argument analysis; the translation may apply Lemma 5.17 greedily (each application removes one quantifier) and the model-side soundness statement is untouched. Unsolved equations — non-variable patterns like $z \approx \widehat{\text{S}}(n)$, non-linear occurrences, defined-function images like $z \approx \widehat{\text{add}}(m, n)$ — simply remain, as they must (Remark 5.19).

Remark 5.19 (Green slime, stated not solved). Nothing in Definition 5.13 requires index terms to be constructor-headed. A family stree with node $: \forall(x y:\text{nat})(l:\text{stree}(x))(r:\text{stree}(y)). \text{stree}(\text{S}(\text{add } x \ y))$ contributes the inversion disjunct $\dots \wedge z \approx \widehat{\text{S}}(\widehat{\text{add}}(x, y))$, next to which the compilation of add has already placed its two defining equations. An ATP confronted with a hypothesis $\widehat{\text{stree}}(w, \widehat{\text{S}}(\widehat{\text{O}}))$ derives $\widehat{\text{S}}(\widehat{\text{add}}(x, y)) \approx \widehat{\text{S}}(\widehat{\text{O}})$, then $\widehat{\text{add}}(x, y) \approx \widehat{\text{O}}$ by injectivity of $\widehat{\text{S}}$, and can close or constrain the branch by superposition with add ’s equations and nat ’s inversion — reasoning that elaboration-time unification, for which add is not a rigid symbol, cannot perform [McB14, GMM06]. The translation performs *specialization by unification* nowhere; the prover performs it as ordinary equational inference, and the soundness proof (Theorem 7.8) is entirely indifferent to the shape of index terms. No decidability claim is made or needed: with defined symbols in the signature the first-order theory of the term algebra is Π_1^1 -complete [KRV17, Thm. 2], and the translation targets refutational usefulness, not decision.

Remark 5.20 (Relation to the implementation). Each axiom family has a direct site in the CoqHammer pipeline: item 1’s constructor typing and inversion generalize the existing declaration-level axioms from inductive predicates to indexed datatypes (the current translation emits them only behind an inhabitation atom); item 3’s equations are the per-constructor case equations of the extraction branch, now specified to be emitted for indexed scrutinees unchanged; the guard clause of Definition 5.10 is the removal of the classifier’s indexed-family gate, with the index terms flowing into the guard atom’s argument places. The per-occurrence *expansion* of guard atoms — the implementation’s enum/subset/singleton classifications — is Section 8.1.

6 Semantics

We now build the structures against which soundness is proved: a proof-irrelevant, classical, syntactic semantics of the source, and a first-order structure $\mathfrak{M}$ over the same individuals interpreting $\Sigma_{\mathbb{E}}$. Throughout, $\mathbb{E}$ is a fixed well-formed definitional environment with $\mathfrak{S}$ declarations. The construction follows [Cza26]; the new material is the interpretation of indexed families as sets of (index tuple, element) pairs, the *index reflection* lemma — the semantic content of fording — and element ranks.

6.1 The domain

Definition 6.1 (Domain). Let $\mathcal{D}$ be the set of closed $\lambda_{\square}$ terms modulo $=_{\square}$, written $[e]$ for classes. $\mathcal{D}$ is non-empty ($[\square] \in \mathcal{D}$) and carries the application $d \cdot d' := [e e']$ for representatives, well defined since $=_{\square}$ is a congruence.

$\mathcal{D}$ deliberately contains *junk*: classes such as $[\mathbf{O} \mathbf{O}]$ or $[\text{fix } f.f]$ that are the value of no source term of any type, and — new in the indexed setting — *well-formed constructor forms at the wrong index*, such as $[\mathbf{vnil}]$ considered at index $[\mathbf{S}(\mathbf{O})]$. By Corollary 4.4, distinct constructor forms lie in distinct classes and junk like $[\mathbf{O} \mathbf{O}]$ lies outside every constructor form. Junk is what makes the guard analysis of Section 8 non-trivial; keeping it in the model is what lets the model arbitrate which axioms need guards, and the wrong-index phenomenon is what makes the *index* arguments of guards non-negotiable (Proposition 8.9).

Definition 6.2 (Valuations and term denotation). A *valuation* ρ assigns to finitely many term variables elements of $\mathcal{D}$ and to finitely many predicate variables X (of kind $(\sigma_1, \dots, \sigma_n) \rightarrow \text{Prop}$) functions $\mathcal{D}^n \rightarrow \{0, 1\}$. Proof variables are not assigned anything. For a CIC_{ix}^- term t whose free term variables are assigned by ρ ,

$$\llbracket t \rrbracket_{\rho} := [\mathcal{E}(t)^{\rho}] \in \mathcal{D},$$

where $\mathcal{E}(t)^\rho$ substitutes a representative of $\rho(x)$ for each free variable x of $\mathcal{E}(t)$; this is well defined exactly as in [Cza26] (free variables of $\mathcal{E}(t)$ are free term variables of t ; classes are representative-independent). For an index tuple we write $\llbracket \vec{u} \rrbracket_\rho = (\llbracket u_1 \rrbracket_\rho, \dots, \llbracket u_m \rrbracket_\rho) \in \mathcal{D}^m$.

6.2 Interpretation of types, propositions and predicates

Definition 6.3 (Interpretations). By recursion on the position of declarations in $\mathbb{E}$ and, within each declaration, on the structure of phrases, we define for every set type σ , proposition φ and predicate expression P over a prefix of $\mathbb{E}$, and every valuation ρ covering their free variables: a set $\llbracket \sigma \rrbracket_\rho \subseteq \mathcal{D}$; a truth value $\langle \varphi \rangle_\rho \in \{0, 1\}$; a function $\llbracket P \rrbracket_\rho : \mathcal{D}^n \rightarrow \{0, 1\}$.

Indexed datatypes. For $I : (\sigma_1, \dots, \sigma_m) \rightarrow \mathbf{Set}$ with constructors $\mathbf{c}_i : \forall \vec{y} : \vec{A}_i. \varphi_{i1} \Rightarrow \dots \Rightarrow \varphi_{i\ell_i} \Rightarrow I(\vec{t}_i)$ (r_i informative arguments), define stages $\llbracket I \rrbracket^s \subseteq \mathcal{D}^m \times \mathcal{D}$ by $\llbracket I \rrbracket^0 = \emptyset$ and

$$\begin{aligned} \llbracket I \rrbracket^{s+1} = \Big\{ & \left(\llbracket \vec{t}_i \rrbracket_{\rho_0}, [\mathbf{c}_i(e_1, \dots, e_{r_i})] \right) \mid i \leq M, \rho_0 = [\vec{y}_i \mapsto [\vec{e}]], \\ & [e_j] \in \llbracket A_{ij} \rrbracket_{\rho_0} \text{ for non-recursive } j, \\ & (\llbracket \vec{v}_{ij} \rrbracket_{\rho_0}, [e_j]) \in \llbracket I \rrbracket^s \text{ for recursive } j \text{ with } A_{ij} = I(\vec{v}_{ij}), \\ & \langle \varphi_{il} \rangle_{\rho_0} = 1 \text{ for all } l \Big\}, \end{aligned}$$

which is well defined: non-recursive argument types and all premises are I -free phrases over the environment prefix and the telescope, so their interpretations and truth values are supplied by the outer recursion, and index-term denotations $\llbracket \cdot \rrbracket_{\rho_0}$ are erasure classes, defined independently of any type interpretation. The stages are increasing in s (induction on s , the defining condition being monotone in the stage at recursive positions). Set $\llbracket I \rrbracket = \bigcup_s \llbracket I \rrbracket^s$,¹ and for an occurrence:

$$\llbracket I(\vec{u}) \rrbracket_\rho = \{ d \in \mathcal{D} \mid (\llbracket \vec{u} \rrbracket_\rho, d) \in \llbracket I \rrbracket \}.$$

Other set types (as in [Cza26]):

$$\begin{aligned} \llbracket \Pi x : \sigma. \tau \rrbracket_\rho &= \{ d \in \mathcal{D} \mid \forall e \in \llbracket \sigma \rrbracket_\rho. d \cdot e \in \llbracket \tau \rrbracket_{\rho[x \mapsto e]} \} \\ \llbracket \varphi \rightarrow \tau \rrbracket_\rho &= \{ d \in \mathcal{D} \mid \langle \varphi \rangle_\rho = 1 \implies d \in \llbracket \tau \rrbracket_\rho \} \\ \llbracket \{x : \sigma \mid \varphi\} \rrbracket_\rho &= \{ d \in \llbracket \sigma \rrbracket_\rho \mid \langle \varphi \rangle_{\rho[x \mapsto d]} = 1 \} \end{aligned}$$

¹A reader may expect the invariant $\llbracket I \rrbracket \subseteq \llbracket \sigma_1 \rrbracket \times \dots \times \llbracket \sigma_m \rrbracket \times \mathcal{D}$ — index tuples inhabiting their index types. It is not built into the stages, and it is stated and needed nowhere in the development: index-term denotations are erasure classes, supplied independently of every type interpretation, and no lemma ever requires an index tuple to inhabit anything. This is itself information about the translation: no guard is ever emitted for an index argument place (Definition 5.13), and the model shows why none is missed.

Inductive predicates. For $J : (\sigma_1, \dots, \sigma_k) \rightarrow \text{Prop}$ with clauses $k_j : \forall \vec{x} : \vec{\tau}_j. \varphi_{j1} \rightarrow \dots \rightarrow \varphi_{j\ell_j} \rightarrow J(\vec{u}_j)$, stages $\langle J \rangle^0 = \emptyset \subseteq \mathcal{D}^k$ and

$$\begin{aligned} \langle J \rangle^{s+1} = \Big\{ & \llbracket \vec{u}_j \rrbracket_{\rho_0} \mid j, \rho_0 = [\vec{x} \mapsto \vec{d}], d_i \in \llbracket \tau_{ji} \rrbracket_{\rho_0}, \\ & \langle \varphi \rangle_{\rho_0} = 1 \text{ for non-recursive premises,} \\ & \llbracket \vec{v} \rrbracket_{\rho_0} \in \langle J \rangle^s \text{ for recursive premises } J(\vec{v}) \Big\}, \end{aligned}$$

with $\langle J \rangle = \bigcup_s \langle J \rangle^s$.

Propositions and predicate expressions: exactly the clauses of [Cza26]: classical truth tables for the connectives, $\min / \max$ over $\llbracket \sigma \rrbracket_\rho$ for quantifiers (with $\min \emptyset = 1$, $\max \emptyset = 0$), $\min$ over all $Q : \mathcal{D}^n \rightarrow \{0, 1\}$ for predicate quantification, $\langle a =_\sigma b \rangle_\rho = 1$ iff $\llbracket a \rrbracket_\rho = \llbracket b \rrbracket_\rho$, $\langle J(\vec{a}) \rangle_\rho = 1$ iff $\llbracket \vec{a} \rrbracket_\rho \in \langle J \rangle$, $\langle X(\vec{a}) \rangle_\rho = \rho(X)(\llbracket \vec{a} \rrbracket_\rho)$, $\langle \lambda(\vec{x} : \vec{\sigma}). \varphi \rangle_\rho(\vec{d}) = \langle \varphi \rangle_{\rho[\vec{x} \mapsto \vec{d}]}$.

The recursion is well founded exactly as before: stage constructions refer only to earlier declarations and structurally smaller phrases of their own declaration; all other clauses recurse on proper sub-phrases. The semantics is classical and proof-irrelevant by construction: truth values do not depend on proofs, and $\llbracket \cdot \rrbracket$ ignores proof subterms. The interpretation of equality is untyped (class equality, not mentioning σ), matching the translation clause $\mathcal{F}(a =_\sigma b) = \mathcal{C}(\mathcal{E}(a)) \approx \mathcal{C}(\mathcal{E}(b))$; and the interpretation of a family is index-aware — membership is a relation between an index tuple and an element — matching the $(1+m)$ -ary guard predicate.

Definition 6.4 (Element rank). For d with $(\vec{d}, d) \in \llbracket I \rrbracket$ for some $\vec{d}$, let $\text{rk}_I(d)$ be the least s such that $(\vec{d}, d) \in \llbracket I \rrbracket^s$ for some $\vec{d}$. (By Lemma 6.5(3) the tuple is in fact unique, but the definition does not need this.) For $\langle J \rangle$, ranks of tuples are defined as in [Cza26]: the least s with $\vec{d} \in \langle J \rangle^s$.

Lemma 6.5 (Index reflection). *Let I be a datatype as above and $(\vec{d}, d) \in \llbracket I \rrbracket$. Then:*

1. (Decomposition.) *There are a constructor index i , closed $\lambda_\square$ terms $\vec{e} = e_1, \dots, e_{r_i}$ and the valuation $\rho_0 = [\vec{y}_i \mapsto [\vec{e}]]$ such that $d = [\text{c}_i(\vec{e})]$; $[e_j] \in \llbracket A_{ij} \rrbracket_{\rho_0}$ for every non-recursive j ; $(\llbracket \vec{v}_{ij} \rrbracket_{\rho_0}, [e_j]) \in \llbracket I \rrbracket$ for every recursive j ; $\langle \varphi_{il} \rangle_{\rho_0} = 1$ for every l ; and*

$$\vec{d} = \llbracket \vec{t}_i \rrbracket_{\rho_0}.$$

2. (Uniqueness.) *The index i and the classes $[e_1], \dots, [e_{r_i}]$ (hence ρ_0 , hence all data of (1)) are uniquely determined by d .*
3. (Index uniqueness.) *If also $(\vec{d}', d) \in \llbracket I \rrbracket$ then $\vec{d}' = \vec{d}$.*

4. (Rank descent.) $\text{rk}_I(d)$ is defined, and for every recursive j in the decomposition of (1), $\text{rk}_I([e_j]) < \text{rk}_I(d)$.
5. (Completeness of stages.) Conversely, any data as in (1) — an i , classes $[\vec{e}]$ with the memberships, premises and recursive conditions displayed there — yield $(\llbracket \vec{t}_i \rrbracket_{\rho_0}, [c_i(\vec{e})]) \in \llbracket I \rrbracket$.

Proof. (1) $(\vec{d}, d) \in \llbracket I \rrbracket^{s+1}$ for some s ; unfolding Definition 6.3 gives exactly the displayed data, with the recursive conditions in $\llbracket I \rrbracket^s \subseteq \llbracket I \rrbracket$.

(2) Suppose $d = [c_i(\vec{e})] = [c_{i'}(\vec{e}')]$ arise from two decompositions. By Corollary 4.4(1), $i = i'$ and $[e_j] = [e'_j]$ for all j . Since all conditions of (1) are conditions on the classes (interpretations, truth values and denotations are class functions), the two decompositions coincide.

(3) Apply (1) to $(\vec{d}, d)$ and to $(\vec{d}', d)$; by (2) both decompositions have the same i and ρ_0 , so $\vec{d} = \llbracket \vec{t}_i \rrbracket_{\rho_0} = \vec{d}'$.

(4) Definedness is immediate. Let $s_0 = \text{rk}_I(d)$, witnessed by $(\vec{d}_0, d) \in \llbracket I \rrbracket^{s_0}$; necessarily $s_0 = s + 1$ for some s , and unfolding the stage gives a decomposition whose recursive components satisfy $(\llbracket \vec{v}_{ij} \rrbracket_{\rho_0}, [e_j]) \in \llbracket I \rrbracket^s$, whence $\text{rk}_I([e_j]) \leq s < s_0$. By (2) this decomposition is *the* decomposition of (1).

(5) Each recursive condition $(\llbracket \vec{v}_{ij} \rrbracket_{\rho_0}, [e_j]) \in \llbracket I \rrbracket$ holds at some finite stage; by monotonicity all hold at their maximum s , and the defining clause puts the conclusion pair in $\llbracket I \rrbracket^{s+1}$. $\square$

Part (1) is the *inversion* direction of fording (used for hypothesis-position content: the forded inversion axiom, the case rule); part (5) is the *introduction* direction (constructor typing, goal-position content); part (3) is the semantic fact that in the term model every element carries its indices — Proposition 8.16 derives its first-order shadow; and parts (1)+(3) together are what uniqueness of identity proofs buys in type theory, spent here by the `scase`(; ;) and `cast`(; ;) cases of Lemma 7.6 (see Section 8.5).

Lemma 6.6 (Elementary properties). 1. $\llbracket \sigma \rrbracket_\rho, \langle \varphi \rangle_\rho, \langle P \rangle_\rho$ depend only on the restriction of ρ to the free variables of the phrase, and are unchanged by extending ρ on fresh variables.

2. Every $(\vec{d}, d) \in \llbracket I \rrbracket$ has $(\vec{d}, d) \in \llbracket I \rrbracket^s$ for all $s > \text{rk}_I(d)$; similarly for $\langle J \rangle$.

Proof. (1) routine induction on the phrase (index terms depend on ρ only through their free variables). (2) monotonicity of stages and Lemma 6.5(3): the witnessing tuple at the minimal stage is the unique tuple. $\square$

Lemma 6.7 (Semantic substitution). Let a be a term, π a proof and P a predicate expression, with ρ covering the relevant free variables.

1. $\llbracket t[a/x] \rrbracket_\rho = \llbracket t \rrbracket_{\rho[x \mapsto \llbracket a \rrbracket_\rho]}$, and analogously for substituted types $(\llbracket \sigma[a/x] \rrbracket_\rho = \llbracket \sigma \rrbracket_{\rho[x \mapsto \llbracket a \rrbracket_\rho]})$, propositions and predicate expressions.

2. *Substitution of proofs is invisible:* $\llbracket t[\pi/p] \rrbracket_\rho = \llbracket t \rrbracket_\rho$, $\llbracket \sigma[\pi/p] \rrbracket_\rho = \llbracket \sigma \rrbracket_\rho$, $\langle \varphi[\pi/p] \rangle_\rho = \langle \varphi \rangle_\rho$.
3. $\langle \varphi[P/X] \rangle_\rho = \langle \varphi \rangle_{\rho[X \mapsto \langle P \rangle_\rho]}$, and analogously for types and terms (*invisible*).

Proof. (1) For terms, by Lemma 4.7 as in [Cza26]. For types, propositions and predicates: induction on the phrase; the new case $\llbracket I(\vec{u}) \rrbracket_\rho$ follows from the term statement applied to each index term, since the clause depends on $\vec{u}$ only through $\llbracket \vec{u} \rrbracket_\rho$. (2) Erasure discards the substituted proof (Lemma 4.7); proofs do not occur in any semantic clause — in particular proofs inside *index terms* are invisible, which is the model-theoretic form of Remark 4.6. (3) As in (1). $\square$

Lemma 6.8 (Conversion invariance). *If $t \simeq t'$ then $\llbracket t \rrbracket_\rho = \llbracket t' \rrbracket_\rho$. If $\sigma \simeq \sigma'$ (both well formed) then $\llbracket \sigma \rrbracket_\rho = \llbracket \sigma' \rrbracket_\rho$; if $\varphi \simeq \varphi'$ then $\langle \varphi \rangle_\rho = \langle \varphi' \rangle_\rho$.*

Proof. For terms: $t \simeq t'$ implies $\mathcal{E}(t) =_{\square} \mathcal{E}(t')$ (Lemma 4.8), and $=_{\square}$ is substitutive. For types and propositions, consider a single step and proceed by induction on the structure of the phrase, using Lemma 3.12: the step occurs inside an embedded term (an index term of a datatype occurrence, an equation side, a predicate-atom argument) — covered by the term statement, since the semantic clauses depend on embedded terms only through $\llbracket \cdot \rrbracket_\rho$ — or inside a component type or proposition, covered by the induction hypothesis at every extension of ρ . The general claim follows by induction on the conversion path. $\square$

6.3 The first-order structure $\mathfrak{M}$

Definition 6.9 (The structure $\mathfrak{M}$). $\mathfrak{M}$ is the first-order $\Sigma_{\mathbb{E}}$ -structure with universe $\mathcal{D}$ and:

- $\approx$ interpreted as identity of classes; $@$ as $\cdot$; $\odot$ as $[\square]$;
- $\hat{I}^{\mathfrak{M}} = \{(d, \vec{d}) \mid (\vec{d}, d) \in \llbracket I \rrbracket\}$ for each datatype I (subject first, matching Definition 5.1), and $\hat{J}^{\mathfrak{M}} = \langle J \rangle$ for each inductive predicate J ;
- $\hat{c}^{\mathfrak{M}}(d_1, \dots, d_r) = [c(e_1, \dots, e_r)]$ for representatives $e_i \in d_i$;
- for every symbol f introduced by compilation with closure arity s , extension arity m and witness e_f (free variables $\vec{w}$): writing $\vec{d}$ for the values at the closure places and $\vec{d}'$ for those at the extension places, $f^{\mathfrak{M}}(\dots) = [e_f[\vec{e}/\vec{w}] e'_1 \cdots e'_m]$ for representatives. The argument places appear in the order of the symbol's introduction in Definition 5.2: closure places first for constants and for λ - and fix-lifts, whereas a case-lift symbol $g(z, \vec{w}')$ takes its single extension place (the scrutinee) first, so for it $g^{\mathfrak{M}}(d, \vec{d}') = [e_g[\vec{e}'/\vec{w}'] e]$. In particular $\hat{c}^{\mathfrak{M}}(\vec{d}) = [c \vec{e}]$ and $(\hat{c}^0)^{\mathfrak{M}} = [c]$.

All interpretations are well defined on classes by congruence of $=_{\square}$. We write $u^{\mathfrak{M},\rho}$ for the denotation of a first-order term u under an assignment ρ , and $\mathfrak{M}, \rho \models \phi$ for Tarskian satisfaction. Valuations restricted to term variables serve as first-order assignments.

6.4 Adequacy of the term translation and of compilation

The two lemmas of this subsection are the companion paper's, re-proved here in full — including the cases for the two head clauses added in Definition 5.2 and the pinned treatment of recursive closures (Remark 5.7) — both because the soundness proof cites their content and because their *independence* of the indexed extension is itself the point: compilation concerns only the program half, which is index-blind.

Definition 6.10 (Coherent template maps). For a template map θ and a substitution σ_{θ} assigning closed $\lambda_{\square}$ terms to $\text{dom}(\theta)$, say $(\theta, \sigma_{\theta})$ is *coherent* for an assignment ρ if for every g with $\theta(g) = (f, \vec{w}, m)$, the variables of the closure terms $\vec{w}$ are assigned by ρ and:

1. $\text{val}_{\theta}(g)^{\mathfrak{M},\rho} = [\sigma_{\theta}(g)]$ — the pointer value denotes the class of the assigned closed term;
2. $f^{\mathfrak{M}}(\vec{w}^{\mathfrak{M},\rho}, \vec{d}) = [\sigma_{\theta}(g) \ e_1 \cdots e_m]$ for all $d_1, \dots, d_m \in \mathcal{D}$ with representatives e_i .

Clause (1) is what the lift clauses consume — a pinned closure place must denote the recursion's value; clause (2) is what the template clause of $\mathcal{C}_{\theta}$ consumes.

Lemma 6.11 (Adequacy of $\mathcal{C}$). *Let e be a $\lambda_{\square}$ term whose free variables are first-order variables assigned by ρ or bound by θ , and let $(\theta, \sigma_{\theta})$ be coherent for ρ . Then $\mathcal{C}_{\theta}(e)^{\mathfrak{M},\rho} = [e^{\rho, \sigma_{\theta}}]$, where $e^{\rho, \sigma_{\theta}}$ substitutes representatives of $\rho(x)$ for the first-order variables and $\sigma_{\theta}(g)$ for the θ -bound variables. In particular $\mathcal{C}(\mathcal{E}(t))^{\mathfrak{M},\rho} = \llbracket t \rrbracket_{\rho}$ for every $\text{CIC}_{\text{ix}}^{-}$ term t with $\theta = \emptyset$.*

Proof. By induction on e following the clauses of Definition 5.2. Fix representatives $\hat{e}_i \in \mathcal{C}_{\theta}(e_i)^{\mathfrak{M},\rho}$; by the induction hypothesis we may take $\hat{e}_i = e_i^{\rho, \sigma_{\theta}}$. Since $@^{\mathfrak{M}}$ is the application of classes, any $@$ -chain below denotes the class of the corresponding applied representatives.

- $e = x$ ordinary: $x^{\mathfrak{M},\rho} = \rho(x) = [x^{\rho, \sigma_{\theta}}]$ by the definition of the substitution. $e = \square$: both sides are $[\square]$. $e = \text{c}(\vec{e})$: $\widehat{\text{c}}(\mathcal{C}_{\theta}(\vec{e}))^{\mathfrak{M},\rho} = \widehat{\text{c}}^{\mathfrak{M}}([\vec{e}^{\rho, \sigma_{\theta}}]) = [\text{c}(\vec{e}^{\rho, \sigma_{\theta}})]$ by the induction hypothesis and Definition 6.9.
- Spine with θ -bound head, $\theta(g) = (f, \vec{w}, m)$: the denotation of $f(\vec{w}, \mathcal{C}_{\theta}(e_1), \dots, \mathcal{C}_{\theta}(e_m))$ is $f^{\mathfrak{M}}(\vec{w}^{\mathfrak{M},\rho}, [\hat{e}_1], \dots, [\hat{e}_m]) = [\sigma_{\theta}(g) \ \hat{e}_1 \cdots \hat{e}_m]$ by coherence (Definition 6.10); the $@$ -chain appends $\hat{e}_{m+1}, \dots, \hat{e}_j$, giving $[\sigma_{\theta}(g) \ \hat{e}_1 \cdots \hat{e}_j] = [e^{\rho, \sigma_{\theta}}]$.

- Constant head c with $j \geq a_c$: $\widehat{c}^{\mathfrak{M}}([\hat{e}_1], \dots, [\hat{e}_{a_c}]) = [c \hat{e}_1 \cdots \hat{e}_{a_c}]$ by Definition 6.9 (the witness of $\widehat{c}$ is the constant c), and the @-chain appends the rest. With $j < a_c$: $(\widehat{c}^0)^{\mathfrak{M}} = [c]$ and the @-chain gives $[c \hat{e}_1 \cdots \hat{e}_j]$.
- Variable head: $\rho(x) \cdot [\hat{e}_1] \cdots [\hat{e}_j] = [x^\rho \hat{e}_1 \cdots \hat{e}_j]$.
- $\square$ -head: $\odot^{\mathfrak{M}} = [\square]$ and the @-chain gives $[\square \hat{e}_1 \cdots \hat{e}_j]$, the class of e^{ρ, σ_θ} — dead code applied is junk, but a perfectly good domain element.
- Constructor head $c(\vec{e})$: by the constructor case, $\widehat{c}(\mathcal{C}_\theta(\vec{e}))^{\mathfrak{M}, \rho} = [c(\vec{e}^{\rho, \sigma_\theta})]$, and the @-chain appends $\hat{e}_1, \dots, \hat{e}_j$.
- Complex heads, via their value translations. In each case the denotation of the pinned closure supplies, at an ordinary member x , the value $\rho(x)$, and at a pinned member $\text{val}_\theta(g)$, the class $[\sigma_\theta(g)]$ — coherence clause (1) — so substituting representatives of these values for the free variables of the witness effects exactly the substitution $(\cdot)^{\rho, \sigma_\theta}$; this is where pinning earns its keep, an assignment ρ assigning nothing to a θ -bound variable. λ -lift h with value $f(\vec{w}^\theta)$: by Definition 6.9, $f^{\mathfrak{M}}(\vec{w}^{\theta, \mathfrak{M}, \rho}) = [h^{\rho, \sigma_\theta}]$, the witness being h itself. Case-lift $h = \text{case}(s; \vec{b}r)$ with value $g(\mathcal{C}_\theta(s), \vec{w}^\theta)$: $g^{\mathfrak{M}}([\hat{s}], \vec{w}^{\theta, \mathfrak{M}, \rho}) = [(\lambda z. \text{case}(z; \vec{b}r))^{\rho, \sigma_\theta} \hat{s}]$, which β -reduces in one step to $[\text{case}(\hat{s}; \vec{b}r^{\rho, \sigma_\theta})] = [h^{\rho, \sigma_\theta}]$, using the induction hypothesis for $\hat{s}$. Fix-lift with value $f^0(\vec{w}^\theta)$: $(f^0)^{\mathfrak{M}}(\vec{w}^{\theta, \mathfrak{M}, \rho}) = [h^{\rho, \sigma_\theta}]$ via the witness $e_{f^0} = h$. In each case the @-chain then appends $\hat{e}_1, \dots, \hat{e}_j$.

The final claim is the special case $e = \mathcal{E}(t)$ with $\theta = \emptyset$: free variables of $\mathcal{E}(t)$ are free term variables of t , so $[\mathcal{E}(t)^{\rho, \emptyset}] = [\mathcal{E}(t)^\rho] = \llbracket t \rrbracket_\rho$ by Definition 6.2. $\square$

Lemma 6.12 (Soundness of compilation). *Consider any invocation of the compilation procedure on the environment or on a term embedded in the goal or in a declaration, and let $B_\theta(L; e)$ range over the body-compilation calls it performs. Then:*

1. *for every such call and every assignment ρ of the variables of L there is a substitution σ_θ such that the pair (θ, σ_θ) is coherent for ρ and $L^{\mathfrak{M}, \rho} = [e^{\rho, \sigma_\theta}]$ (σ_θ is chosen per assignment: the value of a recursion variable depends on the closure values);*
2. *every axiom emitted by the procedure holds in $\mathfrak{M}$ — under every assignment of its universally quantified variables, junk included.*

Proof. By induction on the tree of procedure calls; part (1) is the invariant that makes part (2) a local computation at each emitting step.

Coherence is established uniformly. Wherever a pair (θ, σ_θ) is needed for an assignment ρ below, define $\sigma_\theta(g)$, for each g with $\theta(g) = (f, \vec{w}, m)$, as a closed representative of $\text{val}_\theta(g)^{\mathfrak{M}, \rho} = (f^0)^{\mathfrak{M}}(\vec{w}^{\mathfrak{M}, \rho})$ — defined because the variables of the stored closure terms are variables of the current left-hand

side (they occur in it, at its template application or pinned places, and rule (2) refines stored templates together with L). Clause (1) of Definition 6.10 then holds by construction, and clause (2) follows because f and f^0 carry the *same* witness — the constant c for the root template $(\hat{c}, \hat{c}^0)$, the fix term h for a fix-lift $(\Phi(h), \Phi^0(h))$ — so by Definition 6.9 both $f^{\mathfrak{M}}(\vec{w}^{\mathfrak{M}, \rho}, \vec{d})$ and $[\sigma_\theta(g) e_1 \cdots e_m]$ are the class of the same witness instance applied to the same representatives. What remains per rule is the left-hand-side invariant $L^{\mathfrak{M}, \rho} = [e^{\rho, \sigma_\theta}]$ (a statement about classes, indifferent to the choice of representatives).

Roots. For $c := t : \tau$ with $\mathcal{E}(t) = \lambda \vec{z}. e_0$ (maximal prefix): $\theta = \emptyset$, and for any ρ with representatives $\vec{e}$ of $\rho(\vec{z})$, $\hat{c}(\vec{z})^{\mathfrak{M}, \rho} = [c \vec{e}]$ (Definition 6.9) $= [\mathcal{E}(t) \vec{e}]$ (one δ -step) $= [e_0^\rho]$ ($|\vec{z}|$ β -steps). For $\mathcal{E}(t) = \text{fix } g_0. \lambda \vec{z}. e_0$: here $\sigma_\theta(g_0)$ is a representative of $(\hat{c}^0)^{\mathfrak{M}} = [c]$ — take $\mathcal{E}(t)$, one δ -step away — and the invariant for the initial call $B(\hat{c}(\vec{z}); e_0)$ is the computation $\hat{c}(\vec{z})^{\mathfrak{M}, \rho} = [\mathcal{E}(t) \vec{e}] = [e_0[\mathcal{E}(t)/g_0, \vec{e}/\vec{z}]] = [e_0^{\rho, \sigma_\theta}]$ (one δ -, one μ - and $|\vec{z}|$ β -steps). For a λ -lift $(B_\theta(f(\vec{w}^\theta); h))$ the invariant at any assignment ρ of the new left-hand side's variables is the interpretation of f through its witness, the pinned places denoting $[\sigma_\theta(g)]$ by coherence clause (1); for a case-lift $(B_\theta(g(z, \vec{w}^\theta); \text{case}(z; \vec{br})))$ it is the witness interpretation followed by one β -step (e_g is the λ -wrapped case); for a fix-lift, $\sigma_{\theta'}(g_0)$ is by the uniform recipe a representative of $(f^0)^{\mathfrak{M}}(\vec{w}^{\mathfrak{M}, \rho})$, i.e. the *closed instance* of the witness h at representatives of the pinned closure values, and the invariant for $B_{\theta'}(f(\vec{w}^\theta, \vec{z}); e_0)$ is the root computation one level down: one μ -step and m β -steps from the witness instance.

Rule (1) ($L \mapsto L @ x, e = \lambda x. e'$): an assignment of the extended left-hand side restricts to one of L ; the invariant transfers by one β -step on representatives, $[(\lambda x. e^{\rho, \sigma_\theta}) \hat{x}] = [e'^{\rho[x], \sigma_\theta}]$.

Rule (2) (refinement of a pattern variable): an assignment ρ' of the variables of $L[\hat{c}_i(\vec{z}_i)/x]$ induces the assignment ρ of L that agrees with ρ' elsewhere and maps x to $\hat{c}_i^{\mathfrak{M}}(\rho'(\vec{z}_i)) = [c_i(\vec{e})]$; the two left-hand sides then denote the same element, and likewise the refined templates: $u[\hat{c}_i(\vec{z}_i)/x]^{\mathfrak{M}, \rho'} = u^{\mathfrak{M}, \rho}$ for every stored closure term u by the first-order substitution lemma, so the uniform coherence data for ρ' (over θ_i) and for ρ (over θ) coincide and the invariant transfers. On the right, a head ι -step converts the representative of $\text{case}(x; \dots)^{\rho, \sigma_\theta}$ into that of $(b_i[c_i(\vec{z}_i)/x])^{\rho', \sigma_\theta}$. Note that the ι -step of $\lambda \square$ fires on *any* constructor form, with no index side condition — this is why the emitted equations hold at all elements, wrong-index instantiations included, which Proposition 8.11 isolates.

Rule (3): by the invariant, the left-hand side of the emitted axiom denotes $[\text{case}(s; \vec{br})^{\rho, \sigma_\theta}]$; by Lemma 6.11 — applicable since (θ, σ_θ) is coherent for ρ — and one β -step through the witness of g , so does the right-hand side $g(\mathcal{C}_\theta(s), \vec{w}^\theta)$; and the invariant for the recursive call on $g(z, \vec{w}^\theta)$ is the case-lift root computation above. *Rule (4)*: both sides denote $[e^{\rho, \sigma_\theta}]$, the

left by the invariant and the right through the fix-lift witness of f^0 with the pinned places supplied by coherence clause (1). *Rule (5)*: the invariant identifies the left-hand side's denotation with $[e^{\rho, \sigma_\theta}]$, and Lemma 6.11 identifies the right-hand side's with the same class. *Pointer axioms*: for a constant, both sides of $\widehat{c}^0 @ z_1 @ \dots @ z_{a_c} \approx \widehat{c}(\vec{z})$ denote $[c \vec{e}]$; for a fix-lift, the axiom ranges over fresh variables at the closure places, and at *every* value of those — pinned or junk — both sides denote the same witness instance applied to the extension arguments, f and f^0 sharing their witness. $\square$

6.5 Coincidence: the translation reflects the semantics

Lemma 6.13 (Coincidence). *Let σ be a shallow set type and φ a shallow proposition over $\mathbb{E}$ and a context of term variables, and let ρ be a valuation of their free variables. Then:*

1. *for every first-order term u over the variables of ρ : $\mathfrak{M}, \rho \models \mathcal{G}_\sigma(u) \iff u^{\mathfrak{M}, \rho} \in \llbracket \sigma \rrbracket_\rho$;*
2. $\mathfrak{M}, \rho \models \mathcal{F}(\varphi) \iff \langle \varphi \rangle_\rho = 1$.

Proof. Mutual induction on the structure of σ and φ , following Definition 5.10.

- $\sigma = I(\vec{a})$: $\mathfrak{M}, \rho \models \widehat{I}(u, \mathcal{C}(\mathcal{E}(\vec{a})))$ iff $(u^{\mathfrak{M}, \rho}, \mathcal{C}(\mathcal{E}(\vec{a}))^{\mathfrak{M}, \rho}) \in \widehat{I}^{\mathfrak{M}}$ iff $(\llbracket \vec{a} \rrbracket_\rho, u^{\mathfrak{M}, \rho}) \in \llbracket I \rrbracket$ — using Lemma 6.11 to identify $\mathcal{C}(\mathcal{E}(a_h))^{\mathfrak{M}, \rho} = \llbracket a_h \rrbracket_\rho$ — iff $u^{\mathfrak{M}, \rho} \in \llbracket I(\vec{a}) \rrbracket_\rho$ by Definition 6.3.
- $\sigma = \Pi x: \sigma'. \tau$ (with x α -renamed fresh for u and ρ): $\mathfrak{M}, \rho \models \forall x. \mathcal{G}_{\sigma'}(x) \rightarrow \mathcal{G}_\tau(u @ x)$ iff for every $d \in \mathcal{D}$, $\mathfrak{M}, \rho[x \mapsto d] \models \mathcal{G}_{\sigma'}(x)$ implies $\mathfrak{M}, \rho[x \mapsto d] \models \mathcal{G}_\tau(u @ x)$; by both induction hypotheses (at the extended valuation) this says: for every $d \in \llbracket \sigma' \rrbracket_\rho$, $u^{\mathfrak{M}, \rho} \cdot d \in \llbracket \tau \rrbracket_{\rho[x \mapsto d]}$ — which is the defining condition of $u^{\mathfrak{M}, \rho} \in \llbracket \Pi x: \sigma'. \tau \rrbracket_\rho$ (Definition 6.3).
- $\sigma = \varphi' \rightarrow \tau$: satisfaction of $\mathcal{F}(\varphi') \rightarrow \mathcal{G}_\tau(u)$ says $\langle \varphi' \rangle_\rho = 1$ implies $u^{\mathfrak{M}, \rho} \in \llbracket \tau \rrbracket_\rho$ (both induction hypotheses), the defining condition of $\llbracket \varphi' \rightarrow \tau \rrbracket_\rho$.
- $\sigma = \{x: \sigma' \mid \varphi'\}$: the first conjunct is the induction hypothesis for σ' ; for the second, the first-order substitution lemma gives $\mathfrak{M}, \rho \models \mathcal{F}(\varphi')[u/x]$ iff $\mathfrak{M}, \rho[x \mapsto u^{\mathfrak{M}, \rho}] \models \mathcal{F}(\varphi')$, which by induction hypothesis (2) is $\langle \varphi' \rangle_{\rho[x \mapsto u^{\mathfrak{M}, \rho}]} = 1$; together, the defining condition of the refinement interpretation.
- Connectives, $\top$, $\perp$: $\mathcal{F}$ is homomorphic and both $\models$ and $\langle \cdot \rangle$ interpret the connectives by the classical truth tables, so induction hypothesis (2) on the components concludes.

- Quantifiers: $\mathfrak{M}, \rho \models \forall x. \mathcal{G}_{\sigma'}(x) \rightarrow \mathcal{F}(\varphi')$ iff $\langle \varphi' \rangle_{\rho[x \mapsto d]} = 1$ for every $d \in \llbracket \sigma' \rrbracket_\rho$ (both induction hypotheses, as in the Π -case) iff $\min_{d \in \llbracket \sigma' \rrbracket_\rho} \langle \varphi' \rangle_{\rho[x \mapsto d]} = 1 = \langle \forall x: \sigma'. \varphi' \rangle_\rho$; the empty-range conventions agree (vacuous satisfaction, $\min \emptyset = 1$). Dually for $\exists$ with $\wedge$ and $\max \emptyset = 0$.
- Equations and J -atoms: by Lemma 6.11, $\mathcal{C}(\mathcal{E}(a))^{\mathfrak{M}, \rho} = \llbracket a \rrbracket_\rho$, and the model interprets $\approx$ as class equality and $\widehat{J}$ as $\langle J \rangle$.

Shallowiness is used exactly here: predicate quantifiers and predicate-variable atoms have no translation; all remaining formers are covered. $\square$

The new clause deserves a comment in the direction of Section 2.4: the coincidence for family guards is an equivalence *at every index tuple*, junk tuples included — both sides are false at a wrong-index pair such as $([\text{vnil}], [\text{S}(\text{O})])$ by Lemma 6.5(3). It is this two-sided, index-aware coincidence that licenses using guard atoms in *both* polarities of the translation, and its failure under index-forgetting that Proposition 8.9 exhibits.

7 Soundness

This section proves the main results: every axiom of $\Delta(\mathbb{E}, G)$ holds in $\mathfrak{M}$, and consequently first-order provability of $\mathcal{F}(G)$ entails truth of G in the semantics of Section 6.

7.1 Guard disciplines

Definition 7.1 (Context satisfaction). A valuation ρ *satisfies* a context Γ , written $\rho \models \Gamma$, if in telescope order: $\rho(x) \in \llbracket \sigma \rrbracket_\rho$ for $x : \sigma$; $\rho(X) : \mathcal{D}^n \rightarrow \{0, 1\}$ for $X : (\vec{\sigma}) \rightarrow \text{Prop}$; and $\langle \varphi \rangle_\rho = 1$ for $p : \varphi$.

The treatment of fix interpolates between “ $\rho(f)$ is anything” and “ $\rho(f)$ inhabits its type” by a rank-restricted membership; the only change against [Cza26] is that the rank of the guard argument is its *element rank* in the guard family (Definition 6.4), which Lemma 6.5 makes well defined without reference to the indices at which the argument is used.

Lemma 7.2 (Flattening of telescopes). *Let $T = \Pi x_1: \sigma_1 \dots \Pi x_n: \sigma_n. \tau$. Then $F \in \llbracket T \rrbracket_\rho$ iff for all $d_1, \dots, d_n$ chosen successively with $d_i \in \llbracket \sigma_i \rrbracket_{\rho[x_1 \mapsto d_1, \dots, x_{i-1} \mapsto d_{i-1}]}$ we have $F \cdot d_1 \dots d_n \in \llbracket \tau \rrbracket_{\rho[\vec{x} \mapsto \vec{d}]}$.*

Proof. Unfold Definition 6.3 n times. $\square$

Definition 7.3 (Rank-restricted interpretation). Let $T = \Pi x_1: \sigma_1 \dots \Pi x_n: \sigma_n. \tau$ be a fix type with guard position k , $\sigma_k = I(\vec{v})$, and let $m \in \mathbb{N}$. Define

$\llbracket T \rrbracket_\rho^{k, < m}$ as the set of $F \in \mathcal{D}$ such that for all successively chosen $d_1, \dots, d_n$ as in Lemma 7.2 with additionally $\text{rk}_I(d_k) < m$:

$$F \cdot d_1 \cdots d_n \in \llbracket \tau \rrbracket_{\rho[\vec{x} \mapsto \vec{d}]}$$

(The rank is defined: the chain condition places d_k in $\llbracket I(\vec{v}) \rrbracket_{\rho[\dots]}$, so $(\cdot, d_k) \in \llbracket I \rrbracket$.) By Lemma 7.2 and finiteness of ranks, $\llbracket T \rrbracket_\rho = \bigcap_m \llbracket T \rrbracket_\rho^{k, < m}$, and $\llbracket T \rrbracket_\rho^{k, < m}$ is antitone in m .

Definition 7.4 (Guard discipline). A *guard discipline* $\mathcal{D}$ over Γ is a finite set of triples (f, V_f, m_f) where the f are pairwise distinct term variables declared in Γ with fix types T_f (guard position k_f , guard family I_f), V_f is a set of term variables declared in Γ whose types are occurrences of I_f , and $m_f \in \mathbb{N}$. A phrase t *respects* $\mathcal{D}$ if for each $(f, V_f, m_f) \in \mathcal{D}$, every occurrence of f in t lies at the head of an application spine $f a_1 \cdots a_{n_f}$ occurring as a subterm of t , with $a_{k_f} \in V_f$. A valuation ρ satisfies Γ *under* $\mathcal{D}$, written $\rho \models_{\mathcal{D}} \Gamma$, if ρ satisfies all declarations of Γ as in Definition 7.1 except that for each $(f, V_f, m_f) \in \mathcal{D}$ only $\rho(f) \in \llbracket T_f \rrbracket_\rho^{k_f, < m_f}$ is required, and additionally $\text{rk}_{I_f}(\rho(y)) < m_f$ for every $y \in V_f$.

Lemma 7.5 (Locality of respect). *Let t respect $\mathcal{D}$ and let s be a subterm of t occurring in a non-applied position (any position other than the function part of an application). Then s respects $\mathcal{D}$.*

Proof. As in [Cza26]: if the witnessing spine of an occurrence of f in s were not contained in s , then s would be a proper prefix of that spine and hence occur applied, contradicting the assumption. $\square$

7.2 The fundamental lemma

Throughout, the domain $\mathcal{D}$, the interpretations $\llbracket \cdot \rrbracket$, $\langle \cdot \rangle$, $\langle \cdot \rangle$ and the structure $\mathfrak{M}$ are the ones constructed in Section 6 over the *full* fixed environment $\mathbb{E}$. For a well-formed prefix $\mathbb{E}'$ of $\mathbb{E}$, the *environment hypothesis at $\mathbb{E}'$* is:

- ($H_{\mathbb{E}'}$) for every term definition $c := t : \tau_c$ in $\mathbb{E}'$, $[c] \in \llbracket \tau_c \rrbracket$; and for every lemma $c := \pi : \varphi_c$ in $\mathbb{E}'$, $\langle \varphi_c \rangle = 1$;

we write (H) for ($H_{\mathbb{E}}$) (discharged in Theorem 7.8). The fundamental lemma is stated relative to a prefix precisely so that Theorem 7.8 can establish (H) by induction on declaration position without circularity: the lemma is applied to bodies typed over the *strict* prefix preceding the definition under scrutiny, and only the hypothesis for that prefix is assumed. The parameterisation costs nothing: a derivation over $\mathbb{E}'$ is verbatim a derivation over $\mathbb{E}$ (typing consults declarations by name, and $\mathbb{E}'$ -conversion is contained in $\mathbb{E}$ -conversion, under which the semantics is invariant); all semantic machinery — Definitions 6.3 and 6.9, Lemmas 6.6 to 6.8, the rank apparatus —

is defined once over the full $\mathbb{E}$ and does not depend on any $(H_{\mathbb{E}'})$; and the proof below consults the hypothesis only in the constant cases, for constants the derivation mentions, which lie in $\mathbb{E}'$.

We use the simultaneous form of Lemma 6.7 throughout: for a simultaneous substitution of terms $\vec{a}$ for distinct variables $\vec{x}$, $\llbracket \sigma[\vec{a}/\vec{x}] \rrbracket_\rho = \llbracket \sigma \rrbracket_{\rho[\vec{x} \mapsto \llbracket \vec{a} \rrbracket_\rho]}$, obtained from the single-variable form by α -renaming to fresh variables and iterating; likewise for terms and propositions.

Lemma 7.6 (Fundamental lemma). *Let $\mathbb{E}'$ be a well-formed prefix of $\mathbb{E}$ (possibly $\mathbb{E}$ itself) such that $(H_{\mathbb{E}'})$ holds. Let $\mathcal{D}$ be a guard discipline over a context Γ over $\mathbb{E}'$, and $\rho \models_{\mathcal{D}} \Gamma$.*

1. *If $\Gamma \vdash t : \sigma$ is derivable over $\mathbb{E}'$ and t respects $\mathcal{D}$, then $\llbracket t \rrbracket_\rho \in \llbracket \sigma \rrbracket_\rho$.*
2. *If $\Gamma \vdash \pi : \varphi$ is derivable over $\mathbb{E}'$ and π respects $\mathcal{D}$, then $\langle \varphi \rangle_\rho = 1$.*

Proof. By induction on the size of the typing derivation, mutually for (1) and (2). We freely use Lemma 6.6 (weakening), Lemma 6.7 (semantic substitution, in simultaneous form), Lemma 6.8 (conversion invariance, which handles the two conversion rules), and Lemma 7.5 to propagate the discipline to subphrases. Occurrences of disciplined variables *inside types, propositions and predicate expressions* incur no semantic obligation and need no propagation. Such occurrences do exist — if the codomain of T_f is an index type, a branch body may contain a well-formed annotation $\lambda x : \text{vec}(f \text{ } k \text{ } w) . \dots$ with $w \in V_f$, an applied occurrence of f inside a type that satisfies the guard condition — but every semantic clause of Definition 6.3 consumes embedded terms only through the denotation $\llbracket \cdot \rrbracket_\rho$, which is defined for *every* valuation: membership of $\rho(f)$ in a (rank-restricted) type interpretation is consulted only where this lemma's conclusion demands it for a term or proof judgement, never when a type or proposition is evaluated. When extending ρ at bound variables we α -rename so that disciplined variables, V_f -variables and context variables are not shadowed.

Variables, constants. If $t = x$ is not disciplined, $\rho(x) \in \llbracket \sigma \rrbracket_\rho$ by $\rho \models_{\mathcal{D}} \Gamma$; a bare disciplined f does not respect $\mathcal{D}$, so this case cannot arise for it. For a term constant — declared in $\mathbb{E}'$, since the derivation is over $\mathbb{E}'$ — $\llbracket c \rrbracket_\rho = [c] \in \llbracket \tau_c \rrbracket$ by $(H_{\mathbb{E}'})$, the δ -rule and weakening; for a lemma constant in (2), directly by $(H_{\mathbb{E}'})$.

Disciplined spines. Suppose $t = f a_1 \dots a_j$ with $(f, V_f, m_f) \in \mathcal{D}$ and $j \geq n_f$ (respect makes $f a_1 \dots a_{n_f}$ a spine of t), and let $T_f = \Pi x_1 : \sigma_1 \dots \Pi x_{n_f} : \sigma_{n_f} . \tau$. The typing derivation decomposes into j applications; by induction hypothesis (1) applied to each argument (which respects $\mathcal{D}$ by Lemma 7.5), $d_i := \llbracket a_i \rrbracket_\rho \in \llbracket \sigma_i \rrbracket_{\rho[x_1 \mapsto d_1, \dots]}$ for $i \leq n_f$, the intermediate assignments being handled by semantic substitution as in the application case below. By respect, $a_{k_f} \in V_f$, so $\text{rk}_{I_f}(d_{k_f}) = \text{rk}_{I_f}(\rho(a_{k_f})) < m_f$; since

$$\rho(f) \in \llbracket T_f \rrbracket_\rho^{k_f, < m_f},$$

$$\llbracket f a_1 \cdots a_{n_f} \rrbracket_\rho = \rho(f) \cdot d_1 \cdots d_{n_f} \in \llbracket \tau \rrbracket_{\rho[\vec{x} \mapsto \vec{d}]} = \llbracket \tau[\vec{a}/\vec{x}] \rrbracket_\rho,$$

the last step by semantic substitution; remaining arguments $a_{n_f+1}, \dots, a_j$ are absorbed by the application case.

Applications and abstractions. For $t a$ with head not disciplined: the function part respects $\mathcal{D}$ (its disciplined occurrences have their spines within it, by the argument of Lemma 7.5 applied to the maximal spine decomposition), so by the induction hypotheses $\llbracket t \rrbracket_\rho \in \llbracket \Pi x:\sigma.\tau \rrbracket_\rho$ and $\llbracket a \rrbracket_\rho \in \llbracket \sigma \rrbracket_\rho$, whence $\llbracket t a \rrbracket_\rho = \llbracket t \rrbracket_\rho \cdot \llbracket a \rrbracket_\rho \in \llbracket \tau \rrbracket_{\rho[x \mapsto \llbracket a \rrbracket_\rho]} = \llbracket \tau[a/x] \rrbracket_\rho$. For $t \pi$: $\mathcal{E}(t \pi) = \mathcal{E}(t)$; induction hypothesis (2) gives $\langle \varphi \rangle_\rho = 1$, so induction hypothesis (1) and the clause for $\llbracket \varphi \rightarrow \tau \rrbracket_\rho$ conclude. For $\lambda x:\sigma.t$ and $e \in \llbracket \sigma \rrbracket_\rho$: $\llbracket \lambda x:\sigma.t \rrbracket_\rho \cdot e = \llbracket (\lambda x.\mathcal{E}(t)^\rho) \hat{e} \rrbracket = \llbracket t \rrbracket_{\rho[x \mapsto e]}$ (one β -step), which lies in $\llbracket \tau \rrbracket_{\rho[x \mapsto e]}$ by the induction hypothesis. For $\lambda p:\varphi.t$: if $\langle \varphi \rangle_\rho = 1$ then $\rho \models_{\mathcal{D}} \Gamma, p:\varphi$ and $\llbracket \lambda p:\varphi.t \rrbracket_\rho = \llbracket t \rrbracket_\rho \in \llbracket \tau \rrbracket_\rho$ by the induction hypothesis, as the clause for $\varphi \rightarrow \tau$ requires.

Constructors. Let $t = c_i(\vec{a}; \vec{\pi})$ with the typing premises of Figure 2, so the displayed type is $I(\vec{t}_i[\vec{a}/\vec{y}])$. Let $\rho_0 := [\vec{y} \mapsto \llbracket \vec{a} \rrbracket_\rho]$. By induction hypothesis (1) on each a_j and semantic substitution, $\llbracket a_j \rrbracket_\rho \in \llbracket A_{ij}[\vec{a}_{<j}/\vec{y}_{<j}] \rrbracket_\rho = \llbracket A_{ij} \rrbracket_{\rho_0}$; for recursive j with $A_{ij} = I(\vec{v}_{ij})$ this reads $(\llbracket \vec{v}_{ij} \rrbracket_{\rho_0}, \llbracket a_j \rrbracket_\rho) \in \llbracket I \rrbracket$. By induction hypothesis (2) on each π_l and semantic substitution, $\langle \varphi_{il} \rangle_{\rho_0} = \langle \varphi_{il}[\vec{a}/\vec{y}] \rangle_\rho = 1$. By Lemma 6.5(5),

$$(\llbracket \vec{t}_i \rrbracket_{\rho_0}, [c_i(\vec{e})]) \in \llbracket I \rrbracket, \quad \text{where } [c_i(\vec{e})] = \llbracket t \rrbracket_\rho,$$

and $\llbracket \vec{t}_i \rrbracket_{\rho_0} = \llbracket \vec{t}_i[\vec{a}/\vec{y}] \rrbracket_\rho$ by semantic substitution, so $\llbracket t \rrbracket_\rho \in \llbracket I(\vec{t}_i[\vec{a}/\vec{y}]) \rrbracket_\rho$.

Forced case. Let $t = \text{case}(a; \vec{z}.z.\tau; \{c_i(\vec{y}_i; \vec{p}_i; \vec{q}_i) \Rightarrow t_i\}_i)$ with $\Gamma \vdash a : I(\vec{u})$, and displayed type $\tau[\vec{u}/\vec{z}, a/z]$. By induction hypothesis (1), $(\llbracket \vec{u} \rrbracket_\rho, \llbracket a \rrbracket_\rho) \in \llbracket I \rrbracket$. Lemma 6.5(1) yields the unique decomposition: an i , classes $[\vec{e}]$ with $\llbracket a \rrbracket_\rho = [c_i(\vec{e})]$, the memberships and premise truths at $\rho_0 = [\vec{y}_i \mapsto [\vec{e}]]$, and

$$\llbracket \vec{u} \rrbracket_\rho = \llbracket \vec{t}_i \rrbracket_{\rho_0}. \quad (*)$$

Let $\rho' := \rho[\vec{y}_i \mapsto [\vec{e}]]$ ($\vec{y}_i$ fresh). Then $\rho' \models_{\mathcal{D}} \Gamma, \vec{y}_i:\vec{A}_i, \vec{p}_i:\vec{\varphi}_i, \vec{q}_i:(\vec{u} =_{\vec{\sigma}} \vec{t}_i)$: the Γ -part by weakening; the memberships and premises by the decomposition (their free variables are among earlier declarations and $\vec{y}_i$, on which ρ' agrees with ρ_0); and for the index equations, $\langle u_h =_{\sigma_h} t_{ih} \rangle_{\rho'} = 1$ because $\llbracket u_h \rrbracket_{\rho'} = \llbracket u_h \rrbracket_\rho$ ($\vec{y}_i$ fresh for u_h) = $\llbracket t_{ih} \rrbracket_{\rho_0} = \llbracket t_{ih} \rrbracket_{\rho'}$ by $(*)$. On erasures,

$$\mathcal{E}(t)^\rho = \text{case}(\mathcal{E}(a)^\rho; \dots) =_{\square} \text{case}(c_i(\vec{e}); \dots) \rightarrow_{\square} \mathcal{E}(t_i)^{\rho'},$$

so $\llbracket t \rrbracket_\rho = \llbracket t_i \rrbracket_{\rho'}$. By the induction hypothesis on the branch derivation (t_i respects $\mathcal{D}$ by Lemma 7.5),

$$\llbracket t_i \rrbracket_{\rho'} \in \llbracket \tau[\vec{t}_i/\vec{z}, c_i(\vec{y}_i; \vec{p}_i)/z] \rrbracket_{\rho'} = \llbracket \tau \rrbracket_{\rho'[\vec{z} \mapsto \llbracket \vec{t}_i \rrbracket_{\rho'}, z \mapsto [c_i(\vec{y}_i; \vec{p}_i)]]_{\rho'}}$$

by semantic substitution. Now $\llbracket \vec{t}_i \rrbracket_{\rho'} = \llbracket \vec{t}_i \rrbracket_{\rho_0} = \llbracket \vec{u} \rrbracket_{\rho}$ by $(*)$, and $\llbracket c_i(\vec{y}_i; \vec{p}_i) \rrbracket_{\rho'} = [c_i(\vec{e})] = \llbracket a \rrbracket_{\rho}$; by weakening on the remaining free variables of τ and semantic substitution backwards, the displayed set equals $\llbracket \tau[\vec{u}/\vec{z}, a/z] \rrbracket_{\rho}$, as required. This is the place where the index equations of the model — Lemma 6.5(1) — convert the branch's type, stated at the constructor's indices, to the case's type, stated at the occurrence's indices; in CIC the same conversion is performed by index unification, and in the forced reading by rewriting along $\vec{q}_i$.

Fix. Let $t = \text{fix } f:T. \lambda \vec{x}. \text{case}(x_k; \vec{z}.z.\tau_0; \{c_i(\vec{y}_i; \vec{p}_i; \vec{q}_i) \Rightarrow b_i\}_i)$ with $T = \Pi x_1:\sigma_1 \dots \Pi x_n:\sigma_n.\tau$, $\sigma_k = I(\vec{v})$, $\tau = \tau_0[\vec{v}/\vec{z}, x_k/z]$, and let $F := \llbracket t \rrbracket_{\rho}$. Since $\llbracket T \rrbracket_{\rho} = \bigcap_m \llbracket T \rrbracket_{\rho}^{k, < m}$, it suffices to prove $F \in \llbracket T \rrbracket_{\rho}^{k, < m}$ for all m , by strong induction on m (the *rank induction*, nested inside the main induction).

Let m be given with $F \in \llbracket T \rrbracket_{\rho}^{k, < m'}$ for all $m' < m$, and let $d_1, \dots, d_n$ be a chain as in Definition 7.3 with $m_0 := \text{rk}_I(d_k) < m$. Write $\rho_c := \rho[x_1 \mapsto d_1, \dots, x_{k-1} \mapsto d_{k-1}]$; the chain condition gives $(\llbracket \vec{v} \rrbracket_{\rho_c}, d_k) \in \llbracket I \rrbracket$. By Lemma 6.5(1) there are i and $[\vec{e}]$ with $d_k = [c_i(\vec{e})]$, the memberships and premises at $\rho_0 = [\vec{y}_i \mapsto [\vec{e}]]$, and $\llbracket \vec{v} \rrbracket_{\rho_c} = \llbracket \vec{t}_i \rrbracket_{\rho_0}$; by Lemma 6.5(4), $\text{rk}_I([e_j]) < m_0$ for recursive j . Unfolding one μ -step, n β -steps and one ι -step on a representative of $F \cdot d_1 \dots d_n$:

$$F \cdot d_1 \dots d_n = \llbracket b_i \rrbracket_{\rho''}, \quad \rho'' := \rho[f \mapsto F, \vec{x} \mapsto \vec{d}, \vec{y}_i \mapsto [\vec{e}]].$$

Apply the *main* induction hypothesis to the branch derivation

$$\Gamma, f:T, \vec{x}:\vec{\sigma}, \vec{y}_i:\vec{A}_i, \vec{p}_i:\vec{\varphi}_i, \vec{q}_i:(\vec{v} =_{\vec{\sigma}'} \vec{t}_i) \vdash b_i : \tau_0[\vec{t}_i/\vec{z}, c_i(\vec{y}_i; \vec{p}_i)/z]$$

with discipline $\mathcal{D}' = \mathcal{D} \cup \{(f, \vec{y}_i^{\text{ec}}, m_0)\}$ and valuation ρ'' . The premises hold: b_i respects $\mathcal{D}'$ (for f by the guard condition grd_k ; for the outer disciplines by Lemma 7.5); $\rho''(f) = F \in \llbracket T \rrbracket_{\rho}^{k, < m_0}$ by the rank induction hypothesis; $\text{rk}_I(\rho''(y)) < m_0$ for $y \in \vec{y}_i^{\text{ec}}$ as noted; the $\vec{x}$ - and $\vec{y}_i$ -declarations are satisfied by the chain condition and the decomposition; the premise proofs $\vec{p}_i$ by the decomposition; and the index equations $\vec{q}_i$ by $\llbracket \vec{v} \rrbracket_{\rho''} = \llbracket \vec{v} \rrbracket_{\rho_c} = \llbracket \vec{t}_i \rrbracket_{\rho_0} = \llbracket \vec{t}_i \rrbracket_{\rho''}$ ($\vec{v}$ mentions only $x_1, \dots, x_{k-1}$). The conclusion gives $\llbracket b_i \rrbracket_{\rho''} \in \llbracket \tau_0[\vec{t}_i/\vec{z}, c_i(\vec{y}_i; \vec{p}_i)/z] \rrbracket_{\rho''}$; by semantic substitution this set is $\llbracket \tau_0 \rrbracket_{\rho''[\vec{z} \mapsto \llbracket \vec{t}_i \rrbracket_{\rho''}, z \mapsto d_k]} = \llbracket \tau_0 \rrbracket_{\rho''[\vec{z} \mapsto \llbracket \vec{v} \rrbracket_{\rho''}, z \mapsto \llbracket x_k \rrbracket_{\rho''}]} = \llbracket \tau_0[\vec{v}/\vec{z}, x_k/z] \rrbracket_{\rho''} = \llbracket \tau \rrbracket_{\rho[\vec{x} \mapsto \vec{d}]}$ (weakening: $f, \vec{y}_i$ do not occur in τ). This closes the rank induction, hence the fix case.

Refinement formers. **refine**(t, π): by the induction hypotheses, $\llbracket t \rrbracket_{\rho} \in \llbracket \sigma \rrbracket_{\rho}$ and $\langle \varphi[t/x] \rangle_{\rho} = 1$, i.e. $\langle \varphi \rangle_{\rho[x \mapsto \llbracket t \rrbracket_{\rho}]} = 1$ by semantic substitution; since $\mathcal{E}(\text{refine}(t, \pi)) = \mathcal{E}(t)$, this is membership in $\llbracket \{x:\sigma \mid \varphi\} \rrbracket_{\rho}$. **val**(t): membership in the refinement interpretation includes membership in $\llbracket \sigma \rrbracket_{\rho}$, and $\mathcal{E}(\text{val}(t)) = \mathcal{E}(t)$. For **valid**(t) in (2): the payload component of the same membership, rewritten by semantic substitution using $\llbracket \text{val}(t) \rrbracket_{\rho} = \llbracket t \rrbracket_{\rho}$.

Propositional singleton eliminations. **split**($\pi; p_1 p_2.t$): induction hypothesis (2) gives both conjuncts, so $\rho \models_{\mathcal{D}} \Gamma, p_1:\varphi_1, p_2:\varphi_2$; conclude by induction

hypothesis (1) on t , the erasures agreeing. $\text{cast}(\pi; z.\tau; t)$: induction hypothesis (2) gives $\llbracket a \rrbracket_\rho = \llbracket b \rrbracket_\rho$, hence $\llbracket \tau[a/z] \rrbracket_\rho = \llbracket \tau \rrbracket_{\rho[z \mapsto \llbracket a \rrbracket_\rho]} = \llbracket \tau[b/z] \rrbracket_\rho$; conclude by induction hypothesis (1) on t . $\text{efq}(\pi; \sigma)$ (and $\text{efq}(\pi; \varphi)$ in (2)): vacuous — induction hypothesis (2) would give $\langle \perp \rangle_\rho = 1$.

Singleton elimination. Let $t = \text{scase}_J(\pi; \vec{z}.\tau; \vec{p}.t_0)$ with J a forced singleton (clause $k : \forall \vec{x}:\vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow J(\vec{u}^0)$, solution j), $\Gamma \vdash \pi : J(\vec{u})$, $\theta = [u_{j(i)}/x_i]_i$, and displayed type $\tau[\vec{u}/\vec{z}]$. By induction hypothesis (2), $\llbracket \vec{u} \rrbracket_\rho \in \langle J \rangle$; unfolding the least stage containing the tuple (the predicate analogue of Lemma 6.5(1), immediate from Definition 6.3 since J has a single clause): there is $\rho_0 = [\vec{x} \mapsto \vec{d}]$ with $d_i \in \llbracket \tau_i \rrbracket_{\rho_0}$, $\langle \varphi_l \rangle_{\rho_0} = 1$ for every l (for a recursive premise $J(\vec{v})$, membership of $\llbracket \vec{v} \rrbracket_{\rho_0}$ in an earlier stage gives exactly $\langle J(\vec{v}) \rangle_{\rho_0} = 1$), and $\llbracket \vec{u} \rrbracket_\rho = \llbracket \vec{u}^0 \rrbracket_{\rho_0}$. By forcedness, for each i :

$$\llbracket \theta(x_i) \rrbracket_\rho = \llbracket u_{j(i)} \rrbracket_\rho = (\llbracket \vec{u}^0 \rrbracket_{\rho_0})_{j(i)} = \llbracket u_{j(i)}^0 \rrbracket_{\rho_0} = \llbracket x_i \rrbracket_{\rho_0} = d_i.$$

Hence by semantic substitution $\langle \varphi_l[\theta] \rangle_\rho = \langle \varphi_l \rangle_{\rho[\vec{x} \mapsto \vec{d}]} = \langle \varphi_l \rangle_{\rho_0} = 1$, so $\rho \models_{\mathcal{D}} \Gamma, \vec{p}:\theta(\vec{\varphi})$, and the induction hypothesis (1) on t_0 gives $\llbracket t_0 \rrbracket_\rho \in \llbracket \tau[\theta(\vec{u}^0)/\vec{z}] \rrbracket_\rho = \llbracket \tau \rrbracket_{\rho[\vec{z} \mapsto \llbracket \theta(\vec{u}^0) \rrbracket_\rho]}$. Again by the displayed computation, $\llbracket \theta(u_h^0) \rrbracket_\rho = \llbracket u_h^0 \rrbracket_{\rho_0} = \llbracket u_h \rrbracket_\rho$ for every h , so this set is $\llbracket \tau[\vec{u}/\vec{z}] \rrbracket_\rho$; and $\llbracket t \rrbracket_\rho = \llbracket t_0 \rrbracket_\rho$ since $\mathcal{E}(t) = \mathcal{E}(t_0)$.

Discrimination. For $\text{discr}(\pi)$ with $\Gamma \vdash \pi : c(\vec{a}; \cdot) =_\sigma d(\vec{b}; \cdot)$, $c \neq d$: induction hypothesis (2) would give $[c(\mathcal{E}(\vec{a})^\rho)] = [d(\mathcal{E}(\vec{b})^\rho)]$, contradicting Corollary 4.4(1); the case is vacuous — no $\rho \models_{\mathcal{D}} \Gamma$ reaches it.

Proof formers. The introduction and elimination rules for $\rightarrow$, $\wedge$, $\vee$, $\forall$, $\exists$, $\top$ and predicate quantification are sound for the truth-value semantics by direct computation: each rule matches the corresponding min/max clause of Definition 6.3; the instantiation rules use semantic substitution (parts (1) and (3) of Lemma 6.7); and the witnesses in ecase and predicate-quantifier elimination are semantic (an element $d \in \llbracket \sigma \rrbracket_\rho$, a function $\langle P \rangle_\rho$), requiring no syntactic counterpart. $\text{refl}_\sigma(a)$ is reflexivity of class equality; $\text{rew}(\pi; z.\psi; \pi')$ uses $\llbracket a \rrbracket_\rho = \llbracket b \rrbracket_\rho$ and semantic substitution exactly as $\text{cast}(\cdot; \cdot; \cdot)$.

Predicate-constructor introduction. For $k(\vec{a}; \vec{\pi})$: by induction hypothesis (1) and iterated semantic substitution along the dependent telescope, each $\llbracket a_i \rrbracket_\rho$ lies in $\llbracket \tau_i \rrbracket_{\rho_0}$ where $\rho_0 = [\vec{x} \mapsto \llbracket \vec{a} \rrbracket_\rho]$; by induction hypothesis (2) the non-recursive premises are true at ρ_0 and each recursive premise $J(\vec{v})$ yields $\llbracket \vec{v} \rrbracket_{\rho_0} \in \langle J \rangle$, hence in some finite stage. The stage construction (at the maximum of those stages) puts $\llbracket \vec{u} \rrbracket_{\rho_0}$ in $\langle J \rangle$, and semantic substitution identifies it with $\llbracket \vec{u}[\vec{a}/\vec{x}] \rrbracket_\rho$.

Predicate induction. For $\text{ind}_J(\vec{z}.\psi; \vec{\pi}; \pi_0)$ with $\pi_0 : J(\vec{a})$: by induction hypothesis (2), $\llbracket \vec{a} \rrbracket_\rho \in \langle J \rangle$ and $\langle \Xi_j(\vec{z}.\psi) \rangle_\rho = 1$ for every clause j . We show by strong induction on s that every $\vec{d} \in \langle J \rangle^s$ satisfies $\langle \psi \rangle_{\rho[\vec{z} \mapsto \vec{d}]} = 1$: a tuple enters $\langle J \rangle^s$ (with $s = s' + 1$) through some clause j with witnesses $\vec{d}' \in \llbracket \vec{\tau}_j \rrbracket$ -telescope, true non-recursive premises, and recursive-premise tuples in $\langle J \rangle^{s'}$;

instantiating the semantic universal quantifiers and implications of $\langle \Xi_j \rangle_\rho = 1$ at these witnesses — the inductive-hypothesis premises supplied by the inner induction at s' — yields $\langle \psi[\vec{u}_j/\vec{z}] \rangle_{\rho[\vec{x} \mapsto \vec{d}]} = 1$, which by semantic substitution is $\langle \psi \rangle_{\rho[\vec{z} \mapsto \vec{d}]} = 1$. Conclude at $\llbracket \vec{a} \rrbracket_\rho$.

Datatype induction. Let $\pi = \text{ind}_I(\vec{z}.z.\psi; (\vec{y}_i \vec{p}_i \vec{h}_i.\pi_i)_i; a)$ with $\Gamma \vdash a : I(\vec{u})$ and the clause derivations of Figure 5. By induction hypothesis (1), $(\llbracket \vec{u} \rrbracket_\rho, \llbracket a \rrbracket_\rho) \in \llbracket I \rrbracket$. We show by strong induction on $\text{rk}_I(d)$ that every $(\vec{d}, d) \in \llbracket I \rrbracket$ satisfies $\langle \psi \rangle_{\rho[\vec{z} \mapsto \vec{d}, z \mapsto d]} = 1$. Decompose $(\vec{d}, d)$ by Lemma 6.5(1) into $i, [\vec{e}], \rho_0$; for each recursive j , $(\llbracket \vec{v}_{ij} \rrbracket_{\rho_0}, [e_j]) \in \llbracket I \rrbracket$ with $\text{rk}_I([e_j]) < \text{rk}_I(d)$ (Lemma 6.5(4)), so the inner induction gives $\langle \psi \rangle_{\rho[\vec{z} \mapsto \llbracket \vec{v}_{ij} \rrbracket_{\rho_0}, z \mapsto [e_j]]} = 1$, i.e., by semantic substitution and weakening, $\langle \psi[\vec{v}_{ij}/\vec{z}, y_{ij}/z] \rangle_{\rho'} = 1$ at $\rho' := \rho[\vec{y}_i \mapsto [\vec{e}]]$. Together with the memberships and premise truths of the decomposition, $\rho' \models_{\mathcal{D}}$ the clause context, so the main induction hypothesis (2) applied to π_i gives $\langle \psi[\vec{t}_i/\vec{z}, c_i(\vec{y}_i; \vec{p}_i)/z] \rangle_{\rho'} = 1$, which by semantic substitution is $\langle \psi \rangle_{\rho[\vec{z} \mapsto \vec{d}, z \mapsto d]} = 1$ (using $\llbracket \vec{t}_i \rrbracket_{\rho_0} = \vec{d}$ and $\llbracket c_i(\vec{y}_i; \vec{p}_i) \rrbracket_{\rho'} = d$). Conclude at $(\llbracket \vec{u} \rrbracket_\rho, \llbracket a \rrbracket_\rho)$. $\square$

7.3 Validity of the emitted theory

Lemma 7.7 (Validity of specification axioms). *Let $c := t : \tau$ be a term definition with $\tau \in \mathfrak{S}$, and suppose $[c] \in \llbracket \tau \rrbracket$. Then $\mathfrak{M} \models \mathcal{S}(c)$.*

Proof. By induction on the type τ' in the recursion of Definition 5.11, with the invariant: if ρ assigns the accumulated variables, $v := [c \hat{e}_1 \cdots \hat{e}_j]$ for representatives of the accumulated values, and $v \in \llbracket \tau' \rrbracket_\rho$, then $\mathfrak{M}, \rho \models S(u; \tau')$. The Π -case extends ρ under a guard, which Lemma 6.13 converts to a membership; the premise case uses Lemma 6.13 for $\mathcal{F}(\varphi)$ and leaves the accumulator unchanged; the base case (datatype occurrence or refinement type) is Lemma 6.13(1) — for a family occurrence this includes the index arguments of the final guard atom, whose denotations are the interpretations of the index terms by Lemma 6.11. $\square$

Theorem 7.8 (Environment validity). *Let $\mathbb{E}$ be a well-formed definitional environment with $\mathfrak{S}$ declarations, and G a closed shallow goal. Then:*

1. (H) holds;
2. $\mathfrak{M} \models \Delta(\mathbb{E}, G)$.

Proof. (1) We prove $(H_{\mathbb{E}'})$ for every well-formed prefix $\mathbb{E}'$ of $\mathbb{E}$, by induction on its length; (H) is the case $\mathbb{E}' = \mathbb{E}$. The empty prefix is vacuous. If $\mathbb{E}'$ ends in a datatype or predicate declaration, $(H_{\mathbb{E}'})$ coincides with the induction hypothesis. If $\mathbb{E}'$ ends in $c := t : \tau_c$, the body t is typed over the strict prefix $\mathbb{E}''$ preceding it, for which $(H_{\mathbb{E}''})$ holds by induction; Lemma 7.6 at prefix $\mathbb{E}''$, with empty context, discipline and valuation, gives $\llbracket t \rrbracket \in \llbracket \tau_c \rrbracket$, and $[c] = \llbracket t \rrbracket$

by the δ -rule (a conversion of the full environment, so the classes coincide). Lemmas identically via part (2) of the fundamental lemma.

(2) We check each axiom family of Definition 5.13.

Compilation, pointer and auxiliary axioms: Lemma 6.12.

Specification axioms: Lemma 7.7, using (1).

Premise axioms: (1) and Lemma 6.13 at the empty valuation.

Constructor typing (datatypes): let $\vec{d}$ be an assignment of $\vec{y}$ with $\mathfrak{M} \models \bigwedge_j \mathcal{G}_{A_{ij}}(y_j) \wedge \bigwedge_l \mathcal{F}(\varphi_{il})$ at $\vec{d}$. By Lemma 6.13, $d_j \in \llbracket A_{ij} \rrbracket_{\rho_0}$ for every j and $\langle \varphi_{il} \rangle_{\rho_0} = 1$ for every l , where $\rho_0 = [\vec{y} \mapsto \vec{d}]$ (for recursive j the guard is the atom $\widehat{I}(y_j, \mathcal{C}(\mathcal{E}(\vec{v}_{ij})))$ and coincidence yields $(\llbracket \vec{v}_{ij} \rrbracket_{\rho_0}, d_j) \in \llbracket I \rrbracket$). By Lemma 6.5(5), $(\llbracket \vec{t}_i \rrbracket_{\rho_0}, \widehat{\mathcal{C}}_i^{\mathfrak{M}}(\vec{d})) \in \llbracket I \rrbracket$, and by Lemma 6.11 the index arguments of the conclusion atom denote exactly $\llbracket \vec{t}_i \rrbracket_{\rho_0}$; hence the atom holds.

Inversion (datatypes): let $(d, \vec{d}) \in \widehat{I}^{\mathfrak{M}}$, i.e. $(\vec{d}, d) \in \llbracket I \rrbracket$. Lemma 6.5(1) yields i , $[\vec{e}]$, ρ_0 ; instantiate the existentials of the i -th disjunct with $[\vec{e}]$. The guards and premise conjuncts hold by Lemma 6.13 (memberships and truths of the decomposition); $w \approx \widehat{\mathcal{C}}_i(\vec{y})$ holds by the interpretation of $\widehat{\mathcal{C}}_i$; and each index equation $z_h \approx \mathcal{C}(\mathcal{E}(t_{ih}))$ holds because $d_h = (\llbracket \vec{t}_i \rrbracket_{\rho_0})_h$ and Lemma 6.11 identifies the right-hand side's denotation at the chosen witnesses with $\llbracket t_{ih} \rrbracket_{\rho_0}$.

Injectivity and discrimination: Corollary 4.4(1) — at all elements of $\mathcal{D}$, with no guard.

Predicate clause axioms: let $\vec{d}$ be an assignment of $\vec{x}$ under which the guards $\mathcal{G}_{\tau_{ji}}(x_i)$ and the premise images $\mathcal{F}(\varphi_{jl})$ all hold in $\mathfrak{M}$, and let $\rho_0 = [\vec{x} \mapsto \vec{d}]$. By Lemma 6.13, $d_i \in \llbracket \tau_{ji} \rrbracket_{\rho_0}$ for every i and $\langle \varphi_{jl} \rangle_{\rho_0} = 1$ for every l ; for a recursive premise $J(\vec{v})$ the latter reads $\llbracket \vec{v} \rrbracket_{\rho_0} \in \langle J \rangle$, i.e. membership at some finite stage. The stage construction of Definition 6.3, at the successor of the maximum of those stages, puts $\llbracket \vec{u}_j \rrbracket_{\rho_0}$ in $\langle J \rangle$, and Lemma 6.11 identifies the denotations of the conclusion atom's arguments $\mathcal{C}(\mathcal{E}(\vec{u}_j))$ with exactly this tuple; hence the conclusion atom holds.

Predicate case-analysis axioms: let $\vec{d} \in \widehat{J}^{\mathfrak{M}} = \langle J \rangle$, and let $s + 1$ be the least stage containing it. Unfolding Definition 6.3 yields a clause j and a valuation $\rho_0 = [\vec{x} \mapsto \vec{d}']$ with $d'_i \in \llbracket \tau_{ji} \rrbracket_{\rho_0}$, true non-recursive premises, recursive-premise tuples in $\langle J \rangle^s \subseteq \langle J \rangle$, and $\vec{d} = \llbracket \vec{u}_j \rrbracket_{\rho_0}$. Instantiate the existentials of the j -th disjunct at $\vec{d}'$: the guard and premise conjuncts hold by Lemma 6.13, and each index equation $z_h \approx \mathcal{C}(\mathcal{E}(u_{jh}))$ holds because $d_h = \llbracket u_{jh} \rrbracket_{\rho_0}$ while Lemma 6.11 identifies the right-hand side's denotation at the chosen witnesses with the same class. These are the inversion and constructor-typing arguments one level down: predicates have no element, so the constructor equation is absent and the element-free tuple plays the role of $(w, \vec{z})$. $\square$

7.4 Main theorem and corollaries

Theorem 7.9 (Soundness). *Let $\mathbb{E}$ be a well-formed definitional environment with $\mathfrak{S}$ declarations and G a closed shallow proposition over $\mathbb{E}$. If $\Delta(\mathbb{E}, G) \vdash_{\text{FOL}} \mathcal{F}(G)$ then $\langle G \rangle = 1$: G is true in the proof-irrelevant classical semantics of $\mathbb{E}$.*

Proof. $\mathfrak{M} \models \Delta(\mathbb{E}, G)$ (Theorem 7.8); by soundness of classical first-order logic, $\mathfrak{M} \models \mathcal{F}(G)$; by Lemma 6.13 at the empty valuation, $\langle G \rangle = 1$. $\square$

Corollary 7.10 (Consistency). *$\Delta(\mathbb{E}, G)$ is consistent, for every well-formed definitional $\mathbb{E}$ and goal G . In particular no combination of the emitted axiom families — across all declared families, instances and definitions — can derive $\perp$.*

Proof. $\mathfrak{M} \models \Delta(\mathbb{E}, G)$ and $\mathfrak{M} \not\models \perp$. No consistency assumption on the source is needed: a definitional environment always has the model $\mathfrak{M}$. $\square$

Corollary 7.11 (Refutable goals are not provable). *If $\vdash \pi : \neg G$ for some closed proof over $\mathbb{E}$, then $\Delta(\mathbb{E}, G) \not\vdash_{\text{FOL}} \mathcal{F}(G)$; more generally $\Delta(\mathbb{E}, G)$ never proves both $\mathcal{F}(G)$ and $\mathcal{F}(\neg G)$.*

Proof. By Lemma 7.6 at $\mathbb{E}' = \mathbb{E}$ (with (H) supplied by Theorem 7.8(1)), $\langle \neg G \rangle = 1$, so $\langle G \rangle = 0$ and Theorem 7.9 applies contrapositively; for the second statement, $\mathfrak{M}$ satisfies every consequence of $\Delta(\mathbb{E}, G)$ and $\mathcal{F}(\neg G) = \mathcal{F}(G) \rightarrow \perp$. $\square$

Proposition 7.12 (Standardness on purely structural families). *Call a datatype I purely structural if its constructors have no premises and every non-recursive constructor argument type is an occurrence of a purely structural datatype declared earlier. Then:*

1. *For every purely structural I and $(\vec{d}, d) \in \llbracket I \rrbracket$, d is the class of a closed term built exclusively from constructors of purely structural datatypes; and two such closed constructor terms have the same class only if they carry the same constructor at the root and componentwise equal-class arguments, so each class determines its constructor tree over classes uniquely.*
2. *For vec over a purely structural element type A , the map*

$$(a_1, \dots, a_n) \longmapsto \left([\mathcal{S}^n(\mathcal{O})], [\text{vcons}(\mathcal{S}^{n-1}(\mathcal{O}), \hat{a}_1, \text{vcons}(\mathcal{S}^{n-2}(\mathcal{O}), \hat{a}_2, \dots, \text{vnil} \dots))] \right)$$

(for representatives $\hat{a}_i$) is a bijection from the set of finite sequences over $\llbracket A \rrbracket$ to $\llbracket \text{vec} \rrbracket$; in particular $\llbracket \text{vec}(\mathcal{S}^n(\mathcal{O})) \rrbracket$ is in bijection with $\llbracket A \rrbracket^n$, and $n \mapsto [\mathcal{S}^n(\mathcal{O})]$ is a bijection $\mathbb{N} \rightarrow \llbracket \text{nat} \rrbracket$.

Consequently, for propositions whose quantifiers range over purely structural datatypes, $\langle \cdot \rangle$ is satisfaction in the corresponding standard structure of index-respecting finite trees.

Proof. (1) By induction on declaration order and, within a datatype, on $\text{rk}_I(d)$: Lemma 6.5(1) decomposes d as $[c_i(\vec{e})]$ with non-recursive components in interpretations of earlier purely structural datatypes (induction on declaration order) and recursive components of smaller rank (induction on rank); premises are absent. Distinctness is Corollary 4.4(1) by induction on the terms.

(2) Write enc for the displayed map. *Well-definedness and injectivity*: the image element is a constructor term; injectivity of enc follows from Corollary 4.4(1) by induction on sequence length, together with the injectivity of $n \mapsto [S^n(\mathbf{O})]$ (also Corollary 4.4(1)). *Image in $\llbracket \text{vec} \rrbracket$* : by induction on n using Lemma 6.5(5) and, for the index computation, $\llbracket S n \rrbracket_{[n \mapsto [S^{n-1}(\mathbf{O})]]} = [S^n(\mathbf{O})]$. *Surjectivity*: let $(\vec{d}, d) \in \llbracket \text{vec} \rrbracket$ and induct on $\text{rk}_{\text{vec}}(d)$. If the decomposition is by vnil : $\vec{d} = (\llbracket \mathbf{O} \rrbracket) = ([\mathbf{O}])$ and $d = [\text{vnil}] = \text{enc}(\varepsilon)$. If by vcons with components $[e_n], [e_a], [e_w]$: the recursive condition gives $([e_n], [e_w]) \in \llbracket \text{vec} \rrbracket$ (the index tuple of the recursive argument is $(\llbracket n \rrbracket_{\rho_0}) = ([e_n])$), so by the inner induction $[e_w] = \text{enc}(a_2, \dots, a_n)$ -image with $[e_n] = [S^{n-1}(\mathbf{O})]$; moreover $[e_a] \in \llbracket A \rrbracket$, and $\vec{d} = (\llbracket S n \rrbracket_{\rho_0}) = ([S^n(\mathbf{O})])$; hence $(\vec{d}, d) = \text{enc}([e_a], a_2, \dots, a_n)$. The bijection $\mathbb{N} \cong \llbracket \text{nat} \rrbracket$ is the $m = 0$ instance, as in [Cza26]. The final claim unfolds Definition 6.3 clause by clause under the bijections. $\square$

Remark 7.13 (Postulated axioms). If $\mathbb{E}$ additionally contains postulates $c : \varphi$ (or $c : \sigma$) without bodies, all results relativise as in [Cza26]: Theorem 7.8 goes through provided each postulate is true in the semantics ($\langle \varphi \rangle = 1$; for a postulated constant, $[c] \in \llbracket \sigma \rrbracket$ under the evident extension of $\mathfrak{M}$ — re-run the construction of Section 6 over the term language enlarged by the constant c with no δ -rule; constructor and compiled-symbol interpretations are given by the same clauses with the same witnesses, c itself denotes its own class $[c]$, and every validity argument is unaffected, none of them consulting the absence of extra constants). Excluded middle and proof irrelevance are true; functional extensionality is *not* (Remark 8.20); a postulated inhabitant of an empty instance ($c : \text{fin}(\mathbf{O})$) is not, and voids the guarantee — which is the correct account of the corresponding SMT-datatype hazard (Example 5.15): the danger is never the empty instance itself but an axiom asserting it inhabited.

Remark 7.14 (What is not claimed). Theorem 7.9 does not claim G is provable in CIC_{ix}^- : the semantics validates excluded middle, proof irrelevance and (as spent in the $\text{scase}(\cdot; \cdot)$ case) uniqueness of identity proofs, so $\mathcal{F}(G)$ may be provable for G true-but-unprovable. This matches the design of the implemented tool, which runs classical provers and relies on reconstruction. Section 8.5 localises the UIP debt precisely.

8 Fording at the level of formulas: design questions settled

The model $\mathfrak{M}$ was built once; this section spends it. Each subsection takes a design question documented in implementations of related translations — where may the forced inversion disjunction be inlined? which guards are load-bearing? do match equations need index premises? what may be said about indices without saying it about type formers? — and settles it as a theorem about $\mathfrak{M}$ or about the theory $\Delta(\mathbb{E}, G)$. Throughout, $\mathbb{E}$ is well formed and definitional with $\mathfrak{S}$ declarations.

8.1 Per-occurrence expansion of forced guards

The translation of Section 5 keeps guard atoms atomic and states the ford once per family, in the inversion axiom. A shallower variant — the analogue, for families, of the inline expansion of refinement payloads — replaces a guard atom, at any chosen occurrence, by the corresponding disjunction of constructor payloads and index equations. This is sound, in both polarities, and changes nothing logically:

Definition 8.1 (One-level expansion). For a datatype I as in Definition 5.13, define the first-order formula (with free variables $w, \vec{z}$)

$$\mathcal{X}_I(w, \vec{z}) := \bigvee_i \exists \vec{y}. \bigwedge_j \mathcal{G}_{A_{ij}}(y_j) \wedge \bigwedge_l \mathcal{F}(\varphi_{il}) \wedge w \approx \widehat{c}_i(\vec{y}) \wedge \bigwedge_h z_h \approx \mathcal{C}(\mathcal{E}(t_{ih})),$$

i.e. the right-hand side of the inversion axiom; for an inductive predicate J , let $\mathcal{X}_J(\vec{z})$ be the right-hand side of its case-analysis axiom. An *expansion variant* of the translation is any pair $(\mathcal{G}', \mathcal{F}')$ obtained from Definition 5.10 by replacing, at any chosen set of occurrences in the structural recursion, the clause value $\widehat{I}(u, \mathcal{C}(\mathcal{E}(\vec{a})))$ by $\mathcal{X}_I(u, \mathcal{C}(\mathcal{E}(\vec{a})))$ and/or atoms $\widehat{J}(\mathcal{C}(\mathcal{E}(\vec{a})))$ by $\mathcal{X}_J(\mathcal{C}(\mathcal{E}(\vec{a})))$ (arguments substituted for $w, \vec{z}$). The variant theory $\Delta'(\mathbb{E}, G)$ is Definition 5.13 with items 3–5 (specification, premise and goal formulas and their auxiliary axioms) computed by $(\mathcal{G}', \mathcal{F}')$; items 1–2 (structural axioms) are unchanged.

Note that the expansion is *one level*: the guards $\mathcal{G}_{A_{ij}}(y_j)$ inside $\mathcal{X}_I$ are the atomic guards of the base translation, so the definition is not recursive and expanded formulas are finite regardless of recursion in I . Constraint discharge remains pay-as-you-go, one constructor layer per expansion, exactly as in the source calculus (Section 2.2).

Lemma 8.2 (Expansion is valid). $\mathfrak{M} \models \forall w \vec{z}. \widehat{I}(w, \vec{z}) \leftrightarrow \mathcal{X}_I(w, \vec{z})$, and $\mathfrak{M} \models \forall \vec{z}. \widehat{J}(\vec{z}) \leftrightarrow \mathcal{X}_J(\vec{z})$.

Proof. Left to right is the validity of inversion resp. case analysis (Theorem 7.8). Right to left, for I : given a disjunct's witnesses $\vec{d}$ satisfying its conjuncts, Lemma 6.13 converts the guard and premise conjuncts into memberships and truths at $\rho_0 = [\vec{y} \mapsto \vec{d}]$; Lemma 6.5(5) gives $(\llbracket \vec{t}_i \rrbracket_{\rho_0}, \widehat{c}_i^{\mathfrak{M}}(\vec{d})) \in \llbracket I \rrbracket$; and the equation conjuncts identify $w^{\mathfrak{M}}$ with the constructor value and $\vec{z}^{\mathfrak{M}}$ with the index tuple (using Lemma 6.11 for the right-hand sides). For J : the stage construction closes $\langle J \rangle$ under exactly the displayed conditions. $\square$

Theorem 8.3 (Soundness of expansion variants). *Let $(\mathcal{G}', \mathcal{F}')$ be any expansion variant. Then Lemma 6.13 holds verbatim for $(\mathcal{G}', \mathcal{F}')$; consequently $\mathfrak{M} \models \Delta'(\mathbb{E}, G)$, and if $\Delta'(\mathbb{E}, G) \vdash_{\text{FOL}} \mathcal{F}'(G)$ then $\langle G \rangle = 1$. All corollaries of Section 7.4 hold for the variant.*

Proof. The coincidence proof is by structural induction on types and propositions; the only change is at a replaced occurrence, where $\mathfrak{M}, \rho \models \mathcal{X}_I(u, \mathcal{C}(\mathcal{E}(\vec{a})))$ iff $\mathfrak{M}, \rho \models \widehat{I}(u, \mathcal{C}(\mathcal{E}(\vec{a})))$ by Lemma 8.2 (a universally valid equivalence may be instantiated at the argument terms), and the base-translation coincidence handles the atom. The auxiliary axioms emitted by computing the $\mathcal{C}$ -translations inside expanded occurrences are compilation axioms, valid by Lemma 6.12. With coincidence in place, the proofs of Lemma 7.7, Theorem 7.8 and Theorem 7.9 go through unchanged for items 3–5, and items 1–2 are shared with $\Delta(\mathbb{E}, G)$. $\square$

Proposition 8.4 (Expansion changes nothing logically). $\Delta(\mathbb{E}, G) \vdash_{\text{FOL}} \forall w \vec{z}. \widehat{I}(w, \vec{z}) \leftrightarrow \mathcal{X}_I(w, \vec{z})$ (from the constructor-typing and inversion axioms of I), and likewise for J from clause and case-analysis axioms. Consequently $\Delta(\mathbb{E}, G)$ and any $\Delta'(\mathbb{E}, G)$ are mutually derivable, and the choice among expansion variants is purely operational.

Proof. Left to right is the inversion axiom. Right to left: in the i -th disjunct, the witnesses' guards and premises feed the constructor-typing axiom of c_i , yielding $\widehat{I}(\widehat{c}_i(\vec{y}), \mathcal{C}(\mathcal{E}(\vec{t}_i)))$; rewriting with $w \approx \widehat{c}_i(\vec{y})$ and $z_h \approx \mathcal{C}(\mathcal{E}(t_{ih}))$ gives $\widehat{I}(w, \vec{z})$. Mutual derivability of the theories: each axiom of Δ' differs from the corresponding axiom of $\Delta(\mathbb{E}, G)$ by replacements of subformulas by $\Delta(\mathbb{E}, G)$ -provably equivalent ones (both theories contain items 1–2), so each direction follows by the replacement theorem of classical first-order logic. $\square$

Example 8.5 (The reflect payoff). For rfl_{φ_0} (Example 5.14), a hypothesis occurrence $r : \text{rfl}_{\varphi_0}(b)$ inside a premise or goal expands to

$$(\mathcal{F}(\varphi_0) \wedge r \approx \widehat{\text{RT}} \wedge b \approx \widehat{\text{true}}) \vee (\mathcal{F}(\neg\varphi_0) \wedge r \approx \widehat{\text{RF}} \wedge b \approx \widehat{\text{false}}).$$

Both directions of the view are now local: from $b \approx \widehat{\text{true}}$ the second disjunct dies by discrimination on `bool`, leaving $\mathcal{F}(\varphi_0)$ (elimination is no longer too weak); and in a positive position, establishing the instance at tag $\widehat{\text{RF}}$ together with $b \approx \widehat{\text{true}}$ is impossible for the same reason (introduction cannot cheat

the index). In the atomic variant the same two inferences route through the declaration-level inversion axiom. To see the advertised prover behaviour once, here is the refutation at the clause level for the forward direction of the view — the goal $\forall b r. \widehat{\text{rfl}}(r, b) \rightarrow b \approx \widehat{\text{true}} \rightarrow \mathcal{F}(\varphi_0)$ (the carrier guard $\widehat{\text{bool}}(b)$ contributed by the **bool**-quantifier is elided; it plays no role in the refutation) — from the atomic translation:

1. $\widehat{\text{rfl}}(r_0, b_0); \quad b_0 \approx \widehat{\text{true}}; \quad \neg \mathcal{F}(\varphi_0)$ (negated goal, Skolemised)
2. $(\mathcal{F}(\varphi_0) \wedge r_0 \approx \widehat{\text{RT}} \wedge b_0 \approx \widehat{\text{true}}) \vee (\mathcal{F}(\neg \varphi_0) \wedge r_0 \approx \widehat{\text{RF}} \wedge b_0 \approx \widehat{\text{false}})$
(resolve 1 with the inversion axiom of $\widehat{\text{rfl}}_{\varphi_0}$)
3. left disjunct: $\mathcal{F}(\varphi_0)$ contradicts $\neg \mathcal{F}(\varphi_0)$;
4. right disjunct: $b_0 \approx \widehat{\text{false}}$ with $b_0 \approx \widehat{\text{true}}$ gives $\widehat{\text{true}} \approx \widehat{\text{false}}$, refuted by discrimination for **bool**.

In the expanded variant the hypothesis arrives *as* the disjunction of step 2: the resolution of step 2 disappears, and with it the requirement that premise selection offered the inversion axiom at all. Which variant is operationally better on balance is an empirical question that this paper deliberately does not attempt to answer — no benchmark evaluation is included, and the idealisation of Section 1.4 would make one premature; the content of Theorem 8.3 and Proposition 8.4 is that it is *only* an operational question.

Remark 8.6 (Existential residue and size). Expansion introduces $\exists \vec{y}$ in guard positions; in negative positions these become universals after clausification. For enumeration-like and subset-like families the residue is empty or a single carrier variable after Lemma 5.17 (**rfl**: none; **bnd**: one), which is exactly the class of families for which implementations enable expansion; for recursive families like **vec** the atomic guard is the reasonable default. The cost is bounded and does not compound: each expanded occurrence adds one copy of $\mathcal{X}_I$, of size

$$|\mathcal{X}_I| = \sum_{i \leq M} \left(r_i + \sum_j |\mathcal{G}_{A_{ij}}(\cdot)| + \sum_l |\mathcal{F}(\varphi_{il})| + (m + 1) \right)$$

— per constructor, the existential block, the argument guards, the premise images, the tag equation and the m index equations — linear in the size of I 's *translated declaration*, independent of how many other occurrences are expanded and of any recursion in I , and not iterable: the guards inside $\mathcal{X}_I$ are atoms of the base translation by construction. Shallow encodings have historically died of formula blowup; one-level fording caps the growth at declaration size per occurrence. Nothing in the soundness statement forces either choice.

8.2 Which guards are load-bearing

The companion paper proved that for non-indexed datatypes the guard of the inversion axiom is semantically mandatory (false at junk) while the per-constructor equations need no guards at all. For families both halves sharpen.

Proposition 8.7 (Unguarded inversion is inconsistent). *Let I be a datatype of $\mathbb{E}$ with $m \geq 1$ indices, and let Inv_I^- be the inversion axiom of I with its guard atom $\widehat{I}(w, \vec{z})$ deleted. Write $\text{Str}(I)$ for the injectivity and discrimination axioms of I — axioms the translation must emit in any case. Then:*

1. $\{\text{Inv}_I^-\} \cup \text{Str}(I) \vdash_{\text{FOL}} \forall z z'. z \approx z'$: the unguarded axiom collapses the universe to a point.
2. Consequently $\{\text{Inv}_I^-\} \cup \text{Str}(I) \cup \{\delta\}$ is inconsistent for every discrimination axiom δ of every datatype of $\mathbb{E}$. In particular, if I itself has two constructors, $\{\text{Inv}_I^-\} \cup \text{Str}(I)$ is already inconsistent; and if $\mathbb{E}$ declares any datatype with two constructors (`bool`, `nat`, ...), the unguarded inversion axiom of every family whatsoever is inconsistent with axioms the translation emits unconditionally.
3. If moreover some constructor c_i of I has a constructor-headed index term $t_{ih} = c'(\dots)$ whose datatype I' has a second constructor $d \neq c'$, then discrimination alone suffices: $\{\text{Inv}_I^-\}$ together with the discrimination axioms of I and of I' is inconsistent, with no use of injectivity.

In particular, for `vec` (Example 5.14) the theory consisting of $\text{Inv}_{\text{vec}}^-$ and the discrimination axioms of `nat` and `vec` is inconsistent.

Proof. (1) If I has no constructors, Inv_I^- is the universal closure of the empty disjunction, i.e. of $\perp$, and the collapse — indeed anything — follows outright; assume below that I has at least one constructor. Take fresh variables $w, \vec{z}, \vec{z}'$ and the two instances $\text{Inv}_I^-(w, \vec{z})$ and $\text{Inv}_I^-(w, \vec{z}')$ — the same element variable, two arbitrary index tuples. Reason by cases on both disjunctions. A case pairs a disjunct i , with witnesses $\vec{y}$ giving $w \approx \widehat{c}_i(\vec{y})$ and $z_h \approx \mathcal{C}(\mathcal{E}(t_{ih}))[\vec{y}]$ for all h , against a disjunct i' with witnesses $\vec{y}'$ (the guard and premise conjuncts are discarded). If $i \neq i'$: the two tag equations give $\widehat{c}_i(\vec{y}) \approx \widehat{c}_{i'}(\vec{y}')$, refuted by discrimination for I , and the case closes vacuously. If $i = i'$ and $r_i = 0$: the index equations have identical closed right-hand sides, so $z_h \approx z'_h$. If $i = i'$ and $r_i > 0$: injectivity for c_i turns the two tag equations into $\vec{y} \approx \vec{y}'$ componentwise, and congruence of $\approx$ gives $\mathcal{C}(\mathcal{E}(t_{ih}))[\vec{y}] \approx \mathcal{C}(\mathcal{E}(t_{ih}))[\vec{y}']$ — the same first-order term at componentwise-equal arguments — hence $z_h \approx z'_h$. Every case yields $z_1 \approx z'_1$, so $\forall w \vec{z} \vec{z}'. z_1 \approx z'_1$; as z_1, z'_1 are two distinct universally quantified variables, $\forall z z'. z \approx z'$ follows.

(2) From the collapse, instantiate z, z' at $\widehat{\mathbf{d}}_1(\vec{y})$ and $\widehat{\mathbf{d}}_2(\vec{y}')$ for two distinct constructors of δ 's datatype, contradicting δ .

(3) Instantiate Inv_F^- at $w := \widehat{\mathbf{c}}_i(\vec{x})$, $z_h := \widehat{\mathbf{d}}(\vec{x}')$, the remaining z 's fresh ($\vec{x}, \vec{x}'$ fresh). Every disjunct $i' \neq i$ contains the equation $\widehat{\mathbf{c}}_i(\vec{x}) \approx \widehat{\mathbf{c}}_{i'}(\vec{y})$, refuted by discrimination for I ; the disjunct i contains $\widehat{\mathbf{d}}(\vec{x}') \approx \mathcal{C}(\mathcal{E}(t_{ih}))[\vec{y}]$, whose right-hand side is $\widehat{\mathbf{c}}'$ -headed ($\mathcal{C}(\mathcal{E}(\mathbf{c}'(\dots)))$ is an application of $\mathbf{c}'$ by Definition 5.2), refuted by discrimination for I' . All disjuncts refuted, the theory proves $\perp$. For **vec**, concretely: $w := \widehat{\mathbf{vnil}}$, $z := \widehat{\mathbf{S}}(x')$ — the **vnil** disjunct dies on $\widehat{\mathbf{S}}(x') \approx \widehat{\mathbf{O}}$, the **vcons** disjunct on $\widehat{\mathbf{vnil}} \approx \widehat{\mathbf{vcons}}(n, a, v)$. $\square$

Remark 8.8. Three comments delimit the blast radius exactly.

First, the mechanism of part (1) is worth naming: it is the derivation of *index uniqueness* (Proposition 8.16) with the two guard atoms replaced by instances of the unguarded axiom. Fording makes “every member determines its index tuple” a first-order consequence of the structural axioms; an unguarded inversion asserts that every element is a member at *every* index tuple; the two statements annihilate. This strictly sharpens the non-indexed situation of [Cza26], where guard omission made the disjunctive axiom false in $\mathfrak{M}$ but inconsistency needed further apparatus: here the family's own index equations detonate it, whatever the shape of its index patterns.

Second, the two-constructor hypothesis in part (2) is not slack but the exact boundary. Since part (1) derives the total collapse, the variant theory is consistent if and only if it has a one-element model; any discrimination axiom refutes that, and in its absence the collapse can be realised. Concretely, let $\mathbb{E}_{\text{unit}}$ declare exactly **data** $U : () \rightarrow \mathbf{Set} := \{\mathbf{tt} : U\}$ and **data** $F : (U) \rightarrow \mathbf{Set} := \{\mathbf{mk} : \forall y:U. F(\mathbf{tt})\}$. Its emitted theory with Inv_F^- in place of F 's inversion axiom is satisfied by the one-element structure (universe $\{\bullet\}$, all atoms true: constructor typing, the guarded inversion of U , injectivity and Inv_F^- all hold outright, and no discrimination axiom exists). Yet Inv_F^- is still *false* in $\mathfrak{M}$: instantiate w at the junk class $[\mathbf{tt} \mathbf{tt}]$, which is no **mk**-form by Corollary 4.4(2). Guard omission is thus unsound always, and inconsistent exactly when the ambient theory can distinguish two elements — which any realistic environment can.

Third, for enumeration-shaped families the collapse can also be reached without touching the index equations, by *cardinality*: for $\mathbf{rfl}_{\varphi_0}$, the unguarded inversion entails $\forall w. w \approx \widehat{\mathbf{RT}} \vee w \approx \widehat{\mathbf{RF}}$ (each disjunct contains the corresponding tag equation as a conjunct), a two-element bound on the whole untyped universe; three provably distinct closed terms — $\widehat{\mathbf{O}}$, $\widehat{\mathbf{S}}(\widehat{\mathbf{O}})$, $\widehat{\mathbf{S}}(\widehat{\mathbf{S}}(\widehat{\mathbf{O}}))$ in any environment declaring **nat** — force two of them equal by pigeonhole, hence $\perp$. This is Meng and Paulson's two-element universe [MP08, §2.3]; exhaustiveness sentences are exactly the sentences whose guards can never be dropped [BBPS16].

For the implementation the verdict is uniform: an indexed inversion axiom accidentally emitted unguarded is typically not a weak axiom but a

refutation-generator, poisoning every problem it enters (cf. Corollary 7.10).

Proposition 8.9 (Index arguments of guards are mandatory). *Consider the index-forgetting variant of the translation: guard predicates $\widehat{I}^-$ of arity 1, guard clause $\mathcal{G}_{I(\vec{a})}^-(u) = \widehat{I}^-(u)$, and the axioms of Definition 5.13 recomputed accordingly (index equations deleted from inversion, index arguments deleted from constructor-typing conclusions). This variant is unsound: for $\mathbb{E}$ declaring **nat** and **vec** (over $A = \text{nat}$) there is a closed shallow goal G with $\langle G \rangle = 0$ whose translation is derivable,*

$$G := \exists v : \text{vec}(\mathbf{O}). \neg(v =_{\text{vec}(\mathbf{O})} \text{vnil}),$$

$$\Delta^-(\mathbb{E}, G) \vdash_{\text{FOL}} \exists v. \widehat{\text{vec}}^-(v) \wedge \neg(v \approx \widehat{\text{vnil}}).$$

Proof. $\langle G \rangle = 0$: by Lemma 6.5(1) and Corollary 4.4, every $(\llbracket \mathbf{O} \rrbracket, d) \in \llbracket \text{vec} \rrbracket$ decomposes by **vnil** — the **vcons** index tuple is a class $[S(e)] \neq [\mathbf{O}]$ — so $\llbracket \text{vec}(\mathbf{O}) \rrbracket = \{\llbracket \text{vnil} \rrbracket\}$ and the matrix is false at the only element.

Derivability: the index-forgetting constructor-typing axioms are $\widehat{\text{vec}}^-(\widehat{\text{vnil}})$ and $\forall n a v. \widehat{\text{nat}}(n) \rightarrow \widehat{\text{nat}}(a) \rightarrow \widehat{\text{vec}}^-(v) \rightarrow \widehat{\text{vec}}^-(\widehat{\text{vcons}}(n, a, v))$. From $\widehat{\text{nat}}(\widehat{\mathbf{O}})$ (constructor typing for **nat**) instantiate $n = a := \widehat{\mathbf{O}}$, $v := \widehat{\text{vnil}}$ to get $\widehat{\text{vec}}^-(\widehat{\text{vcons}}(\widehat{\mathbf{O}}, \widehat{\mathbf{O}}, \widehat{\text{vnil}}))$; discrimination for **vec** gives $\neg(\widehat{\text{vcons}}(\dots) \approx \widehat{\text{vnil}})$; the displayed existential follows with witness $\widehat{\text{vcons}}(\widehat{\mathbf{O}}, \widehat{\mathbf{O}}, \widehat{\text{vnil}})$. $\square$

Remark 8.10 (The F^* #1542 shape). Proposition 8.9 is the indexed-family form of a documented unsoundness. In F^* #1542 [Ase18], the type-indexed extensional equality **feq** # a — “equal pointwise at domain a ” — was encoded through the SMT solver’s untyped $=$; an equality whose *meaning depends on an index* was represented by an atom that forgot the index, and facts established at index **nat** were consumed at index **int**, proving **False**. Here, family membership — whose meaning depends on the index — is represented in the variant by an atom $\widehat{\text{vec}}^-(v)$ that forgets it, and membership established at index $S(\mathbf{O})$ is consumed at index $\mathbf{O}$. The polarity analysis explains where the damage occurs: forgetting indices in *negative* (premise) positions merely strengthens hypotheses and weakens the translation; in *positive* (conclusion, existential-witness) positions it lets the prover manufacture members of the wrong instance, as in the derivation above. Since one guard clause serves both polarities, the indices must stay. The translation of Section 5 is immune for a reason worth stating precisely: it is not that its untyped $\approx$ is somehow safer than F^* ’s — it is that *no index-dependent notion of the source is represented by an index-blind notion of the target*. Source equality $a =_\sigma b$ is index-blind (σ is not consulted) because its *semantics* in $\mathfrak{M}$ is index-blind class equality (Definition 6.3), and family membership is index-aware because its semantics is (Lemma 6.13); each atom is exactly as discriminating as the semantic notion it names. The residual caveat is the intensionality of that semantics: see Remark 8.20.

8.3 Match equations for indexed scrutinees

Proposition 8.11 (Unguarded equations, impossible branches included). *Every equation emitted by the compilation of a definition matching on a family — with no guard on the matched term, no guards on the constructor-argument variables, and no index premises, and including the equations of branches whose index patterns clash with the scrutinee occurrence’s indices — holds in $\mathfrak{M}$ under every assignment. In particular (Example 5.9):*

$$\begin{aligned}\mathfrak{M} \models \forall n. \widehat{\text{vhead}}(n, \widehat{\text{vnil}}) &\approx \odot, \\ \mathfrak{M} \models \forall n k a w. \widehat{\text{vhead}}(n, \widehat{\text{vcons}}(k, a, w)) &\approx a.\end{aligned}$$

Proof. An instance of Lemma 6.12(2). The point of displaying it: the ι -rule of $\lambda_{\square}$ fires on any constructor form with no index side condition, so validity of the equations is a fact about the *program*, blind to the typing discipline that made the branch impossible; the branch body is the erasure of the $\perp$ -elimination, namely $\square$. $\square$

Proposition 8.12 (Index-premised equations are strictly weaker). *Fix the definition vhead and let the premised variant of its vnil -equation be*

$$\text{Eq}_{\text{vnil}}^{\pi} := \forall n. \widehat{S}(n) \approx \widehat{O} \rightarrow \widehat{\text{vhead}}(n, \widehat{\text{vnil}}) \approx \odot,$$

and analogously $\text{Eq}_{\text{vcons}}^{\pi} = \forall n k a w. \widehat{S}(n) \approx \widehat{S}(k) \rightarrow \widehat{\text{vhead}}(n, \widehat{\text{vcons}}(k, a, w)) \approx a$ (premises: occurrence indices $\approx$ constructor index pattern). Then:

1. *each premised equation is derivable from its unguarded form, hence valid in $\mathfrak{M}$;*
2. *the unguarded vnil -equation is not derivable from the theory obtained from $\Delta(\mathbb{E}, G)$ by replacing vhead ’s equations with their premised forms;*
3. *deleting the vnil -equation altogether (rigid-clash pruning) preserves Theorem 7.8, hence Theorem 7.9 and Corollary 7.10.*

Proof. (1) Weakening. (3) Validity of a subset of a valid axiom set.

(2) Let $\mathbb{E}_0$ declare nat , $A := \text{nat}$, vec and vhead only, and let Δ^{π} be the resulting theory with vhead ’s two equations replaced by their premised forms. We build $\mathfrak{M}' \models \Delta^{\pi}$ falsifying the unguarded vnil -equation. A pointwise reinterpretation of $\widehat{\text{vhead}}$ is not available: the pointer axiom $\forall z_1 z_2. \widehat{\text{vhead}}^0 @ z_1 @ z_2 \approx \widehat{\text{vhead}}(z_1, z_2)$ ties the symbol to the applicative structure of $\mathcal{D}$, which is fixed. Instead, reinterpret both symbols through an *alternative witness*: let

$$\kappa := \lambda n v. \text{case}(v; \{\text{vnil} \Rightarrow \text{O} \mid \text{vcons}(k, a, w) \Rightarrow a\})$$

— the erasure of $\mathbf{vhead}$ with the dead code of the impossible branch replaced by $\mathbf{O}$ — and let $\mathfrak{M}'$ agree with $\mathfrak{M}$ on all symbols except $\widehat{\mathbf{vhead}}^0_{\mathfrak{M}'} := [\kappa]$ and $\widehat{\mathbf{vhead}}^{\mathfrak{M}'}(d_1, d_2) := [\kappa e_1 e_2]$ (representatives $e_i \in d_i$). Then: the pointer axiom holds by construction ($@$ is application); the premised $\mathbf{vnil}$ -equation holds vacuously, its premise $\widehat{\mathbf{S}}(n) \approx \widehat{\mathbf{O}}$ being false (Corollary 4.4; constructor interpretations are unchanged); the premised $\mathbf{vcons}$ -equation holds because $\kappa e_1 \mathbf{vcons}(\vec{e}) \rightarrow_{\square}^* e_a$, and this is where κ agrees with $\mathcal{E}(\mathbf{vhead})$; the specification axiom $\forall n. \widehat{\mathbf{nat}}(n) \rightarrow \forall v. \widehat{\mathbf{vec}}(v, \widehat{\mathbf{S}}(n)) \rightarrow \widehat{\mathbf{nat}}(\widehat{\mathbf{vhead}}(n, v))$ constrains the symbol only at $v \in \llbracket \mathbf{vec}(\mathbf{S} n) \rrbracket$ -instances, which by Lemma 6.5(1,3) are $\mathbf{vcons}$ -forms ($[\mathbf{vnil}]$'s unique index tuple is $[\mathbf{O}]$), where $\mathfrak{M}'$ computes the same values as $\mathfrak{M}$, so the axiom transfers from Theorem 7.8; and all axioms not mentioning $\mathbf{vhead}$'s symbols hold as in $\mathfrak{M}$. Finally the unguarded $\mathbf{vnil}$ -equation fails: $\widehat{\mathbf{vhead}}^{\mathfrak{M}'}(d, [\mathbf{vnil}]) = [\kappa e \mathbf{vnil}] = [\mathbf{O}] \neq [\square] = \odot^{\mathfrak{M}'}$, the last two being distinct $\rightarrow_{\square}$ -normal forms (Theorem 4.3). $\square$

Remark 8.13 (A family of countermodels: dead code is undetermined). The countermodel is one of a family, uniformly in the repair. For an *arbitrary* closed $\lambda_{\square}$ term e_0 , let κ_{e_0} be $\mathcal{E}(\mathbf{vhead})$ with the dead branch body replaced by e_0 (the proof used $e_0 = \mathbf{O}$), and let $\mathfrak{M}'_{e_0}$ reinterpret $\widehat{\mathbf{vhead}}^0$ and $\widehat{\mathbf{vhead}}$ through the witness κ_{e_0} exactly as above. Every verification step of the proof holds verbatim, because none consults the dead-branch value: the pointer axiom holds by construction; the premised $\mathbf{vnil}$ -equation is vacuous whatever the branch computes; and the premised $\mathbf{vcons}$ -equation and the specification axiom only ever evaluate κ_{e_0} at $\mathbf{vcons}$ -forms, where all repairs agree with $\mathcal{E}(\mathbf{vhead})$. Hence $\mathfrak{M}'_{e_0} \models \Delta^{\pi}$ for every e_0 , while $\widehat{\mathbf{vhead}}^{\mathfrak{M}'_{e_0}}(d, [\mathbf{vnil}]) = [e_0]$ ranges over *all* of $\mathcal{D}$ as e_0 ranges over closed terms. So Δ^{π} determines nothing whatever about the dead branch: it cannot distinguish the extracted program from any repair of its dead code, every axiom that could see the branch being either premised on a refutable index equation or guarded by an uninhabited instance. The unguarded theory, by contrast, pins the branch to $\odot$ — which is exactly the additional, sound content Proposition 8.11 attributes to it.

Proposition 8.14 (Pruned branches are semantically dead). *Say two terms rigidly clash if both are constructor-headed with distinct constructors, or both are applications of the same constructor and some pair of corresponding informative arguments rigidly clashes; extend to index tuples by “some position clashes”. If, in a definition $c := \lambda \vec{x}. \mathbf{case}(x_k; \dots; \dots)$ (or the $\mathbf{fix}$ form) with $x_k : I(\vec{v})$, the occurrence indices $\vec{v}$ rigidly clash with the index pattern $\vec{t}_i$ of constructor $\mathbf{c}_i$, then for every valuation ρ of $\vec{x}$ with $\rho(x_k) \in \llbracket I(\vec{v}) \rrbracket_{\rho}$, the value $\rho(x_k)$ is not of the form $[\mathbf{c}_i(\vec{e})]$. Consequently the pruned equation constrains $\widehat{c}$ only at argument tuples excluded by the guards of c 's specification axiom.*

Proof. First, rigid clash implies semantic distinctness at all valuations: by induction on the definition, if u and t rigidly clash then $\llbracket u \rrbracket_{\rho_1} \neq \llbracket t \rrbracket_{\rho_2}$ for all ρ_1, ρ_2 — distinct constructor heads give distinct classes (Corollary 4.4(1)), and for equal heads, componentwise equality of classes would contradict the inner clash by the induction hypothesis (erasure preserves constructor heads of informative arguments). Now suppose $\rho(x_k) = [c_i(\vec{e})] \in \llbracket I(\vec{v}) \rrbracket_\rho$. By Lemma 6.5(1,3) the unique index tuple of this element is $\llbracket t_i \rrbracket_{\rho_0}$, so $\llbracket \vec{v} \rrbracket_\rho = \llbracket \vec{t}_i \rrbracket_{\rho_0}$, contradicting semantic distinctness at the clashing position. $\square$

Remark 8.15 (Choice of equation form). Propositions 8.11, 8.12 and 8.14 bound the design space: the unguarded equations are the strongest sound form and are what the factored architecture naturally produces; index premises are sound but strictly weaker, buying nothing semantically (their one virtue is to keep the equation silent at wrong-index instantiations, a reconstruction-signal nicety); pruning is sound and removes only equations that no guarded reasoning can reach. Whether pruning is also *conservative* — whether removing the pruned equations changes the set of provable translated goals — we do not know. Both theories hold in $\mathfrak{M}$, so the model separates neither from the other; we know of no goal whose derivation requires a junk instance of a pruned equation, but we also have no countermodel argument in the style of Proposition 8.12 (the pruned equation constrains the compiled symbol at instances that the specification axiom’s guards do not reach, but other defining equations may). Until one of the two is exhibited, pruning is justified operationally, not semantically. All three verdicts are instances of the companion paper’s division of labour: equations carry computation, valid everywhere; typing content lives in guarded specification and inversion axioms.

8.4 Indices, elements, and injectivity of type formers

Proposition 8.16 (Index uniqueness is a first-order consequence). *For every datatype I ,*

$$\Delta(\mathbb{E}, G) \vdash_{\text{FOL}} \forall w \vec{z} \vec{z}'. \widehat{I}(w, \vec{z}) \rightarrow \widehat{I}(w, \vec{z}') \rightarrow \bigwedge_h z_h \approx z'_h,$$

and the formula holds in $\mathfrak{M}$.

Proof. Model validity is Lemma 6.5(3). Derivability: apply inversion to both hypotheses, obtaining disjuncts i (witnesses $\vec{y}$) and i' (witnesses $\vec{y}'$) with $w \approx \widehat{c}_i(\vec{y}) \approx \widehat{c}_{i'}(\vec{y}')$. If $i \neq i'$, discrimination refutes the branch. If $i = i'$, injectivity gives $y_j \approx y'_j$ for all j ; each index equation pair $z_h \approx \mathcal{C}(\mathcal{E}(t_{ih}))[\vec{y}]$, $z'_h \approx \mathcal{C}(\mathcal{E}(t_{ih}))[\vec{y}']$ then yields $z_h \approx z'_h$ by congruence of $\approx$ (the two right-hand sides are the same first-order term at componentwise-equal arguments). For $r_i = 0$ the two right-hand sides are identical closed terms. (Proposition 8.7(1) is this derivation with the two guard atoms replaced by

instances of the unguarded axiom — which is why unguarding is not merely false but explosive.) $\square$

Remark 8.17 (Injectivity of type formers is neither expressible nor wanted). Index uniqueness is the sound first-order shadow of the type-theoretic intuition “ $I(\vec{u})$ and $I(\vec{u}')$ are disjoint unless $\vec{u} = \vec{u}'$ ”: it speaks about *elements*. The stronger, type-level statement — injectivity of the type former, $I\vec{u} = I\vec{u}' \rightarrow \vec{u} = \vec{u}'$ — is not even a formula of $\Sigma_{\mathbb{E}}$: types are not terms in this translation (Definition 5.1), so there is no equality between type images to invert. This inexpressibility is a feature. In type theory, injectivity of inductive type constructors is *anti-classical*: Hur’s construction [Hur10] refutes excluded middle in Agda with injective type constructors, by a Cantor-style diagonal through an injection $(\text{Set} \rightarrow \text{Set}) \rightarrow \text{Set}$ — so a translation validating classical logic, as any ATP-facing translation must, cannot emit any image of it. Encodings that do represent type expressions as first-order terms (types-as-terms with a `HasType` predicate) must therefore be careful that no axiom makes the type-term constructor injective-and-surjective in a way that imports the paradox; in the present design the question cannot arise. The same discipline governs constructor discrimination: the axioms of Definition 5.13 concern fully applied constructor images only — there are no partially applied constructor symbols in $\Sigma_{\mathbb{E}}$ at all, constructors being r -ary function symbols — which is exactly the boundary drawn by Counterexample 35 of [CD18]: conflict between *under-applied* constructors is refutable under functional extensionality (Agda issue #1497) and must not be asserted by a translation whose source may consistently assume it.

8.5 Singleton collapse and uniqueness of identity proofs

The `scase`(; ;) and `cast`(; ;) cases of Lemma 7.6, and the unconditional identity equations that user-written transports compile to (Example 5.16), are the exact locations where the model’s proof irrelevance is spent as *uniqueness of identity proofs*. We make the accounting precise.

Fact 8.18 (The semantic UIP surrogate). *Let J be a forced singleton with clause $k : \forall \vec{x}:\vec{\tau}. \vec{\varphi} \rightarrow J(\vec{u}^0)$, solution j , and let $J(\vec{u})$ be an occurrence with solution substitution θ . For every valuation ρ with $\langle J(\vec{u}) \rangle_{\rho} = 1$:*

$$\llbracket \theta(\vec{u}^0) \rrbracket_{\rho} = \llbracket \vec{u} \rrbracket_{\rho} \quad \text{and} \quad \langle \varphi_l[\theta] \rangle_{\rho} = 1 \text{ for all } l.$$

Proof. Extracted from the `scase`(; ;) case of Lemma 7.6, which proves exactly this. $\square$

In type-theoretic terms, Fact 8.18 says: the mere *truth* of the singleton family at $\vec{u}$ collapses the motive instance at the generic (constructor) indices with the motive instance at $\vec{u}$, and recovers the premises — without inspecting any proof. Gilbert, Cockx, Sozeau and Tabareau prove that this

collapse, availed as a definitional principle for an equality-like family under proof irrelevance, *is* UIP [GCST19, §2.2]; and the without-K analysis of pattern matching isolates the same strength in the unification steps of deletion and of injectivity at non-linear or non-self-unifiable indices [CDP14, CD18]. Our model validates all of them at once, because $\langle \cdot \rangle$ is proof-irrelevant and Lemma 6.5 holds; consequently:

- *Deletion* costs nothing: a match on $\text{eq}_\sigma(a, a)$ (or a $\text{cast}(\cdot; \cdot)$ along it) is validated by Fact 8.18 with $\theta(\vec{u}^0)$ and $\vec{u}$ literally equal.
- *Non-linear index patterns* cost nothing: nothing in Definition 3.7 or in Lemma 6.5 restricts repeated variables or self-unifiability; the without-K side conditions exist to avoid UIP, which the model has.
- *Heterogeneity* costs nothing at the target: the index equations of Definition 5.13 are pointwise untyped equations; the formation problem that motivates John Major equality [McB99] does not exist in $\Sigma_{\mathbb{E}}$.

What the strength *costs* is reconstruction, not soundness: a first-order proof that uses a transport-collapse equation at a type without decidable equality may need UIP (or an axiom-free inversion argument where one exists [Mon10, CD18]) to replay in CIC. This is the same debt the companion paper documents for $\text{cast}(\cdot; \cdot)$ -erasure, unchanged in kind; the premise-carrying variant below eliminates it at the price of weaker axioms.

Proposition 8.19 (Werner’s guarded collapse as an emission variant). *Let $\mathbb{E}_0$ declare bool , the forced singleton eq_{bool} (Example 3.4), and*

$$c := \lambda x y:\text{bool}. \lambda e:\text{eq}_{\text{bool}}(x, y). \lambda w:\text{bool}. \quad \text{s case}_{\text{eq}_{\text{bool}}}(e; \vec{z}.\text{bool}; \cdot.w),$$

with emitted equation $\forall x y w. \widehat{c}(x, y, w) \approx w$ (Example 5.16, at the constant motive). The premised variant

$$\text{Eq}_c^\pi := \forall x y w. y \approx x \rightarrow \widehat{c}(x, y, w) \approx w$$

(premise: the non-trivial index equation of the ford, $\vec{u} \approx \theta(\vec{u}^0)$) is derivable from the unguarded equation, hence $\mathfrak{M}$ -valid; and the unguarded equation is not derivable from the theory Δ^π obtained from $\Delta(\mathbb{E}_0, \top)$ by replacing it with Eq_c^π .

Proof. Derivability is weakening. For non-derivability, a witness-repair modification of $\mathfrak{M}$ in the style of Proposition 8.12 is not available: the pointer axiom forces the behaviour of $\widehat{c}$ to be the applicative behaviour of some element of $\mathcal{D}$, and the premised equation constrains that behaviour on *every* diagonal triple (d, d, w) , junk included — where a $\lambda_\square$ program that inspects

its arguments is stuck; rather than pursue a definability analysis of $\mathcal{D}$, we exhibit a finite countermodel. Let $\mathfrak{N}$ have universe $\{0, 1\}$ with: $\approx$ interpreted as identity; $\widehat{\text{true}} := 0$, $\widehat{\text{false}} := 1$; $\widehat{\text{bool}} := \{0, 1\}$; $\widehat{\text{eq}_{\text{bool}}} := \{(0, 0), (1, 1)\}$; $\odot := 0$; $u @ v := u \oplus v$ (exclusive or); $\widehat{c}^0 := 0$; and $\widehat{c}(x, y, w) := x \oplus y \oplus w$. Every axiom of Δ^π holds: constructor typing and inversion for **bool** (the universe is $\{0, 1\}$), discrimination ($0 \neq 1$), the clause and case-analysis axioms of eq_{bool} (its interpretation is the diagonal of $\widehat{\text{bool}}$), the specification axiom of c (its conclusion $\widehat{\text{bool}}(\widehat{c}(x, y, w))$ is unconditionally true), the pointer axiom ($((\widehat{c}^0 \oplus x) \oplus y) \oplus w = x \oplus y \oplus w$), and the premised equation ($y = x$ gives $x \oplus x \oplus w = w$). The unguarded equation fails: $\widehat{c}(0, 1, 0) = 1 \neq 0$. $\square$

This is precisely the situation Werner analysed for proof-irrelevant type theories [Wer08]: the unconditional singleton reduction ($\text{Eq_rec } A P a b p e \triangleright p$) “easily breaks the subject reduction property in incoherent contexts,” and his repair keeps the index condition as a guard: reduce only *if* $a =_{\beta_\varepsilon} b$ (generalized to arbitrary singletons: *if* $\vec{u} =_{\beta_\varepsilon} \vec{a}$). The premised equation is that rule with “provable” in place of “definitionally checked” — a strictly weaker demand, which is why both variants are sound here while Werner needs the check for decidability and subject reduction. An implementation can therefore expose the choice as a dial: unguarded collapse (strongest, UIP-debt at reconstruction) or premised collapse (weaker, replayable by plain inversion), per definition or globally. Brady, McBride and McKinnin’s run-time warning — collapsing “runs the risk of reducing a proof of something which cannot be constructed, such as $5 \leq 4$ ” in open contexts [BMM04] — is the same point seen from the operational side; in the model-theoretic setting the absurd-context hazard is neutralised because equations are consequences of one consistent structure (Corollary 7.10), and what remains of it is exactly the premised-versus-unguarded choice.

Remark 8.20 (Intensionality: the residual caveat). $\mathfrak{M}$ interprets equality as $\lambda_\square$ -convertibility, which is intensional: for the standard recursive **add**, $[\lambda x. \text{add } 0 \ x] \neq [\lambda x. x]$ in $\mathcal{D}$ (the left representative is stuck at the variable) although the two classes agree applicatively on all of $\llbracket \text{nat} \rrbracket$. So functional extensionality is *not* validated: environments whose premise lemmas are proved from a funext postulate step outside Theorem 7.9 (Remark 7.13), though Corollary 7.10 is unaffected since no extensionality axiom is ever emitted. This caveat is the honest residue of the F^* #1542 analysis (Remark 8.10): that incident needed *both* an extensionality axiom *and* an index-blind equality encoding; the present translation emits neither, and each half of the immunity is a theorem — Proposition 8.9 for the index-blindness half, the present remark for the extensionality half.

9 Shallowness

Shallowness is the design constraint that shaped every clause of Section 5: first-order ATPs are heavily optimised for the primitives of first-order logic, so a translation is efficient to the extent that the logical structure of the source arrives at the prover *as* first-order structure, with no inhabitation predicate and no bridge axioms to instantiate [Cza18, §1.1]. The companion paper articulates this as a checklist S1–S5 and verifies it for the non-indexed translation; we re-verify each item on the extended translation and add the indexed-specific item S6.

- S1 (Native logical structure).** $\mathcal{F}$ maps every connective and quantifier of a shallow proposition to its first-order counterpart and source equality to $\approx$, at translation time (Definition 5.10); nothing indexed changes this. Expansion variants (Definition 8.1) go further in the same direction: the *content* of a family occurrence — payload and index equations — becomes part of the formula in which it occurs.
- S2 (No inhabitation predicate).** $\Sigma_{\mathbb{E}}$ contains no binary has-type predicate and no term encoding of types (Definition 5.1). The typing-flavoured atoms are the per-family guard predicates, now $(1+m)$ -ary; their index argument places carry *terms*, which is first-order structure the prover indexes and unifies natively. This is the point where the present design parts company with the F^* encoding — to our knowledge, the one production system emitting inversion axioms with index equations for type-level families [The24a, Agu16]: there, types are terms and typing is a fuel-instrumented binary predicate, so inversion must be throttled by fuel and triggers to avoid matching loops between inversion and elimination axioms; here there is no self-instantiating typing axiom to loop on — the inversion axiom is guarded by an atom that only constructor typing produces, and (in expansion variants) hypothesis-side content is inlined at translation time rather than recovered by E-matching at solve time. The residual caveat is SMT-specific: a trigger discipline for the guarded inversion axioms of recursive families would be needed by SMT backends, which have no analogue of a saturation prover’s fair clause selection.
- S3 (Per-occurrence payload expansion).** Refinement payloads are expanded inline at every occurrence, in both polarities, as before. The forded view extends this uniformly to families: Theorem 8.3 licenses inlining constructor payloads *and index equations* at any chosen occurrence, in both polarities, and Proposition 8.4 shows the choice is operational. “Index equations are just more propositional payload.”
- S4 (Proofs never become terms).** Erasure drops proof arguments, premise proofs of constructors, and the index-equation binders of

fording branches (Definition 4.5); specification extraction keeps the term unapplied across propositional premises (Definition 5.11). Proof subterms *inside index terms* vanish too (Remark 4.6), so no proof material reaches the prover even through the new index argument places.

S5 (Definitions as oriented computation). Pattern-matching definitions over families compile to unguarded, orientable, non-overlapping rewrite equations (Remark 5.8, Proposition 8.11) — for vmap_g , literally the recursive program. Case-analysis content (a search operation) is segregated into the guarded inversion axioms, emitted once per family; provers pay for splitting only when a proof needs it.

S6 (Index constraints as equations, stated not solved). All index information of a family reaches the prover as first-order equations between ordinary terms — in inversion disjuncts, in expanded guards, in constructor-typing conclusions — computed at translation time by fording, never by unification. Three classical obstructions dissolve at this interface, each by a theorem of this paper rather than by fiat: heterogeneous telescopic equations are stated pointwise in the untyped $\approx$ (Definition 5.13; the formation problem of [McB99] does not exist in $\Sigma_{\mathbb{E}}$); defined functions in index positions are handled by superposition against their defining equations (Remark 5.19), where elaboration-time unification is stuck [McB14]; and the K/UIP side conditions of dependent pattern matching are validated by the model (Section 8.5). Dually, the three soundness obligations the interface creates are discharged as theorems: guarded exhaustiveness only (Proposition 8.7), index-carrying guard atoms only (Proposition 8.9), and no structural axioms beyond constructor images (Remark 8.17).

The residue. As in the companion paper, partial applications go through $@$ and pointer constants, and anonymous λ -abstractions, matches and fix-points inside formulas are lifted to fresh symbols — unavoidable in a language without binders, and local to terms. The indexed extension adds one residue class: *induction on families* is not first-order and is not emitted (Section 11.4); the emitted content of a family is exactly its ford — constructor typing, guarded inversion, injectivity, discrimination, and the defining equations of functions over it.

10 Related work

Fording and the elimination of dependent pattern matching. The constraint technique is McBride’s [McB99, §3.5], including coalescence (our Lemma 5.17) and the worked vtail example that Example 3.16 replays inside

CIC_{ix}^- ; the datatype-level formulation used in Section 2 is stated by Zhang [Zha23]. Goguen, McBride and McKinna [GMM06] eliminate Coquand-style dependent pattern matching [Coq92] into eliminators plus heterogeneous equality and K, via specialization by unification; their five unification transitions (deletion, solution, injectivity, conflict, cycle) are exactly the first-order theory of constructors, and Remark 5.19 is the observation — made there for elaboration, here as a soundness theorem — that a saturation prover performs these transitions as ordinary equational inference, with defined-function-headed indices handled strictly better. The without-K refinements of Cockx et al. [CDP14, CD18] supply the exact accounting of which transitions require K/UIP; Section 8.5 shows the model validates the unrestricted set, and Remark 8.17 imports their negative results (Counterexample 35; no type-former injectivity, after Hur [Hur10]). Coq’s `inversion` tactic performs fording at use sites [CT96]; Monin’s small inversions [Mon10] show the generated content is derivable with no axioms, which is what makes the reconstruction obligations of our inversion axioms benign; the Equations package [SM19] implements the whole stack inside Coq, with forced-argument analysis (after Brady, McBride and McKinna [BMM04]) as the UIP-avoidance mechanism that Lemma 5.17 repurposes as an optimisation.

Erasure and proof irrelevance. The program half follows Letouzey [Let03, Let04, Let08]: sort-directed, index-blind erasure, with stored indices kept; MetaCoq mechanises it with the `Subsingleton` criterion for case erasure [SBF⁺20]. Our `scase(;;)` is the stratified image of CIC’s singleton elimination [The26], extended from “parameters put aside” to *verbatim-forced* arguments — a strict subset of the forced arguments of [BMM04, GCST19], which also count *pattern-forced* positions occurring under constructors in an index and recoverable by injectivity; Remark 3.10 delimits the two notions and notes the pattern-forced generalisation as future work. That extensions of this kind under proof irrelevance are exactly UIP is the analysis of Gilbert, Cockx, Sozeau and Tabareau [GCST19], whose corrected criterion (non-forced arguments propositional, orthogonal index patterns, guardedness) delimits what can be *definitional* — our model needs only what is *true* in a proof-irrelevant semantics, which is why orthogonality is not required here (Definition 3.7). Werner’s guarded singleton reduction [Wer08] is the syntactic ancestor of Proposition 8.19; the proof-irrelevant model tradition [Wer97, MW03, LW11] is where our truth-value semantics lives, and Lee–Werner’s “singleton with a canonical element” is the denotational form of $\odot$, whose algebraic form is Atkey’s $!_0x = !_0y$ in quantitative type theory [Atk18] and whose operational form is Idris 2’s quantity-0 erasure [Bra21]. Brady, McBride and McKinna’s forcing, detagging and collapsing [BMM04] delete at run time what our translation asserts as equations; their warning about collapsing in open contexts is discussed after Proposition 8.19.

Hammers and type encodings. The immediate context is the CoqHammer translation [CK18, CK16], its proof-theoretically sound core [Cza18], and the companion paper [Cza26] whose architecture this paper extends; Section 1.4 and Remark 5.20 give the correspondence. On the Agda side, the closest relative is the agda2atp/Apia route of Bove, Dybjer and Sicard-Ramírez [BDSR12], which ships a first-order sub-language of Agda to untyped TPTP; there inductive types reach the ATPs through user-supplied postulates rather than a generated theory with a once-and-for-all soundness proof. SMTCoq [AFG⁺11, EMT⁺17] secures the Coq-solver connection by the complementary discipline: per-run certificates replayed inside Coq by a reflexive checker, so trust never rests on the encoding, at the price of coverage restricted to the checker’s theories — whereas the model-theoretic route proves the translation sound once, for every goal in the fragment, with reconstruction as the per-proof arbiter. For HOL hammers, the translation stack and its model-with-junk soundness proofs are due to Meng and Paulson [MP08] and Blanchette et al. [BBPS16]; their monotonicity analysis — a type’s guards are erasable iff no positive equality literal bounds it — identifies exhaustiveness axioms as never-erasable, which Proposition 8.7 sharpens for families. HOL systems never face type-level index unification: inductive families arrive pre-forded, either as predicates whose `.cases` rules are inversion-with-equations shipped as ordinary lemmas, or as subset types (`vec A n` as $\{l \mid \text{length } l = n\}$) — fording is the default in weaker logics, which is indirect evidence for its suitability as a translation target. Sledgehammer’s SMT route [BBP13] and the SMT datatypes theory [BST07, RB17] axiomatise closed-world initial algebras — exhaustiveness, selectors, demand-driven splitting — which is exactly what an indexed family is not; Lean-auto [QCB25] accordingly ground-monomorphises index instances before shipping `declare-datatypes`. F^{*} [SHK⁺16] is the closest production design: a single-sorted target with a fuel-instrumented typing predicate and per-family inversion axioms carrying index equations [The24a, Agu16]; its documented incidents — #1542 [Ase18] and the partially-applied-inductive #692 — and the Verus rank-axiom incident [The24b] are analysed against this paper’s design in Remark 8.10 and Example 5.15. Aguirre’s soundness analysis of the F^{*} encoding [Agu16] proves a fragment proof-theoretically and observes that HOL-style monotonicity-based guard removal does not apply to indexed inductive types — consistent with our finding that family guards are the load-bearing ones. Liquid Haskell [VSJ⁺14] performs per-occurrence inversion with measure equations in the *checker*, keeping the solver quantifier-free — the most consistent embodiment of occurrence-driven expansion, at the price of checker-driven unfolding depth; Dafny [Lei10] deliberately emits no exhaustiveness axiom and per-case definitional axioms with constructor-shaped triggers, matching the shape of Definition 5.4’s output; Why3’s `eliminate_algebraic` emits projection-form inversion sound only in a many-sorted target [FP13]; the

Dedukti/CoqInE route [BB12] turns inversion into rewriting, with index specialization by computation. The first-order metatheory of term algebras — completeness of injectivity/distinctness/domain-closure/acyclicity, the non-finite-axiomatisability of acyclicity, and the Π_1^1 -completeness with uninterpreted symbols — is Kovács, Robillard and Voronkov [KRV17]; the translation emits the finitely axiomatisable guarded fragment and omits acyclicity (Section 11.4).

Semantics of inductive families. Indexed inductive definitions in type theory are Dybjer’s [Dyb94]; the stage-based interpretation of Definition 6.3 is the classical iterated-inductive-definition construction [Acz77], in the indexed form used by set-theoretic models of CIC [Wer97, LW11]. The rank-restricted treatment of fixpoints is the semantic counterpart of guarded/sized recursion [Gim95, BFG⁺04], extended here with element ranks made well defined by Lemma 6.5(3).

11 Limitations and future work

The fragment CIC_{ix}^- was chosen so that every emitted axiom is provably sound while covering the indexed phenomena the fording architecture is about: constructor-constrained indices, forded case analysis, index-determined singletons, recursion over families. We delimit the boundary, separating presentational restrictions from semantic ones, and record the reconstruction obligations.

11.1 Dependent index telescopes

Index types in CIC_{ix}^- are closed (Definition 3.3): a family may not be indexed over a telescope in which a later index type mentions an earlier index ($\text{dvec} : \Pi n : \text{nat. fin } n \rightarrow \text{Set}$). The restriction is inherited from the shape of source-level equations: the forded branch equations $\vec{u} =_{\vec{\sigma}} \vec{t}_i$ are homogeneous, and with dependent telescopes the equation at position h would relate terms whose types agree only after the earlier equations are substituted — McBride’s motivation for heterogeneous equality [McB99], whose eliminator is K . The *target* side has no such problem: untyped pointwise equations state telescopic constraints directly, and the emitted axioms of Definition 5.13 would be unchanged in form. Lifting the restriction therefore requires work only at the source: either heterogeneous equality in the proof layer (with its K -strength absorbed by the model, which validates it), or cast-chained telescopic equations. We expect no semantic obstacle — the stage construction and Lemma 6.5 nowhere use closedness of index types — but the bookkeeping in Figures 2 to 5 is substantial and is left as future work.

11.2 Polymorphism, large indices, large elimination

CIC_{ix}^- is monomorphic: type parameters are ground instances (Section 3.7), so families indexed by *types* — $\text{eq Set } A \ B$, universe-indexed structures — are outside the fragment, consistently with the intended pipeline in which heuristic monomorphisation runs first. Large elimination remains excluded; fording removes its programming role in impossible branches (Example 3.16), the primitive **discr** covers its role in discrimination, and its remaining uses (types computed by recursion) would break the rigidity of set types (Lemma 3.12) on which the structural definitions of $\mathcal{G}$ and $\llbracket \cdot \rrbracket$ rest.

11.3 Well-founded recursion and **Acc**-style predicates

The hard constraint identified in the companion paper stands, and the indexed extension gives it a second face. **fix** captures structural recursion only; **Acc**-based recursion is excluded, because defining equations obtained by recursion through an erased proof are sound only with the erased propositional premises retained — the unconditional unfolding equation of Euclidean division at divisor 0 is *inconsistent* with $\forall n. n \not\approx \text{S}(n)$ -style facts. On the declaration side, **Acc**'s clause has its recursive premise under a quantifier, which Definition 3.3 excludes; semantically, its guardedness is not syntactic — Gilbert et al. observe that definitional proof irrelevance for **Acc** makes conversion undecidable [GCST19] — and our rank machinery, which is the semantic image of syntactic guardedness, does not apply. Both faces point to the same future work: premised unfolding equations with a semantics interpreting the defined symbol by an arbitrary total extension.

11.4 Acyclicity, induction, completeness

The translation emits no acyclicity axioms ($x \not\approx \text{S}(x)$ and its relatives) and no induction: acyclicity is not finitely axiomatisable over term algebras and induction is a schema [KRV17]; both are sound in $\mathfrak{M}$ (acyclicity by Corollary 4.4 and rank induction), so *adding* premise-selected instances would be sound, but the translation makes no completeness claim that would force them. The same holds for letting the prover generate induction instances internally, in the saturation-with-induction style [RV19, HHK⁺22]: the guarded first-order induction schema of a declared family is true in $\mathfrak{M}$ — the datatype-induction case of Lemma 7.6 is exactly its semantic form — so such prover-side extensions compose soundly with this translation. Completeness fails further along the axes inherited from the companion paper: the semantics is classical and proof-irrelevant (Remark 7.14); conversion under binders for lifted symbols is not reflected; goals needing induction on families (e.g. $\forall n (v:\text{vec}(n)). \widehat{\text{vmap}}_{\text{id}}(n, v) \approx v$) need the reconstruction back-end or user induction; and expansion variants unfold one constructor layer per occurrence, so properties requiring unbounded inversion depth need an

inductive lemma — the same trade-off Liquid Haskell documents for checker-driven unfolding [VSJ⁺14]. A completeness theorem for the *ground equational* fragment over purely structural families — if $\langle a =_\sigma b \rangle = 1$ for closed a, b of purely structural type then $\Delta(\mathbb{E}, G) \vdash_{\text{FOL}} \mathcal{C}(\mathcal{E}(a)) \approx \mathcal{C}(\mathcal{E}(b))$ — is plausibly within reach, but it is not a corollary of what is proved here, and we state the missing piece exactly rather than gesture at “standard rewriting arguments”. Remark 5.8 makes the emitted equations a non-overlapping rewrite system (orthogonal in the absence of recursive closures) and Proposition 7.12 makes $\mathfrak{M}$ standard on the relevant types; what is missing is a simulation-and-standardisation development relating that rewrite system to $\lambda_\square$ conversion on compiled images. The hypothesis $\langle a =_\sigma b \rangle = 1$ says the *erasures* are $\lambda_\square$ -convertible; this paper proves no normalisation or standardisation theorem for $\lambda_\square$ (none is needed anywhere else), and conversion steps under the binders of lifted symbols are precisely what the emitted equations do not reflect. For ground terms of structural type the under-binder steps should be avoidable along a weak-head standard reduction, so we expect the theorem to hold; proving that simulation is the actual content of the folklore argument, and we leave it as future work.

11.5 Functional extensionality

$\mathfrak{M}$ is intensional (Remark 8.20): premise sets containing funext-derived lemmas step outside Theorem 7.9. Repairing this requires a coarser domain (quotient by observational equivalence), which would disturb Corollary 4.4 and the compilation argument; nothing indexed-specific changes the analysis, but Remark 8.10 raises its stakes — the F^* incident shows extensionality axioms and index-blind encodings interacting is how translations of this kind have actually failed.

11.6 Reconstruction

Soundness here is model-theoretic; the implemented pipeline’s final arbiter is proof reconstruction, and the paper’s results localise its obligations. Uses of inversion axioms and expanded guards replay by *inversion*-style reasoning, derivable without axioms [CT96, Mon10] — K-free, since the motive never depends on the generated equations. Uses of the unconditional singleton-collapse equations, of deletion, and of the extra permissiveness of Definition 3.7 beyond CIC’s parameter mechanism (Remark 3.10) may need UIP-class tactics at types without decidable equality; Proposition 8.19 is the premise-carrying alternative that eliminates this debt. A proof-theoretic soundness theorem in the style of [Cza18] — transforming first-order derivations from $\Delta(\mathbb{E}, G)$ into CIC_{ix}^- proof terms modulo excluded middle, proof irrelevance and UIP — would turn this localisation into an algorithm and is the natural next step, with the lifted-symbol substitution stability of the

companion paper’s discussion as the known hard part.

11.7 Other restrictions

Mutual and nested inductive families, deep and mutual recursion, and coinductive families are excluded syntactically, as in the companion paper; each constructor of a mutual block fords independently, so we expect the extension to be bookkeeping, but coinductive families — whose case-analysis content is a greatest-fixpoint inversion and whose equality is bisimilarity — are a genuine further step. Two further restrictions of the declaration format (Definition 3.3) deserve explicit listing. First, recursive occurrences must be direct: argument types of the form $\sigma \rightarrow I(\vec{v})$ — function-space recursive arguments, infinitary branching, W-types — are excluded; only first-order recursive positions $I(\vec{v})$ are admitted. Second, all propositional premises of a constructor come after all informative arguments; a CIC constructor in which the *type* of a later informative argument mentions an earlier propositional argument (a proof subterm inside an index term of a subsequent argument type) has no direct image in the format. Under proof irrelevance the reordering is semantically harmless — the premise proofs are erased and the index terms lose their proof subterms either way (Remark 4.6) — but it is a genuine syntactic restriction of the fragment. Enumeration-, subset- and singleton-shaped families receive no special-cased treatment in the formal development; the classification exists in implementations as a choice of expansion policy, and Theorem 8.3 is deliberately policy-agnostic so that any such classification is covered.

References

- [Acz77] Peter Aczel. An introduction to inductive definitions. In Jon Barwise, editor, *Handbook of Mathematical Logic*, pages 739–782. North-Holland, 1977.
- [AFG⁺11] Michaël Armand, Germain Faure, Benjamin Grégoire, Chantal Keller, Laurent Théry, and Benjamin Werner. A modular integration of SAT/SMT solvers to Coq through proof witnesses. In *Certified Programs and Proofs (CPP 2011)*, volume 7086 of *LNCS*, pages 135–150. Springer, 2011.
- [Agu16] Alejandro Aguirre. Towards a provably correct encoding from F* to SMT. MPRI internship report, Inria Paris, 2016.
- [Ase18] Aseem Rastogi and the F* Development Team. Unsoundness in the SMT encoding of functional extensionality. FStarLang/FStar issue #1542, <https://github.com/FStarLang/FStar/issues/1542>, 2018.

- [Atk18] Robert Atkey. Syntax and semantics of quantitative type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2018)*, pages 56–65. ACM, 2018.
- [BB12] Mathieu Boespflug and Guillaume Burel. CoqInE: Translating the calculus of inductive constructions into the $\lambda\Pi$ -calculus modulo. In *Proceedings of the Second International Workshop on Proof Exchange for Theorem Proving (PxTP 2012)*, pages 44–50, 2012.
- [BBP13] Jasmin Christian Blanchette, Sascha Böhme, and Lawrence C. Paulson. Extending Sledgehammer with SMT solvers. *Journal of Automated Reasoning*, 51(1):109–128, 2013.
- [BBPS16] Jasmin Christian Blanchette, Sascha Böhme, Andrei Popescu, and Nicholas Smallbone. Encoding monomorphic and polymorphic types. *Logical Methods in Computer Science*, 12(4), 2016.
- [BDSR12] Ana Bove, Peter Dybjer, and Andrés Sicard-Ramírez. Combining interactive and automatic reasoning in first order theories of functional programs. In *Foundations of Software Science and Computational Structures (FoSSaCS 2012)*, volume 7213 of *LNCs*, pages 104–118. Springer, 2012.
- [BFG⁺04] Gilles Barthe, Maria João Frade, Eduardo Giménez, Luís Pinto, and Tarmo Uustalu. Type-based termination of recursive definitions. *Mathematical Structures in Computer Science*, 14(1):97–141, 2004.
- [BKPU16] Jasmin Christian Blanchette, Cezary Kaliszyk, Lawrence C. Paulson, and Josef Urban. Hammering towards QED. *Journal of Formalized Reasoning*, 9(1):101–148, 2016.
- [BMM04] Edwin Brady, Conor McBride, and James McKinna. Inductive families need not store their indices. In *Types for Proofs and Programs (TYPES 2003)*, volume 3085 of *LNCs*, pages 115–129. Springer, 2004.
- [BN98] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- [Bra21] Edwin Brady. Idris 2: Quantitative type theory in practice. In *35th European Conference on Object-Oriented Programming (ECOOP 2021)*, volume 194 of *LIPICs*, pages 9:1–9:26. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021.

- [BST07] Clark Barrett, Igor Shikanian, and Cesare Tinelli. An abstract decision procedure for a theory of inductive data types. *Journal on Satisfiability, Boolean Modeling and Computation*, 3(1–2):21–46, 2007.
- [CD18] Jesper Cockx and Dominique Devriese. Proof-relevant unification: Dependent pattern matching with only the axioms of your type theory. *Journal of Functional Programming*, 28:e12, 2018.
- [CDP14] Jesper Cockx, Dominique Devriese, and Frank Piessens. Pattern matching without K. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*, pages 257–268. ACM, 2014.
- [CK16] Łukasz Czajka and Cezary Kaliszyk. Goal translation for a hammer for Coq (extended abstract). In *Proceedings First International Workshop on Hammers for Type Theories (HaTT 2016)*, volume 210 of *EPTCS*, pages 13–20, 2016.
- [CK18] Łukasz Czajka and Cezary Kaliszyk. Hammer for Coq: Automation for dependent type theory. *Journal of Automated Reasoning*, 61(1–4):423–453, 2018.
- [CK26] Łukasz Czajka and Cezary Kaliszyk. CoqHammer: Automated reasoning for Rocq (Coq). <https://coqhammer.github.io/>, 2026.
- [Coq92] Thierry Coquand. Pattern matching with dependent types. In *Proceedings of the Workshop on Types for Proofs and Programs*, pages 66–79, Båstad, Sweden, 1992.
- [CT96] Cristina Cornes and Delphine Terrasse. Automating inversion of inductive predicates in Coq. In *Types for Proofs and Programs (TYPES 1995)*, volume 1158 of *LNCS*, pages 85–104. Springer, 1996.
- [Cza18] Łukasz Czajka. A shallow embedding of pure type systems into first-order logic. In *22nd International Conference on Types for Proofs and Programs (TYPES 2016)*, volume 97 of *LIPIcs*, pages 9:1–9:39. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018.
- [Cza26] Łukasz Czajka. Program extraction and specification extraction: A sound shallow translation of dependent type theory into first-order logic. Companion manuscript, submitted for publication; provided as supplementary material with the present submission, 2026.

- [Dyb94] Peter Dybjer. Inductive families. *Formal Aspects of Computing*, 6(4):440–465, 1994.
- [EMT⁺17] Burak Ekici, Alain Mebsout, Cesare Tinelli, Chantal Keller, Guy Katz, Andrew Reynolds, and Clark Barrett. SMTCoq: A plug-in for integrating SMT solvers into Coq. In *Computer Aided Verification (CAV 2017), Part II*, volume 10427 of *LNCS*, pages 126–133. Springer, 2017.
- [FP13] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 — where programs meet provers. In *Programming Languages and Systems (ESOP 2013)*, volume 7792 of *LNCS*, pages 125–128. Springer, 2013.
- [GCST19] Gaëtan Gilbert, Jesper Cockx, Matthieu Sozeau, and Nicolas Tabareau. Definitional proof-irrelevance without K. *Proceedings of the ACM on Programming Languages*, 3(POPL):3:1–3:28, 2019.
- [Gim95] Eduardo Giménez. Codifying guarded definitions with recursive schemes. In *Types for Proofs and Programs (TYPES 1994)*, volume 996 of *LNCS*, pages 39–59. Springer, 1995.
- [GMM06] Healfdene Goguen, Conor McBride, and James McKinna. Eliminating dependent pattern matching. In *Algebra, Meaning, and Computation: Essays dedicated to Joseph A. Goguen*, volume 4060 of *LNCS*, pages 521–540. Springer, 2006.
- [HHK⁺22] Márton Hajdú, Petra Hozzová, Laura Kovács, Giles Reger, and Andrei Voronkov. Getting saturated with induction. In *Principles of Systems Design: Essays Dedicated to Thomas A. Henzinger on the Occasion of His 60th Birthday*, volume 13660 of *LNCS*, pages 306–322. Springer, 2022.
- [Hur10] Chung-Kil Hur. Agda with excluded middle is inconsistent. Message to the Agda mailing list, <https://lists.chalmers.se/pipermail/agda/2010/001543.html>, January 2010.
- [Klo80] Jan Willem Klop. *Combinatory Reduction Systems*. PhD thesis, Utrecht University, 1980. Mathematical Centre Tracts 127.
- [KRV17] Laura Kovács, Simon Robillard, and Andrei Voronkov. Coming to terms with quantified reasoning. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*, pages 260–270. ACM, 2017.
- [Lei10] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intel-*

- ligence, and Reasoning (LPAR-16)*, volume 6355 of *LNCS*, pages 348–370. Springer, 2010.
- [Let03] Pierre Letouzey. A new extraction for Coq. In *Types for Proofs and Programs (TYPES 2002)*, volume 2646 of *LNCS*, pages 200–219. Springer, 2003.
- [Let04] Pierre Letouzey. *Programmation fonctionnelle certifiée – L’extraction de programmes dans l’assistant Coq*. PhD thesis, Université Paris-Sud, 2004. HAL tel-00150912.
- [Let08] Pierre Letouzey. Extraction in Coq: An overview. In *Logic and Theory of Algorithms (CiE 2008)*, volume 5028 of *LNCS*, pages 359–369. Springer, 2008.
- [LW11] Gyesik Lee and Benjamin Werner. Proof-irrelevant model of CC with predicative induction and judgmental equality. *Logical Methods in Computer Science*, 7(4:5), 2011.
- [McB99] Conor McBride. *Dependently Typed Functional Programs and their Proofs*. PhD thesis, University of Edinburgh, 1999. Technical report ECS-LFCS-00-419.
- [McB14] Conor McBride. How to keep your neighbours in order. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*, pages 297–309. ACM, 2014.
- [Mon10] Jean-François Monin. Proof trick: Small inversions. Second Coq Workshop, Edinburgh; HAL inria-00489412, 2010.
- [MP08] Jia Meng and Lawrence C. Paulson. Translating higher-order clauses to first-order clauses. *Journal of Automated Reasoning*, 40(1):35–60, 2008.
- [MW03] Alexandre Miquel and Benjamin Werner. The not so simple proof-irrelevant model of CC. In *Types for Proofs and Programs (TYPES 2002)*, volume 2646 of *LNCS*, pages 240–258. Springer, 2003.
- [QCBA25] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. Lean-auto: An interface between Lean 4 and automated theorem provers. In *Computer Aided Verification (CAV 2025)*. Springer, 2025. arXiv:2505.14929.
- [RB17] Andrew Reynolds and Jasmin Christian Blanchette. A decision procedure for (co)datatypes in SMT solvers. *Journal of Automated Reasoning*, 58(3):341–362, 2017.

- [RV19] Giles Reger and Andrei Voronkov. Induction in saturation-based proof search. In *Automated Deduction – CADE 27*, volume 11716 of *LNCS*, pages 477–494. Springer, 2019.
- [SBF⁺20] Matthieu Sozeau, Simon Boulier, Yannick Forster, Nicolas Tabareau, and Théo Winterhalter. Coq Coq correct! Verification of type checking and erasure for Coq, in Coq. *Proceedings of the ACM on Programming Languages*, 4(POPL):8:1–8:28, 2020.
- [SHK⁺16] Nikhil Swamy, Cătălin Hrițcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean-Karim Zinzindohoué, and Santiago Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2016)*, pages 256–270. ACM, 2016.
- [Sho67] Joseph R. Shoenfield. *Mathematical Logic*. Addison-Wesley, 1967.
- [SM19] Matthieu Sozeau and Cyprien Mangin. Equations reloaded: High-level dependently-typed functional programming and proving in Coq. *Proceedings of the ACM on Programming Languages*, 3(ICFP):86:1–86:29, 2019.
- [Tak95] Masako Takahashi. Parallel reductions in λ -calculus. *Information and Computation*, 118(1):120–127, 1995.
- [The24a] The F* Development Team. Understanding how F* uses Z3. The F* book, https://fstar-lang.org/tutorial/book/under_the_hood/uth_smt.html, 2024.
- [The24b] The Verus Development Team. Unsoundness from box/unbox axioms for empty structs and height axioms. verus-lang/verus issue #1366, <https://github.com/verus-lang/verus/issues/1366>, 2024.
- [The26] The Rocq Development Team. The Rocq prover reference manual. <https://rocq-prover.org/doc/>, 2026.
- [VSJ⁺14] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*, pages 269–282. ACM, 2014.
- [Wer97] Benjamin Werner. Sets in types, types in sets. In *Theoretical Aspects of Computer Software (TACS 1997)*, volume 1281 of *LNCS*, pages 530–546. Springer, 1997.

- [Wer08] Benjamin Werner. On the strength of proof-irrelevant type theories. *Logical Methods in Computer Science*, 4(3:13), 2008.
- [Zha23] Tesla Zhang. Two tricks to trivialize higher-indexed families. arXiv:2309.14187, 2023.

A Confluence of $\lambda_{\square}$

We prove Theorem 4.3 by the Tait–Martin-Löf method in Takahashi’s formulation [Tak95], extended to the ι , μ and δ rules. The proof is that of the companion paper, reproduced for self-containedness; it depends on the constructor alphabet only through arities, so the indexed extension changes nothing. Recall that the δ -rule unfolds a constant to the erasure of its body, $c \rightarrow_{\square} \mathcal{E}(t_c)$, and that bodies mention only earlier constants; no property of this ordering is needed beyond the fact that each constant has a *fixed* unfolding.

Definition A.1 (Parallel reduction). The relation $\Rightarrow$ on $\lambda_{\square}$ terms is defined inductively:

1. $x \Rightarrow x$, $\square \Rightarrow \square$, $c \Rightarrow c$;
2. $c \Rightarrow \mathcal{E}(t_c)$ for a defined constant c ;
3. $\lambda x.e \Rightarrow \lambda x.e'$ if $e \Rightarrow e'$;
4. $e_1 e_2 \Rightarrow e'_1 e'_2$ if $e_i \Rightarrow e'_i$;
5. $c(\vec{e}) \Rightarrow c(\vec{e}')$ if $e_i \Rightarrow e'_i$ componentwise;
6. $\text{case}(e; \vec{b}r) \Rightarrow \text{case}(e'; \vec{b}r')$ if $e \Rightarrow e'$ and branch bodies reduce componentwise;
7. $\text{fix } f.e \Rightarrow \text{fix } f.e'$ if $e \Rightarrow e'$;
8. $(\lambda x.e) a \Rightarrow e'[a'/x]$ if $e \Rightarrow e'$ and $a \Rightarrow a'$;
9. $\text{case}(c_i(\vec{e}); \{c_j(\vec{x}_j) \Rightarrow b_j\}_j) \Rightarrow b'_i[\vec{e}'/\vec{x}_i]$ if $e_l \Rightarrow e'_l$ and $b_i \Rightarrow b'_i$;
10. $\text{fix } f.e \Rightarrow e'[\text{fix } f.e'/f]$ if $e \Rightarrow e'$.

Lemma A.2. (i) $e \Rightarrow e$ for all e . (ii) $\rightarrow_{\square} \subseteq \Rightarrow \subseteq \rightarrow_{\square}^*$. (iii) If $e \Rightarrow e'$ and $u \Rightarrow u'$ then $e[u/x] \Rightarrow e'[u'/x]$.

Proof. (i) Induction on e using clauses (1) and (3)–(7). (ii) For the first inclusion, a one-step reduction is a root rule applied inside a context: the corresponding root clause (2), (8), (9) or (10) with reflexive subderivations, propagated by the congruence clauses. For the second, induction on $e \Rightarrow e'$: each clause is realised by finitely many $\rightarrow_{\square}$ -steps. (iii) Induction on the derivation of $e \Rightarrow e'$. The variable clause $x \Rightarrow x$ becomes $u \Rightarrow u'$; clause

(2) is unaffected because $\mathcal{E}(t_c)$ is closed; the congruence clauses follow from the induction hypothesis; the redex clauses use the standard substitution identities, e.g. $(e'[a'/y])[u'/x] = (e'[u'/x])[a'[u'/x]/y]$ for $y \notin \text{FV}(u') \cup \{x\}$. $\square$

Definition A.3 (Complete development). The complete development e^* is defined by recursion on e :

$$\begin{aligned} x^* &= x, & \square^* &= \square, & c^* &= \mathcal{E}(t_c), \\ (\lambda x.e)^* &= \lambda x.e^*, & \mathbf{c}(\vec{e})^* &= \mathbf{c}(\vec{e}^*), & (\text{fix } f.e)^* &= e^*[\text{fix } f.e^*/f], \\ ((\lambda x.e) a)^* &= e^*[a^*/x], & (e_1 e_2)^* &= e_1^* e_2^* \quad (e_1 \text{ not a } \lambda), \\ \text{case}(\mathbf{c}_i(\vec{e}); \vec{br})^* &= b_i^*[\vec{e}^*/\vec{x}_i], \\ \text{case}(e; \vec{br})^* &= \text{case}(e^*; \vec{br}^*) \quad (e \text{ not a constructor form}). \end{aligned}$$

(For a constant, the development unfolds once and does not recurse into the unfolding; for a fix, the root μ -redex is contracted once, its residuals in the contractum being new redexes that are not developed.)

Lemma A.4 (Triangle property). *If $e \Rightarrow e'$ then $e' \Rightarrow e^*$.*

Proof. Induction on the derivation of $e \Rightarrow e'$, with a case analysis on the last clause used.

- Clause (1): for variables and $\square$ immediate; for $c \Rightarrow c$ we need $c \Rightarrow c^* = \mathcal{E}(t_c)$, which is clause (2).
- Clause (2): $e' = \mathcal{E}(t_c) = c^*$, and $e' \Rightarrow e'$ by Lemma A.2(i).
- Clauses (3), (5), (7), and clause (6) with non-constructor scrutinee: by the induction hypothesis componentwise. Two subtleties: for $\text{fix } f.e \Rightarrow \text{fix } f.e'$ (clause (7)) the target is $(\text{fix } f.e)^* = e^*[\text{fix } f.e^*/f]$, reached from $\text{fix } f.e'$ by clause (10) with $e' \Rightarrow e^*$ (induction hypothesis). For $\text{case}(e; \vec{br})$ with e not a constructor form, the scrutinee's reduct e'' may or may not be a constructor form, but the development $\text{case}(e^*; \vec{br}^*)$ is reached by clause (6) either way.
- Clause (6) with constructor-form scrutinee — the parallel step is a congruence but the development fires the ι -redex. Here $e = \text{case}(\mathbf{c}_i(\vec{e}); \{\mathbf{c}_j(\vec{x}_j) \Rightarrow b_j\}_j)$ and $e^* = b_i^*[\vec{e}^*/\vec{x}_i]$ by Definition A.3. The step gives $e' = \text{case}(e''; \{\mathbf{c}_j(\vec{x}_j) \Rightarrow b'_j\}_j)$ with $\mathbf{c}_i(\vec{e}) \Rightarrow e''$ and $b_j \Rightarrow b'_j$; the scrutinee derivation can only be by clause (5) — a constructor form is neither a variable, $\square$, a constant, nor an application, so no other clause matches its shape — hence $e'' = \mathbf{c}_i(\vec{e}'')$ with $e_l \Rightarrow e'_l$ componentwise. The induction hypotheses give $e'_l \Rightarrow e_l^*$ and $b'_l \Rightarrow b_l^*$, so clause (9) applied to $e' = \text{case}(\mathbf{c}_i(\vec{e}''); \{\mathbf{c}_j(\vec{x}_j) \Rightarrow b'_j\}_j)$ yields $e' \Rightarrow b_i^*[\vec{e}''^*/\vec{x}_i] = e^*$. (This is the redex-created-by-development configuration the Takahashi

method is designed for; the symmetric configuration — step fires the redex, development likewise — is clause (9) below.)

- Clause (4) with $e_1 = \lambda x.b$: then $e_1 \Rightarrow e'_1$ is by clause (3), $e'_1 = \lambda x.b'$, and $e^* = b^*[a^*/x]$; from $e' = (\lambda x.b') a'$ apply clause (8) with $b' \Rightarrow b^*$, $a' \Rightarrow a^*$ (induction hypotheses).
- Clause (4) with e_1 not a λ : $e^* = e_1^* e_2^*$; apply clause (4) with the induction hypotheses.
- Clause (8): $e = (\lambda x.b) a$, $e' = b'[a'/x]$; by induction hypotheses $b' \Rightarrow b^*$ and $a' \Rightarrow a^*$, so Lemma A.2(iii) gives $e' \Rightarrow b^*[a^*/x] = e^*$.
- Clause (9): as clause (8), with the componentwise induction hypotheses for the constructor arguments and the branch, and Lemma A.2(iii) extended to simultaneous substitutions. The scrutinee $c_i(\vec{e})$ can only have been reduced componentwise (clause (5)), so the case analysis in Definition A.3 is stable.
- Clause (10): $e = \text{fix } f.b$, $e' = b'[\text{fix } f.b'/f]$; by the induction hypothesis $b' \Rightarrow b^*$, whence $\text{fix } f.b' \Rightarrow \text{fix } f.b^*$ by clause (7), and Lemma A.2(iii) gives $e' \Rightarrow b^*[\text{fix } f.b^*/f] = e^*$.

□

Proof of Theorem 4.3. By Lemma A.4, $\Rightarrow$ has the diamond property: if $e \Rightarrow e_1$ and $e \Rightarrow e_2$ then $e_1 \Rightarrow e^*$ and $e_2 \Rightarrow e^*$. A relation with the diamond property has confluent reflexive-transitive closure (tiling), and by Lemma A.2(ii) the closure of $\Rightarrow$ is $\rightarrow_\square^*$. The characterisation of $=_\square$ by common reducts follows by induction on the conversion zig-zag (Church–Rosser). □